



1. Introduction

Data Structure and Algorithms

Dr. Udaya Raj Dhungana

Assist. Professor

Pokhara University, Nepal

Guest Faculty (2018-2022)

Hochschule Darmstadt University of Applied Sciences, Germany

E-mail: udaya@pu.edu.np and udayas.epost@gmail.com

Overview

Unit 1: Introduction (5 hrs)

1. Philosophy of Data Structures

- 1.1. Need of Data Structures
- 1.2. Characteristics and Types

2. Abstract Data Type (ADT) and Data Structures

3. Algorithm Design Techniques

- 3.1. Divide and Conquer
- 3.2. Greedy Algorithms
- 3.3. Backtracking

4. Algorithm Analysis:

- 4.1. Best, Worst and Average Case Analysis
- 4.2. Rate of Growth
- 4.3. Asymptotic Notations- Big Oh, Big Omega and Big Theta

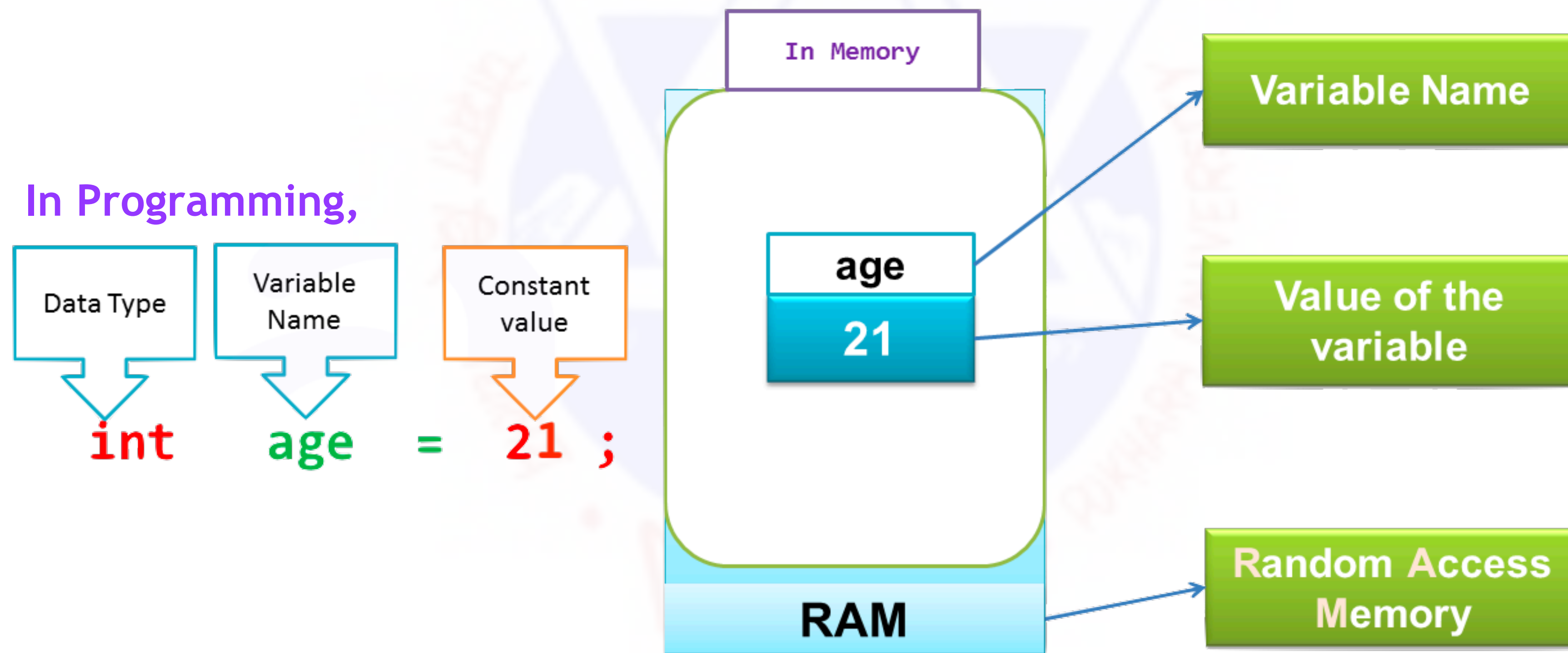


Review on

**ARRAY, STRUCTURE, UNION,
CLASS AND POINTER**

Variable

- **Variable:** A variable is a name given to a memory location which can hold a single value.



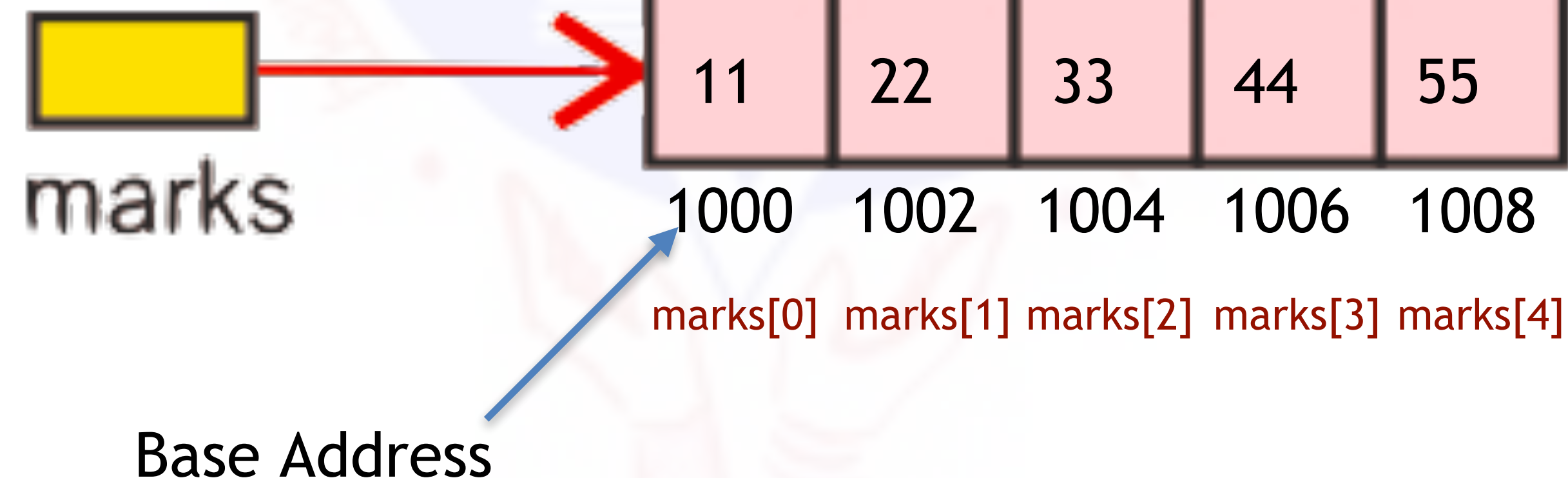
Array

- **Array:** An array is a collection of variables of same data types that are referred by a common name.

Int marks[5] = {11, 22, 33, 44, 55};

marks[0] = 11
marks[1] = 22
marks[2] = 33
marks[3] = 44
marks[4] = 55

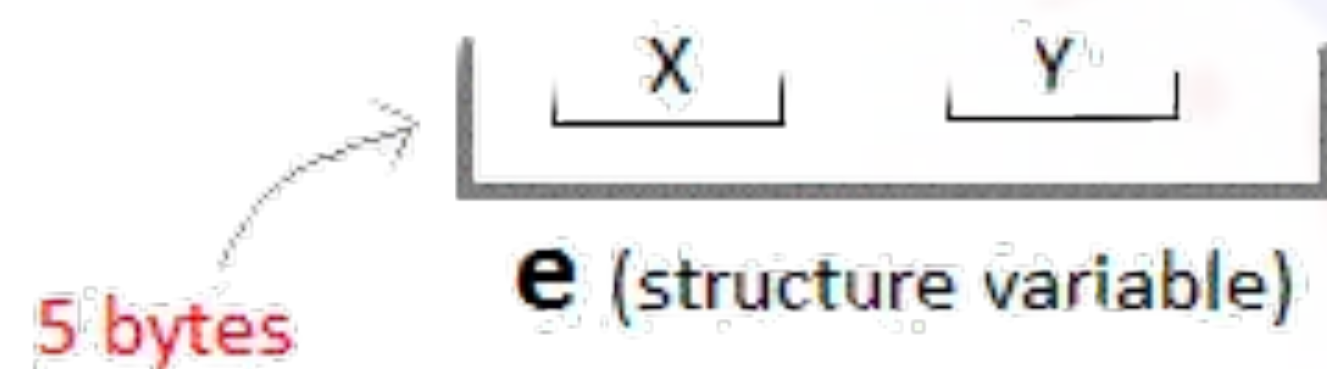
(Name of an array is a pointer to its first element)



Structure and Union

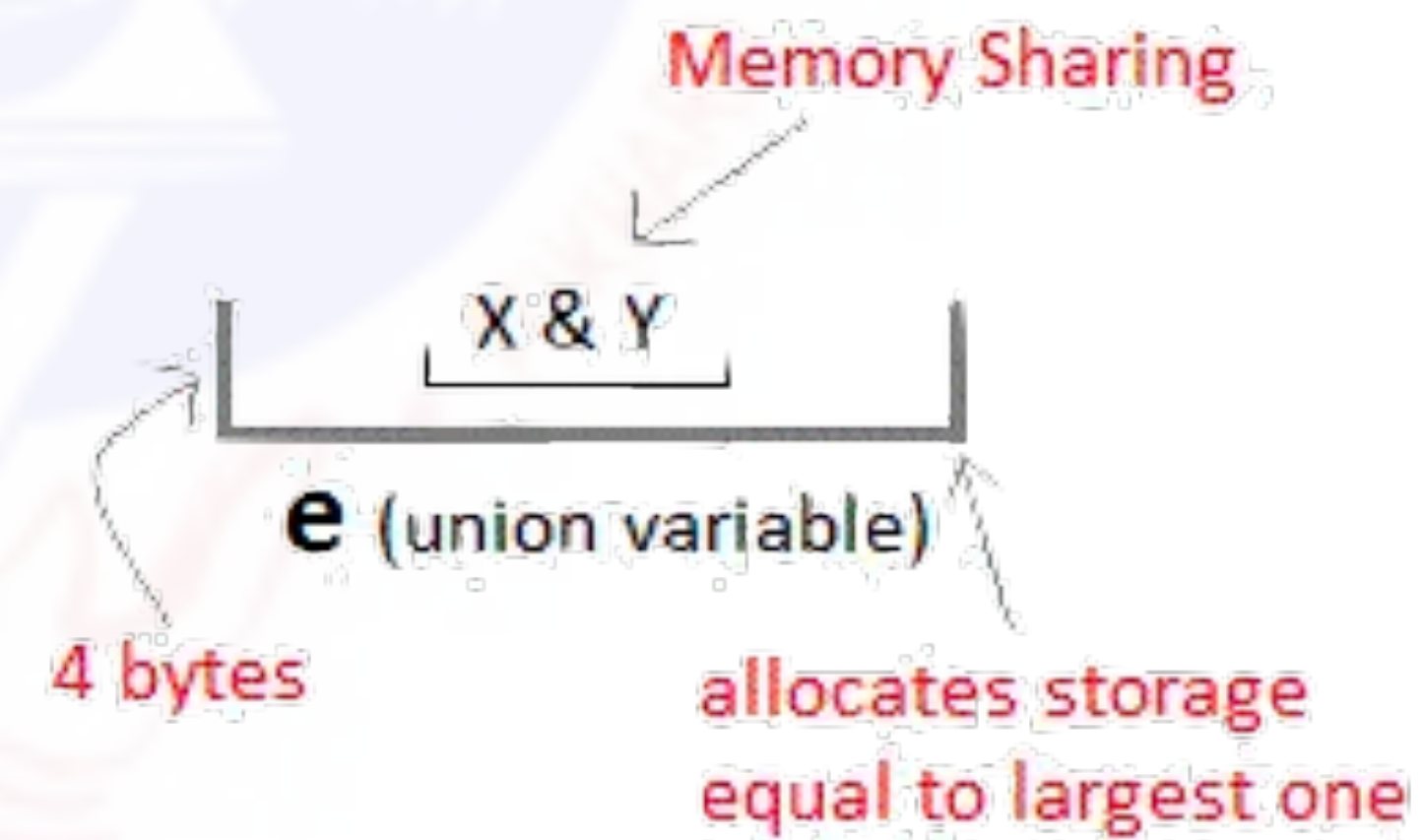
Structure

```
struct Emp  
{  
  char X; // size 1 byte  
  float Y; // size 4 byte  
} e;
```



Unions

```
union Emp  
{  
  char X;  
  float Y;  
} e;
```



Object Oriented Programming (C++)

- **Procedure-oriented programming**
 - Program is divided into procedures (functions)
 - Focus is on procedure (not on data)
- **Object-oriented programming**
 - Everything is treated as objects and classes
 - Focus on data privacy
- **Classes**
 - A class is a blueprint for an object (Generalization of individuals)
 - A class is a user defined data type that contains its data members and member functions.
- **Objects**
 - An object is an instance of a class
 - Example: “Ram” is an instance of class “Student”.



Write a program to add two integer numbers using class in C++.

Object Oriented Programming (C++)

```
#include <iostream>
using namespace std;

class addition {
    int a, b, result;
public:
    void get_ab(int x, int y);
    void sum();
    void display();
};

void addition :: get_ab(int x, int y) {
    a = x;
    b = y;
}

void addition :: sum() {
    result = a+b;
}

void addition :: display() {
    cout<<"\nSum = "<<result;
}
```

```
int main()
{
    addition obj1, obj2;

    obj1.get_ab(4,5);
    obj1.sum();
    obj1.display();

    obj2.get_ab(40,55);
    obj2.sum();
    obj2.display();

    return 0;
}
```

Output:

```
Sum = 9
Sum = 95
```



Structure and Class (C++)

```
struct array {  
    private:  
        int arr[5];  
        int indx;  
    public:  
        array() { indx = -1; }  
        void insert(int item);  
        int del();  
        int search(int item);  
};
```

```
class array {  
    int arr[5];  
    int indx;  
    public:  
        array() { indx = -1; }  
        void insert(int item);  
        int del();  
        int search(int item);  
};
```

In C++, struct and class can be used interchangeably. The only difference is that (if you didn't mention the access specifier) the members in struct are public by default and the members in class are private by default.



The following program illustrates how pointer and structure or class can be used to define a simple list data structure.

Pointer and Class/Structure

```

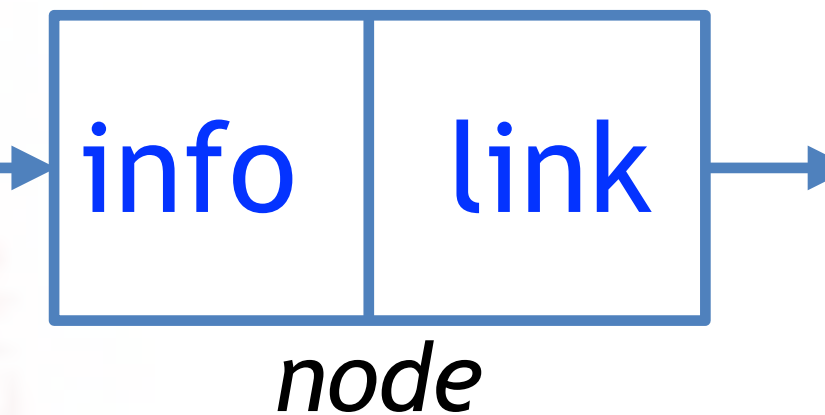
1  #include<iostream>
2  #include<conio.h>
3  #include<stdlib.h>
4
5  class node
6  {
7      int info;
8      node *link;
9  };
10
11 int main()
12 {
13     system("cls");
14
15     node *first;
16
17     first = new node;
18     first->info=10;
19
20     node *second;
21     second = new node;
22     second->info = 20;
23     first->link=second;
24

```

```

25     node *third;
26     third = new node;
27     third->info=30;
28     second->link=third;
29     third->link=NULL;
30
31     node *temp;
32     temp=first;
33     while(temp!=NULL)
34     {
35         printf("%d", temp->info);
36         temp=temp->link;
37     }
38     delete temp, first, second, third;
39     getch();
40     return 0;
41 }

```



Introduction to

DATA STRUCTURES AND ALGORITHMS

Philosophy of Data Structures

- **Problems:** Let us consider some problems:
 - How can we find a path from one location to another in a map?
 - How can we reverse a string?
 - How can we manage a queue of processes that are waiting to be executed by CPU?
 - Can we connect all of our telephone customers with less than 1,000 miles of cable?
- **Need:** To answer the questions like these:
 - it is not enough to have the necessary information.
 - We must organize/store that information in a way that allows us to find the answers with minimum time and space requirements.



Philosophy of Data Structures

- The **primary purpose** of most **algorithms** is
 - not only to perform calculations
 - but also to store and retrieve the information — **usually as fast as possible using minimum of space.**
- For a given problem, there are often many algorithms to solve the problem.
 - How do we choose better algorithms among them?
- At the heart of algorithm design are two goals:
 - To design an algorithm that is easy to understand, code, and debug.
 - To design an algorithm that makes efficient use of the computer's resources.



Philosophy of Data Structures

- This course helps you to understand
 - How to organize/structure information to support efficient processing. (**Data Structures**)
- How do we measure efficiency?
 - Asymptotic analysis: allows you to see how each algorithm compares to other algorithms for solving the same problem in terms of its efficiency. (**Algorithm Analysis**)
- In addition you will learn:
 - **Algorithms Design Techniques**



Philosophy of Data Structures

- **Example:** Let us consider you want to develop an application to find a path from your home to your school.

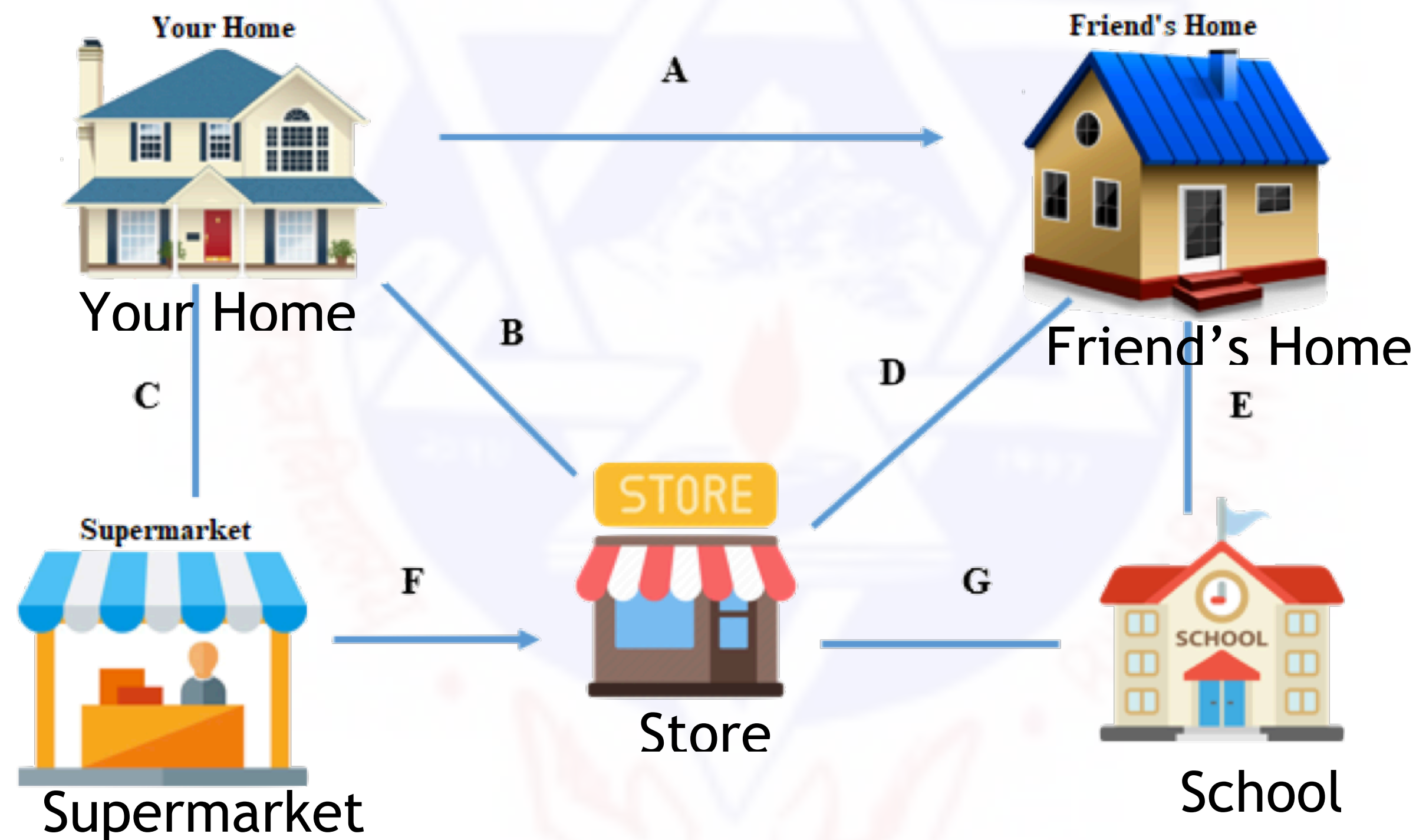
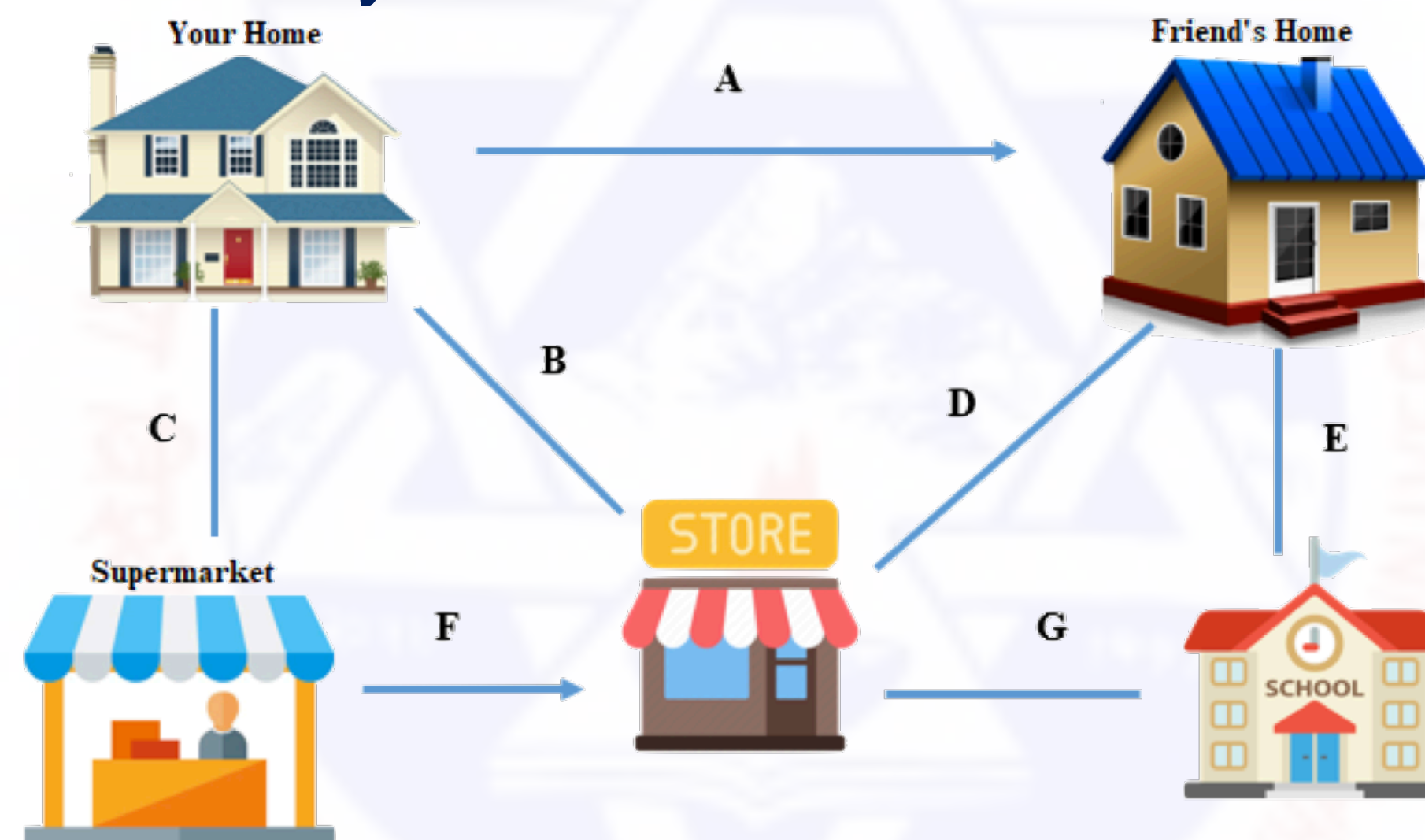


Figure 1: A small section of the map containing your home, your friend's home, a supermarket, a store and school

Philosophy of Data Structures

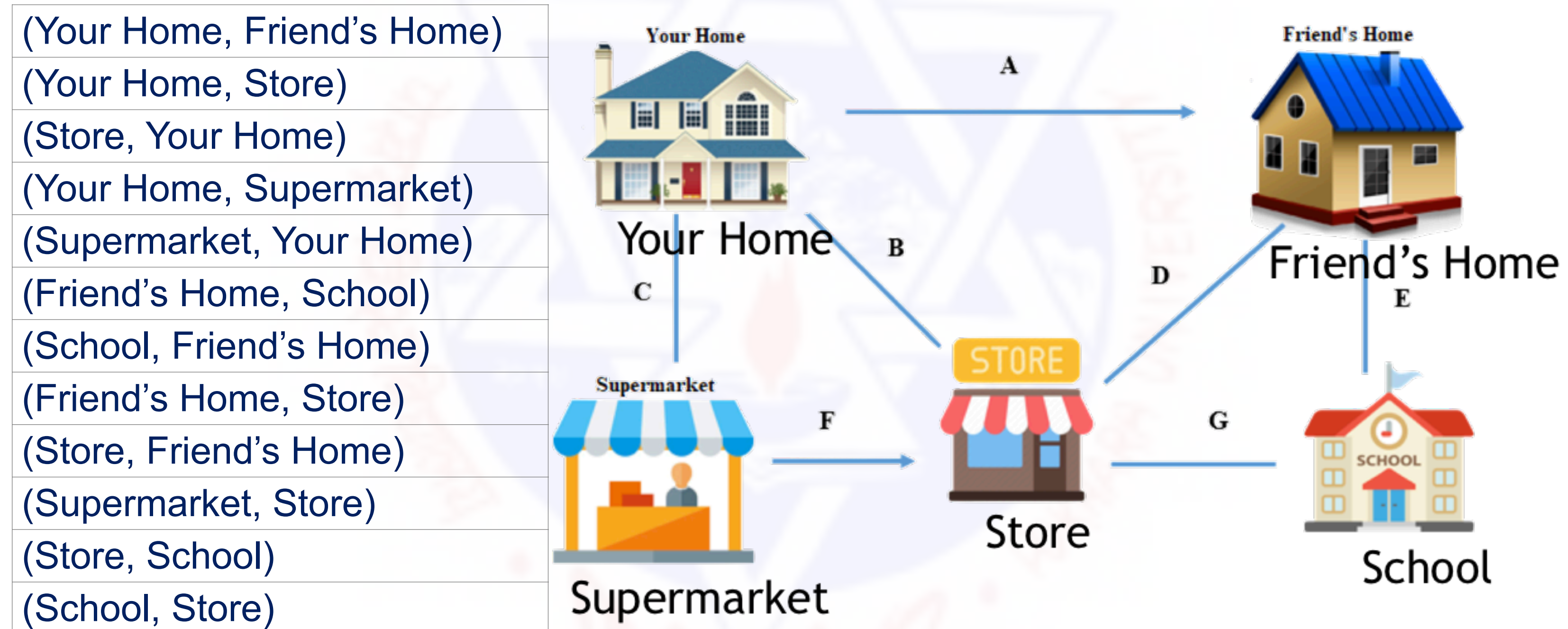
- Example: Let us consider you want to develop an application to find a path from your home to your school.



- The streets are named as A, B, C, D, E, F and G.
- A street line with arrow indicates one way street
- A street line without arrow indicates a two way street.
- How to store the information of these places and their connections?

Philosophy of Data Structures

- **Method 1:** One way of organizing the information (**Data Structure 1**).



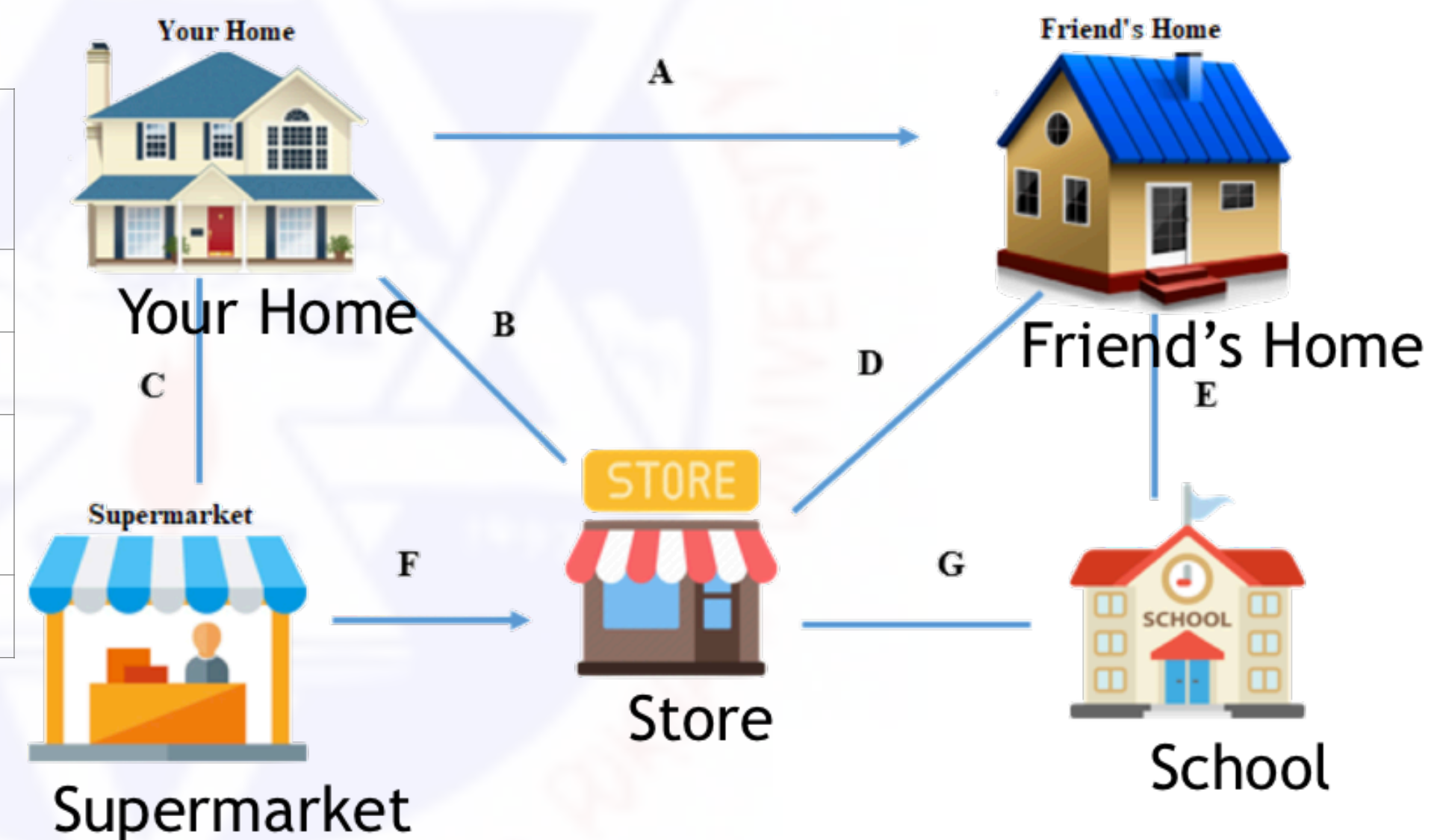
(a) **Data Structure 1** which looks like
a list or an array



Philosophy of Data Structures

- **Method 2:** Second way of organizing the information (**Data Structure 2**).

Your Home	(Friend's Home, Store, Supermarket)
Friend's Home	(Store, School)
Supermarket	(Your Home, Store)
Store	(Your Home, Friend's Home, School)
School	(Friend's Home, Store)



(b) **Data Structure 2** which looks like a hash table

Philosophy of Data Structures

- These two ways are basically two different ways of storing exactly the same set of data.
- You can see they have different organizations/structures.
- Data can be organized/stored in computer in different ways.
- A particular organization of data is called a **data structure**.



Philosophy of Data Structures

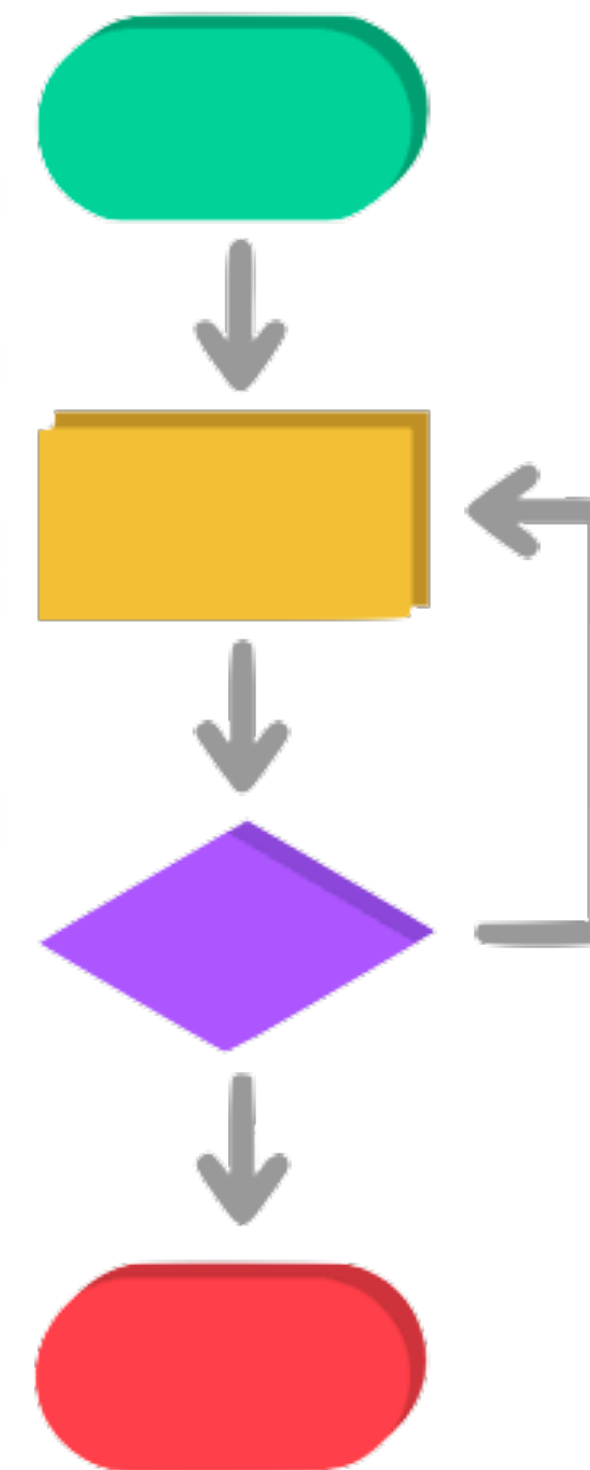
- Generally, a **data structure** defines a way of data organization with its associated operations.
- People use the term “data structure” to mean an organization or structuring for a collection of data items.
- **Example: A sorted list of integers stored in an array**
 - Collection of data items = Sorted list of integers
 - Associated Operations = Insertion, deletion, search etc.



Philosophy of Data Structures

- An algorithm is a systematic ordered set of steps (instructions) each of which processes some data to accomplish certain tasks.
- During the processing of data, algorithms need to store and retrieve its data into/from a data structure

An algorithm, named for the ninth-century Persian mathematician al-KhowArizmi, is simply a set of rules for carrying out some calculation, either by hand or, more usually, on a machine.



Philosophy of Data Structures

- Example: Let us consider the previous example. Suppose you want to **find a path from your home to the school**.

The following six possible paths:

Path 1: (Your Home, Friend's Home, School)

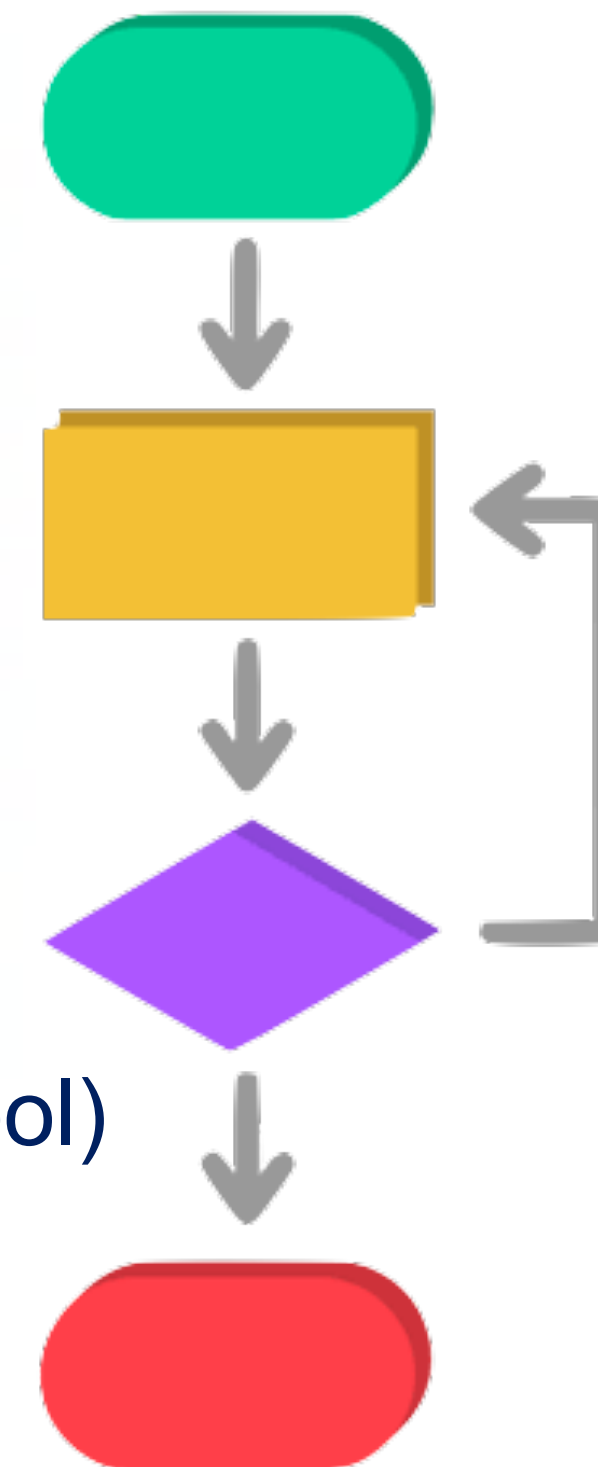
Path 2: (Your Home, Friend's Home, Store, School)

Path 3: (Your Home, Store, School)

Path 4: (Your Home, Store, Friend's Home, School)

Path 5: (Your Home, Supermarket, Store, Friend's Home, School)

Path 6: (Your Home, Supermarket, Store, School)

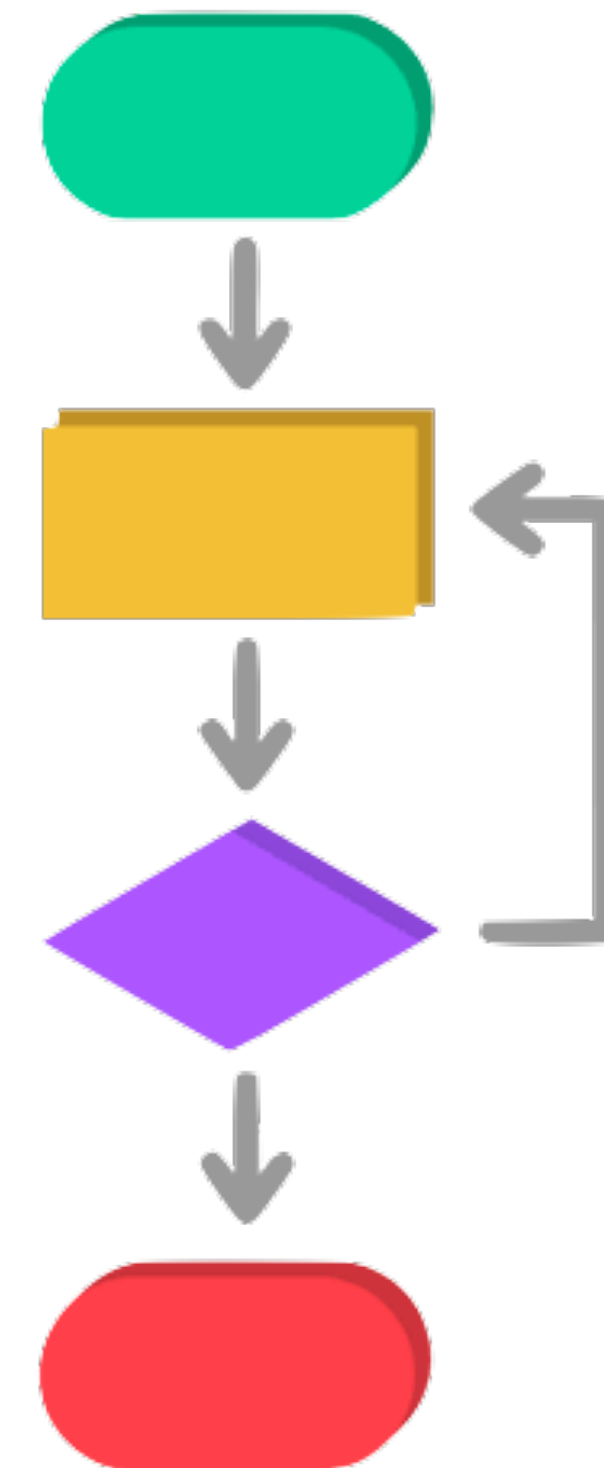


Philosophy of Data Structures

- Example: Let us consider the previous example. Suppose you want to find a shortest path from your home to the school.

ALGORITHM: FIND-PATH

1. Set $Path = \text{Your Home}$
2. Find all possible places you can go from **Your Home**.
3. For each place P_i found in step 2, repeat until you reach to **School**
 - a. Add the place P_i to $Path$
 - b. Find all possible places you can go from P_i .
4. Return the path.



Philosophy of Data Structures

- Example: Find a path from **your home** to the **school**.

Start from Your Home.

Current Location: **Your Home**.

Places:

Cost: 1

Path = Your Home



(Your Home, Friend's Home)
(Your Home, Store)
(Store, Your Home)
(Your Home, Supermarket)
(Supermarket, Your Home)
(Friend's Home, School)
(School, Friend's Home)
(Friend's Home, Store)
(Store, Friend's Home)
(Supermarket, Store)
(Store, School)
(School, Store)

(a) **Data Structure 1** which looks like
a list or an array



Philosophy of Data Structures

- Example: Find a path from **your home** to the **school**.

Current Location: **Your Home.**

Friend's Home.

Places: Friend's Home

Cost: 2

Path = Your Home

(Your Home, Friend's Home)

(Your Home, Store)

(Store, Your Home)

(Your Home, Supermarket)

(Supermarket, Your Home)

(Friend's Home, School)

(School, Friend's Home)

(Friend's Home, Store)

(Store, Friend's Home)

(Supermarket, Store)

(Store, School)

(School, Store)

(a) **Data Structure 1** which looks like
a list or an array



Philosophy of Data Structures

- Example: Find a path from **your home** to the **school**.

Current Location: **Your Home.**

↓
Store

Places: Friend's Home, Store

Cost: 4

Path = Your Home

(Your Home, Friend's Home)

(Your Home, Store)

(Store, Your Home)

(Your Home, Supermarket)

(Supermarket, Your Home)

(Friend's Home, School)

(School, Friend's Home)

(Friend's Home, Store)

(Store, Friend's Home)

(Supermarket, Store)

(Store, School)

(School, Store)

(a) **Data Structure 1** which looks like
a list or an array



Philosophy of Data Structures

- Example: Find a path from **your home** to the **school**.



Current Location: **Your Home.**



Places: Friend's Home, Store

Cost: 5

Path = Your Home

(Your Home, Friend's Home)
(Your Home, Store)
(Store, Your Home)
(Your Home, Supermarket)
(Supermarket, Your Home)
(Friend's Home, School)
(School, Friend's Home)
(Friend's Home, Store)
(Store, Friend's Home)
(Supermarket, Store)
(Store, School)
(School, Store)

(a) **Data Structure 1** which looks like
a list or an array



Philosophy of Data Structures

- Example: Find a path from **your home** to the **school**.

Current Location: **Your Home**.

↓
Super Market

Places: Friend's Home, Store, Super Market

Cost: 7

Path = Your Home

(Your Home, Friend's Home)
(Your Home, Store)
(Store, Your Home)
(Your Home, Supermarket)
(Supermarket, Your Home)
(Friend's Home, School)
(School, Friend's Home)
(Friend's Home, Store)
(Store, Friend's Home)
(Supermarket, Store)
(Store, School)
(School, Store)

(a) **Data Structure 1** which looks like
a list or an array



Philosophy of Data Structures

- Example: Find a path from **your home** to the **school**.

Current Location: **Your Home**.



Places: Friend's Home, Store, Super Market

Cost: 8

Path = Your Home

(Your Home, Friend's Home)
(Your Home, Store)
(Store, Your Home)
(Your Home, Supermarket)
(Supermarket, Your Home)
(Friend's Home, School)
(School, Friend's Home)
(Friend's Home, Store)
(Store, Friend's Home)
(Supermarket, Store)
(Store, School)
(School, Store)

(a) **Data Structure 1** which looks like
a list or an array



Philosophy of Data Structures

- Example: Find a path from **your home** to the **school**.

Current Location: **Your Home.**



Places: Friend's Home, Store, Super Market

Cost: 9

Path = Your Home

(Your Home, Friend's Home)
(Your Home, Store)
(Store, Your Home)
(Your Home, Supermarket)
(Supermarket, Your Home)
(Friend's Home, School)
(School, Friend's Home)
(Friend's Home, Store)
(Store, Friend's Home)
(Supermarket, Store)
(Store, School)
(School, Store)

(a) **Data Structure 1** which looks like
a list or an array



Philosophy of Data Structures

- Example: Find a path from **your home** to the **school**.

Current Location: **Your Home.**



Places: Friend's Home, Store, Super Market

Cost: 10

Path = Your Home

(Your Home, Friend's Home)
(Your Home, Store)
(Store, Your Home)
(Your Home, Supermarket)
(Supermarket, Your Home)
(Friend's Home, School)
(School, Friend's Home)
(Friend's Home, Store)
(Store, Friend's Home)
(Supermarket, Store)
(Store, School)
(School, Store)

(a) **Data Structure 1** which looks like
a list or an array



Philosophy of Data Structures

- Example: Find a path from **your home** to the **school**.

Current Location: **Your Home.**



Places: Friend's Home, Store, Super Market

Cost: 11

Path = Your Home

(Your Home, Friend's Home)
(Your Home, Store)
(Store, Your Home)
(Your Home, Supermarket)
(Supermarket, Your Home)
(Friend's Home, School)
(School, Friend's Home)
(Friend's Home, Store)
(Store, Friend's Home)
(Supermarket, Store)
(Store, School)
(School, Store)

(a) **Data Structure 1** which looks like
a list or an array



Philosophy of Data Structures

- Example: Find a path from **your home** to the **school**.

Current Location: **Your Home.**



Places: Friend's Home, Store, Super Market

Cost: 12

Path = Your Home

(Your Home, Friend's Home)
(Your Home, Store)
(Store, Your Home)
(Your Home, Supermarket)
(Supermarket, Your Home)
(Friend's Home, School)
(School, Friend's Home)
(Friend's Home, Store)
(Store, Friend's Home)
(Supermarket, Store)
(Store, School)
(School, Store)

(a) **Data Structure 1** which looks like
a list or an array



Philosophy of Data Structures

- Example: Find a path from **your home** to the **school**.

Current Location: **Your Home.**



Places: Friend's Home, Store, Super Market

Cost: 13

Path = Your Home

(Your Home, Friend's Home)
(Your Home, Store)
(Store, Your Home)
(Your Home, Supermarket)
(Supermarket, Your Home)
(Friend's Home, School)
(School, Friend's Home)
(Friend's Home, Store)
(Store, Friend's Home)
(Supermarket, Store)
(Store, School)
(School, Store)

(a) **Data Structure 1** which looks like
a list or an array



Philosophy of Data Structures

- Example: Find a path from **your home** to the **school**.



Current Location: **Your Home.**



Places: Friend's Home, Store, Super Market

Cost: 14

Path = Your Home

(Your Home, Friend's Home)
(Your Home, Store)
(Store, Your Home)
(Your Home, Supermarket)
(Supermarket, Your Home)
(Friend's Home, School)
(School, Friend's Home)
(Friend's Home, Store)
(Store, Friend's Home)
(Supermarket, Store)
(Store, School)
(School, Store)

(a) **Data Structure 1** which looks like
a list or an array



Philosophy of Data Structures

- Example: Find a path from **your home** to the **school**.



Current Location: **Your Home.**



Places: Friend's Home, Store, Super Market

Cost: 15

Path = Your Home

(Your Home, Friend's Home)
(Your Home, Store)
(Store, Your Home)
(Your Home, Supermarket)
(Supermarket, Your Home)
(Friend's Home, School)
(School, Friend's Home)
(Friend's Home, Store)
(Store, Friend's Home)
(Supermarket, Store)
(Store, School)
(School, Store)

(a) **Data Structure 1** which looks like
a list or an array



Philosophy of Data Structures

- Example: Find a path from **your home** to the **school**.

Current Location: **Friend's Home**.



Places: Store, Super Market

Cost: 16

Path = Your Home, Friend's Home

(Your Home, Friend's Home)
(Your Home, Store)
(Store, Your Home)
(Your Home, Supermarket)
(Supermarket, Your Home)
(Friend's Home, School)
(School, Friend's Home)
(Friend's Home, Store)
(Store, Friend's Home)
(Supermarket, Store)
(Store, School)
(School, Store)

(a) **Data Structure 1** which looks like
a list or an array



Philosophy of Data Structures

- Example: Find a path from **your home** to the **school**.



Current Location: **Friend's Home.**



Places: Store, Super Market

Cost: 17

Path = Your Home, Friend's Home

(Your Home, Friend's Home)
(Your Home, Store)
(Store, Your Home)
(Your Home, Supermarket)
(Supermarket, Your Home)
(Friend's Home, School)
(School, Friend's Home)
(Friend's Home, Store)
(Store, Friend's Home)
(Supermarket, Store)
(Store, School)
(School, Store)

(a) **Data Structure 1** which looks like
a list or an array



Philosophy of Data Structures

- Example: Find a path from **your home** to the **school**.

Current Location: **Friend's Home**.



Places: Store, Super Market

Cost: 18

Path = Your Home, Friend's Home

(Your Home, Friend's Home)
(Your Home, Store)
(Store, Your Home)
(Your Home, Supermarket)
(Supermarket, Your Home)
(Friend's Home, School)
(School, Friend's Home)
(Friend's Home, Store)
(Store, Friend's Home)
(Supermarket, Store)
(Store, School)
(School, Store)

(a) **Data Structure 1** which looks like
a list or an array



Philosophy of Data Structures

- Example: Find a path from **your home** to the **school**.

Current Location: **Friend's Home.**



Places: Store, Super Market

Cost: 19

Path = Your Home, Friend's Home

(Your Home, Friend's Home)
(Your Home, Store)
(Store, Your Home)
(Your Home, Supermarket)
(Supermarket, Your Home)
(Friend's Home, School)
(School, Friend's Home)
(Friend's Home, Store)
(Store, Friend's Home)
(Supermarket, Store)
(Store, School)
(School, Store)

(a) **Data Structure 1** which looks like
a list or an array



Philosophy of Data Structures

- Example: Find a path from **your home** to the **school**.

Current Location: **Friend's Home**.



Places: Store, Super Market

Cost: 20

Path = Your Home, Friend's Home

(Your Home, Friend's Home)
(Your Home, Store)
(Store, Your Home)
(Your Home, Supermarket)
(Supermarket, Your Home)
(Friend's Home, School)
(School, Friend's Home)
(Friend's Home, Store)
(Store, Friend's Home)
(Supermarket, Store)
(Store, School)
(School, Store)

(a) **Data Structure 1** which looks like
a list or an array



Philosophy of Data Structures

- Example: Find a path from **your home** to the **school**.

Current Location: **Friend's Home**.



Places: Store, Super Market

Cost: 22

Path = Your Home, Friend's Home, School

(Your Home, Friend's Home)
(Your Home, Store)
(Store, Your Home)
(Your Home, Supermarket)
(Supermarket, Your Home)
(Friend's Home, School)
(School, Friend's Home)
(Friend's Home, Store)
(Store, Friend's Home)
(Supermarket, Store)
(Store, School)
(School, Store)

(a) **Data Structure 1** which looks like
a list or an array



Philosophy of Data Structures

- Example: Find a path from **your home** to the **school**.

Start from Your Home.

Current Location: **Your Home**.

Friend's Home, Store, Super Market.

Places: Friend's Home, Store, Super Market

Cost: 2

Path = Your Home

Your Home	(Friend's Home, Store, Supermarket)
Friend's Home	(Store, School)
Supermarket	(Your Home, Store)
Store	(Your Home, Friend's Home, School)
School	(Friend's Home, Store)

(b) **Data Structure 2** which looks like a hash table



Philosophy of Data Structures

- Example: Find a path from **your home** to the **school**.

Start from Your Home.

Current Location: Your Home.

Friend's Home, Store, Super Market.

Places: Friend's Home, Store, Super Market

Cost: 3

Path = Your Home

Your Home	(Friend's Home, Store, Supermarket)
Friend's Home	(Store, School)
Supermarket	(Your Home, Store)
Store	(Your Home, Friend's Home, School)
School	(Friend's Home, Store)

(b) **Data Structure 2** which looks like a hash table



Philosophy of Data Structures

- Example: Find a path from **your home** to the **school**.

Start from Your Home.

Current Location: **Your Home**.

Friend's Home, Store, **Super Market**.

Places: Friend's Home, Store, Super Market

Cost: 4

Path = Your Home

Your Home	(Friend's Home, Store, Supermarket)
Friend's Home	(Store, School)
Supermarket	(Your Home, Store)
Store	(Your Home, Friend's Home, School)
School	(Friend's Home, Store)

(b) **Data Structure 2** which looks like a hash table



Philosophy of Data Structures

- Example: Find a path from **your home** to the **school**.

Start from Your Home.

Current Location: Friend's Home.

Store, School.

Places: Store, Super Market

Cost: 5

Path = Your Home, Friend's Home



Your Home	(Friend's Home, Store, Supermarket)
Friend's Home	(Store, School)
Supermarket	(Your Home, Store)
Store	(Your Home, Friend's Home, School)
School	(Friend's Home, Store)

(b) **Data Structure 2** which looks like a hash table



Philosophy of Data Structures

- Example: Find a path from **your home** to the **school**.

Start from Your Home.

Current Location: **Friend's Home**.

Store, School.

Places: Store, Super Market

Cost: 7

Path = Your Home, Friend's Home

Your Home	(Friend's Home, Store, Supermarket)
Friend's Home	(Store, School)
Supermarket	(Your Home, Store)
Store	(Your Home, Friend's Home, School)
School	(Friend's Home, Store)

(b) **Data Structure 2** which looks like a hash table



Philosophy of Data Structures

- Example: Find a path from **your home** to the **school**.

Start from Your Home.

Current Location: **Friend's Home**.

Store, **School**.

Places: Store, Super Market

Cost: 8

Path = Your Home, Friend's Home, School

Your Home	(Friend's Home, Store, Supermarket)
Friend's Home	(Store, School)
Supermarket	(Your Home, Store)
Store	(Your Home, Friend's Home, School)
School	(Friend's Home, Store)

(b) **Data Structure 2** which looks like a hash table



Philosophy of Data Structures

- The ways in which data are organized in data structures determine
 - **the number of computations** to solve a problem. (A data structure determines the performance of an algorithm.)
 - the steps (slightly or completely different) in algorithms
- Therefore, it is very important to choose a **right data structure** for an algorithm to get an **efficient solution** for a problem.
- A good data structure should be able to organize or represent data and their relationships as they appear in real world.

Note: you may probably noticed that based on the logical way how a data structure organizes the data, the same problem may have slightly or completely different algorithms to solve the same problem.



Philosophy of Data Structures

- You might think that **with todays powerful computers, algorithm efficiency is becoming less important.**
- After all, processor speed and memory size still continue to improve. Won't today's efficiency problem be solved by tomorrow's more powerful hardware?
- As we develop more powerful computers, the additional computing power has to tackle more complex problems with more sophisticated user interfaces, bigger problem sizes, or new problems previously deemed computationally infeasible.
- More complex problems demand more computation, making the **need for efficient algorithms even greater.**
- An algorithm to be efficient **needs an appropriate data structure** for storing and retrieving its data during processing.



Philosophy of Data Structures

- Given sufficient space to store a collection of **data items**, it is always possible to access (insert/search) specified items within the collection.
- One obvious way is that we can store the collection of data items in an unsorted array.
- It is possible to perform all necessary operations on the unsorted array.
- However, using the **proper data structure** can make the difference between an algorithm running in a few seconds and one requiring many days.
- For example, searching for a given record in a sorted array is much faster than searching for it in an unsorted array. (Even much faster in **hash table**)



Characteristics of Data Structures

- Different data structures have different characteristics.
- An appropriate data structure is selected for an algorithm based on its characteristics.
- Some of the characteristics of data structures are:
 1. Linear or non-linear
 2. Homogeneous or heterogeneous
 3. Static or dynamic
 4. Time and Space Complexity
 5. Correctness
 6. Reusability
 7. Data Abstraction



Abstract Data Types

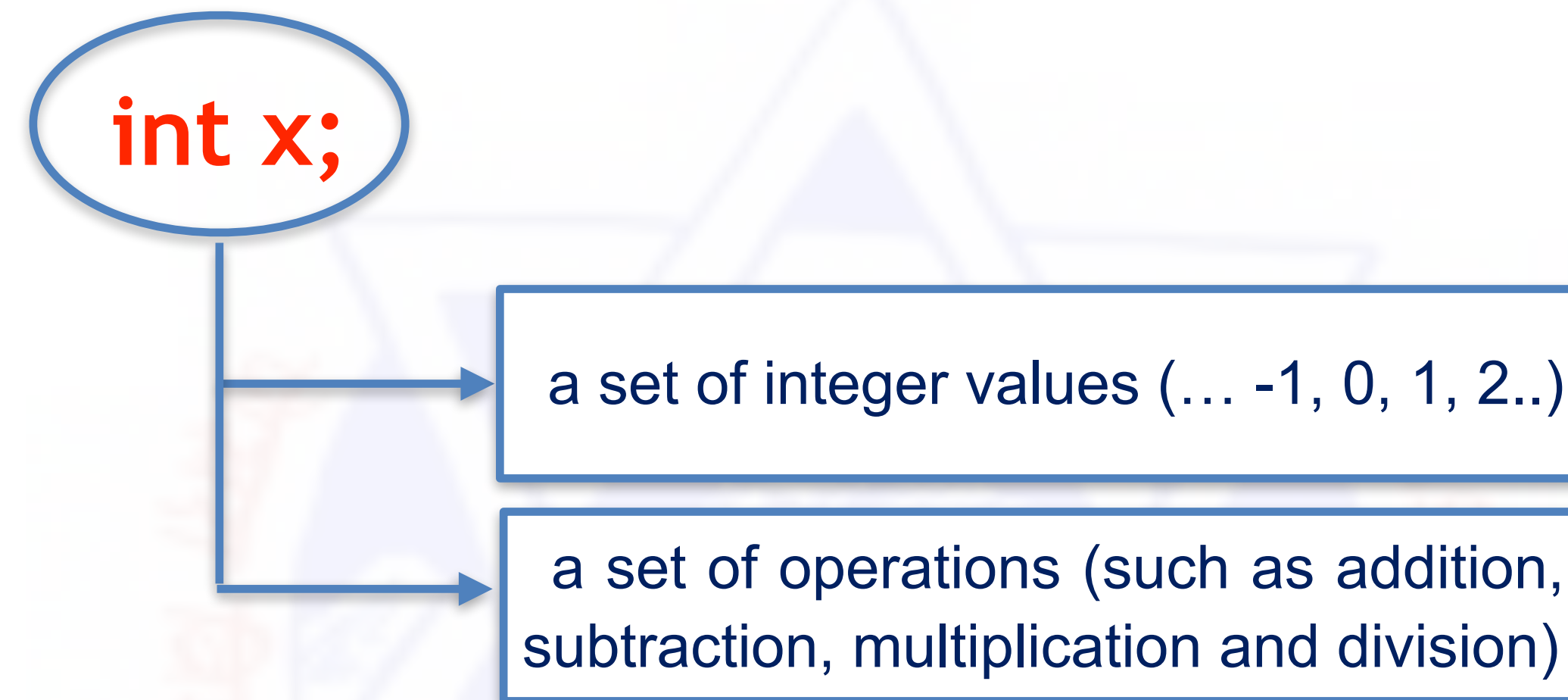
- An abstract data type (ADT) is a set of data values and associated operations.

Abstract
Data Types = A Set of
Data + A Set of
Operations on
these Data

- ADT does not provide the implementation detail.



Abstract Data Types



- For example, when you declare a variable in any programming language,
 - If you declare `int X;`, it specifies a set of integer values and a set of operations (such as addition, subtraction, multiplication and division) on X.
 - You don't need to know how ints are represented nor how the operations-addition, subtraction, multiplication and division are implemented.
 - Therefore, the type `int` is an **abstract data type**.



Abstract Data Types

- **EXAMPLE:** When operating a car, the primary activities are steering, accelerating, and braking. When you drive a car, you steer by turning the steering wheel, accelerate by pushing the gas pedal, and brake by pushing the brake pedal. This design for cars can be viewed as an ADT with operations "steer", "accelerate", and "brake". Two cars might implement these operations in radically different ways, say with different types of engine, or front- versus rear-wheel drive. Yet, most drivers can operate many different cars because the ADT presents a uniform method of operation that does not require the driver to understand the specifics of any particular engine or drive design. These differences are deliberately hidden.



Abstract Data Types

- An *abstract data type* is a specification of a *data type* independent of its *implementation*.
- The interface for an ADT is defined in terms of a *type* (a set of values) and a set of operations that manipulate the values on that type.
- The ADT does not tell about how its data type is implemented. That means implementation details are hidden from the ADT users. It protects from outside access. This abstraction is known as encapsulation.
- An ADT together with its implementation form a *class* in object-oriented programming languages.
- A variable defines a space that is required for a data item.
- Each operation in an ADT is implemented using a member function or a method.



Abstract Data Types

- The implementation of an ADT is a **data structure**.

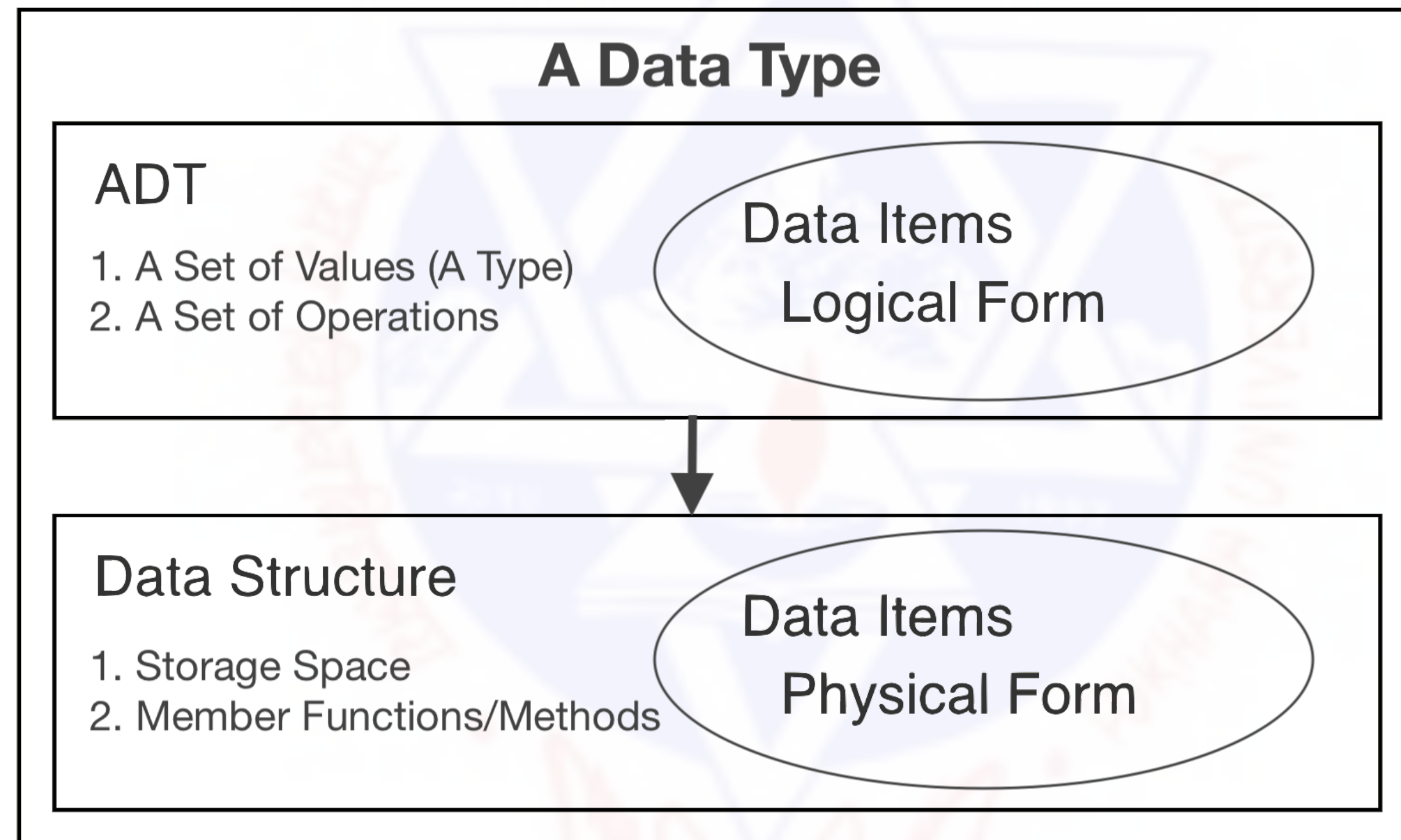


Fig: A data type with its ADT and Data Structure

Abstract Data Types

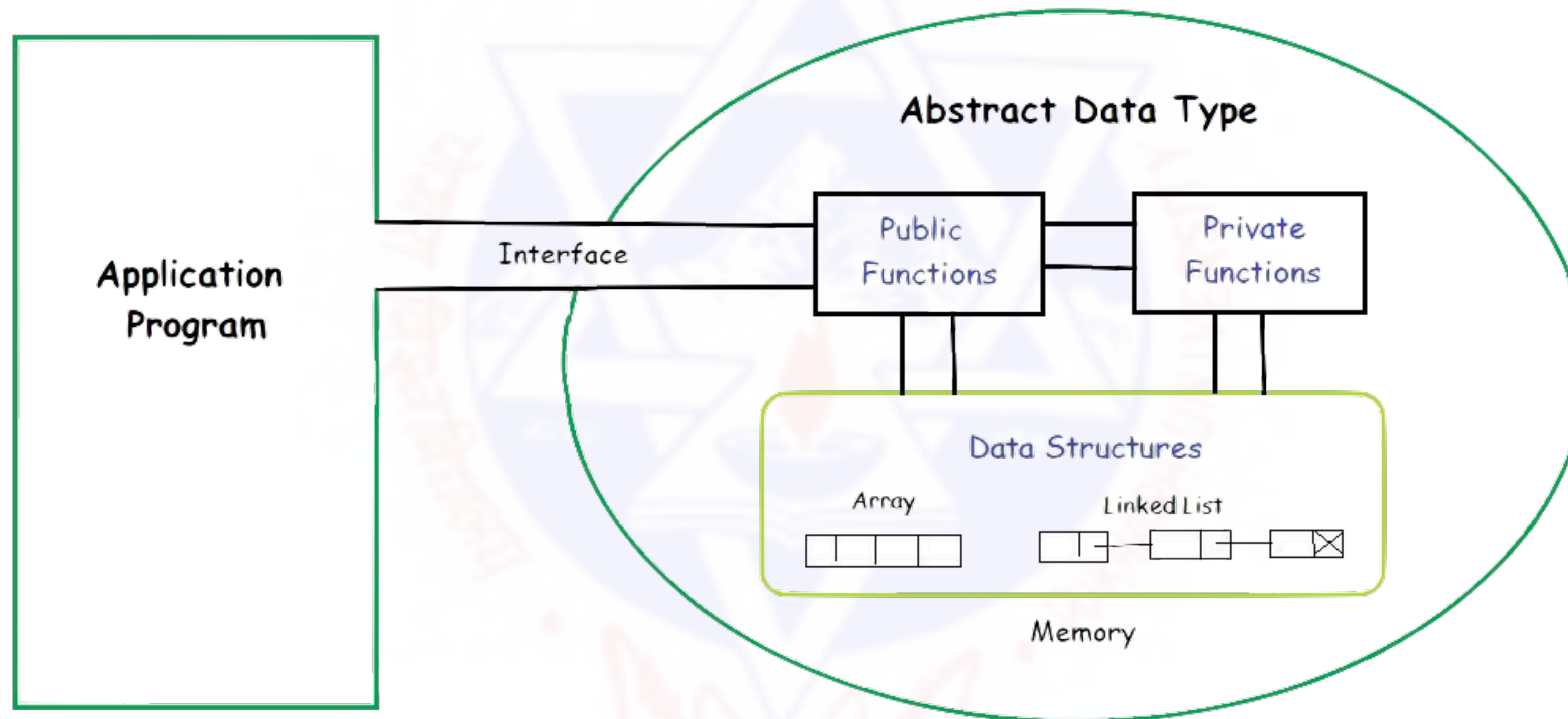


Fig: The ADT Model

Abstract Data Types

- The ADT encapsulates the data and the operations and hide it from users.
- It is called “abstract” because it gives an implementation-independent view.
- The process of providing only the essentials and hiding the details is known as abstraction.
- Abstract data types provide an interface hiding the implementation detail for users.
- This abstraction mechanism provides the flexibility to change in the implementation without affecting the user’s interface.
- It also provide a layer of security
 - user does not know the underlying implementation and
 - So, they cannot change the implementation by mistakenly or intentionally



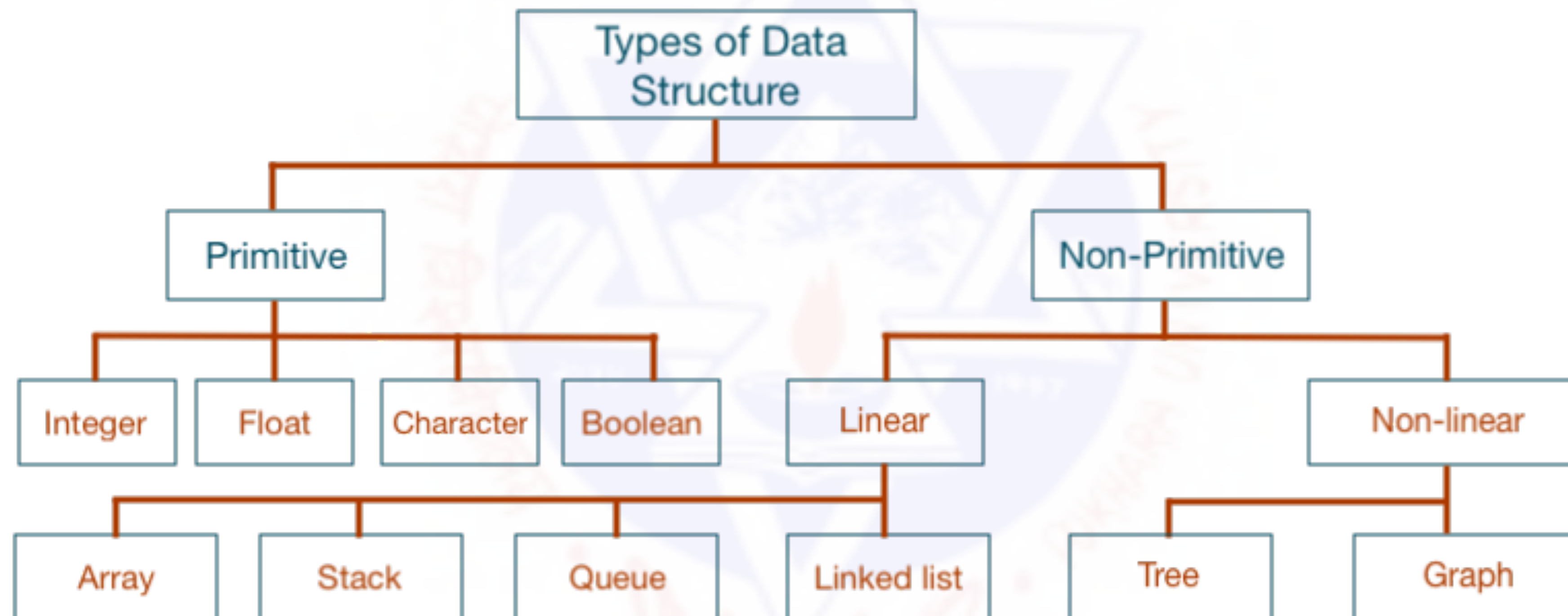
Abstract Data Types

- Any ADT can be implemented using an array or a linked-list.
 - The array implementation results in the **static ADT** and
 - linked-list implementation results in **dynamic ADT**.
- Programming languages such as C++, C#, Java, Python etc. support **abstract data type** through **class** mechanism.
- Classes are defined in terms of data and operations.
- The implementation of the operations of class is done separately.



Types of Data Structures

- primitive data structures contain atomic data
- Non-primitive contain collection of data



Based on the way how data structures
organize and store the data



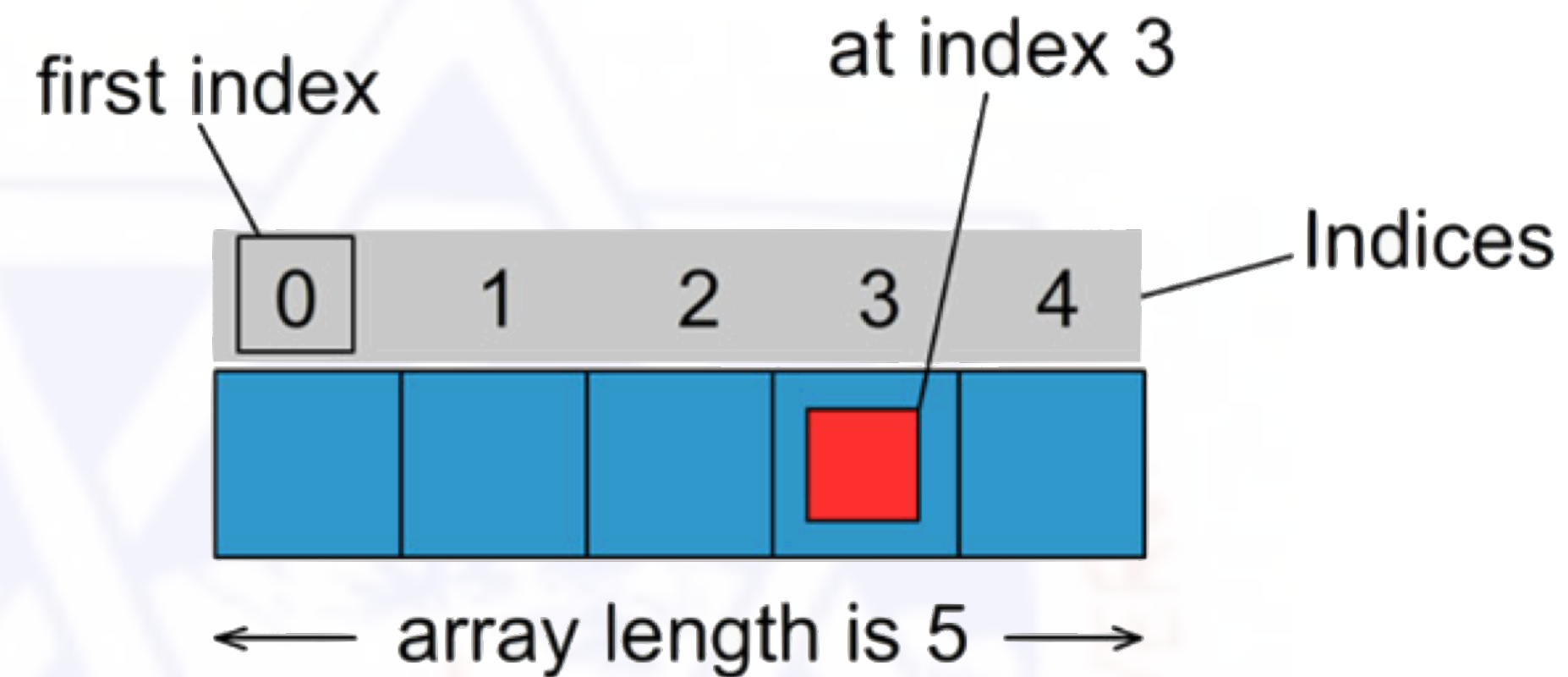
Types of Data Structures

- Based on whether the size of data structure is fixed or variable, **data structures** can be
 1. **static** (e.g. Array)
 2. **dynamic** (e.g. Linked list)
- A data structure is said to be **static** if the size of the data structure is declared at creation time and remains fix all time.
- A data structure is said to be **dynamic** if the size of the data structures can be increased or decreased on demand.

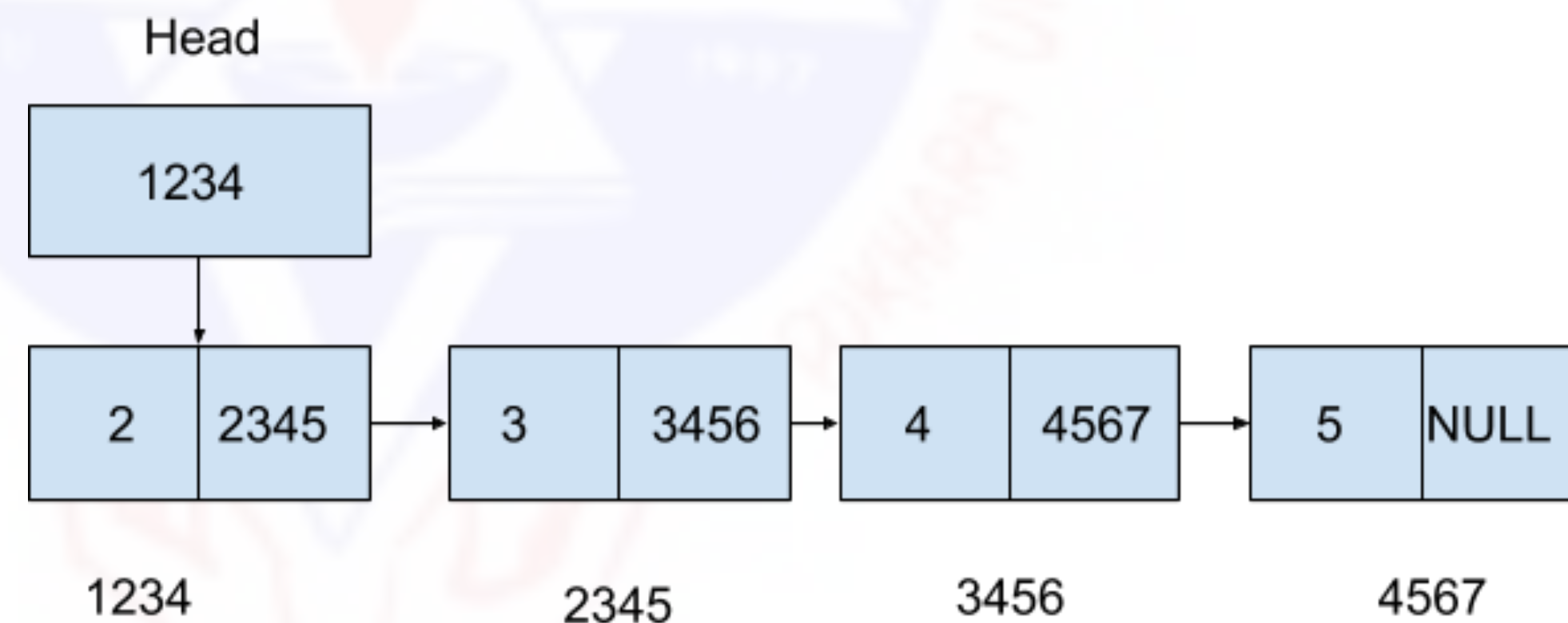


Types of Data Structures

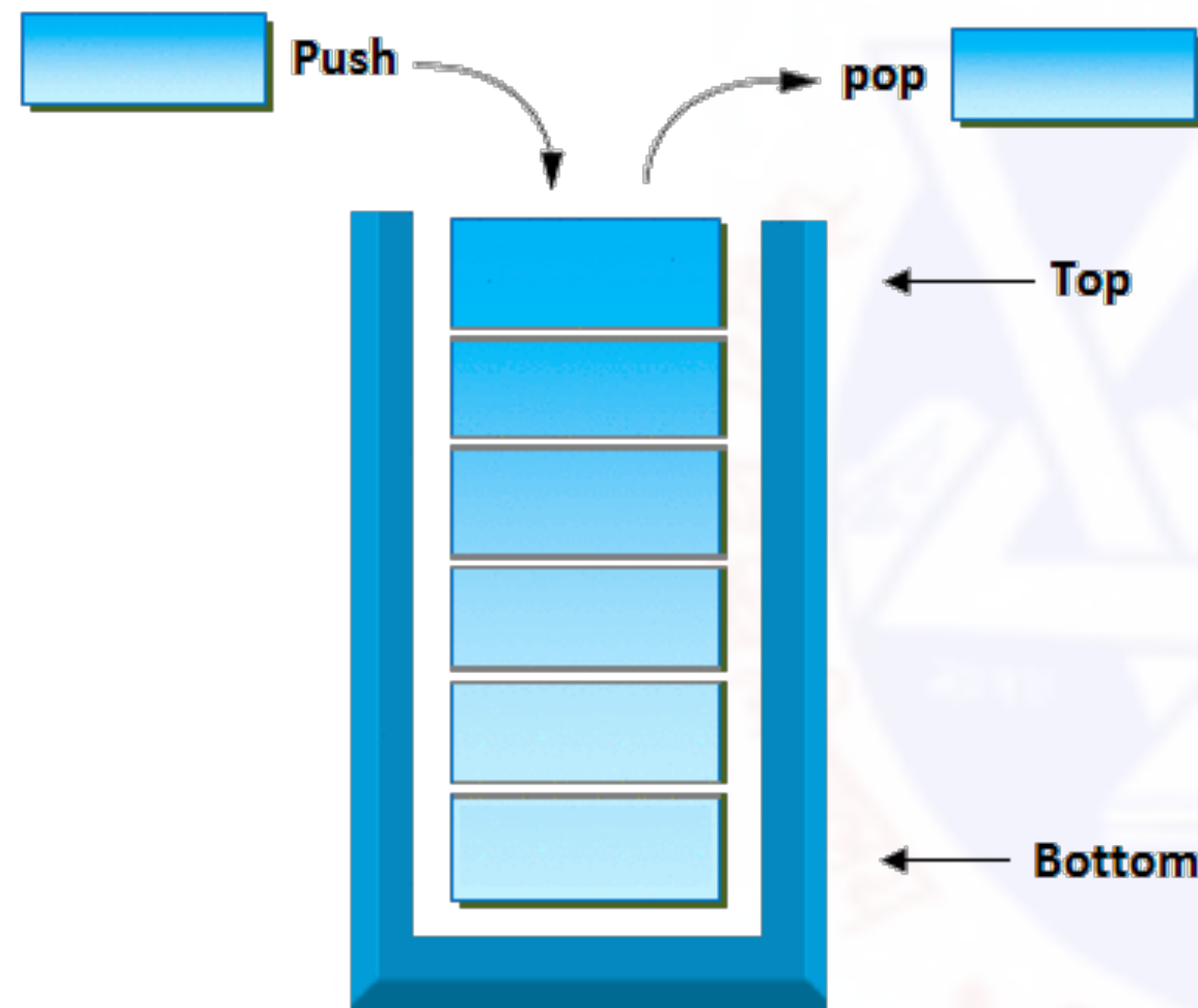
(a) Array



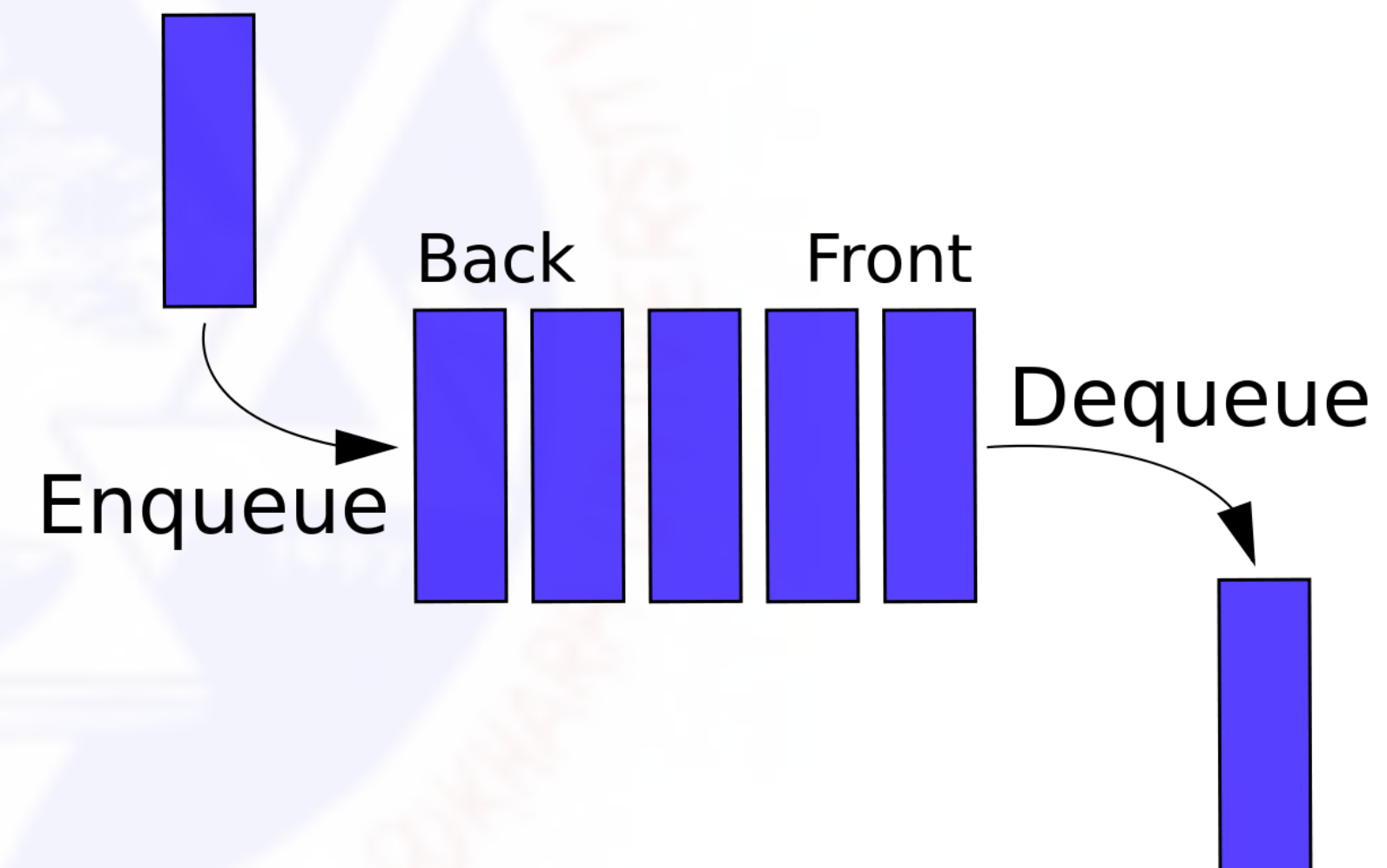
(b) Linked list



Types of Data Structures



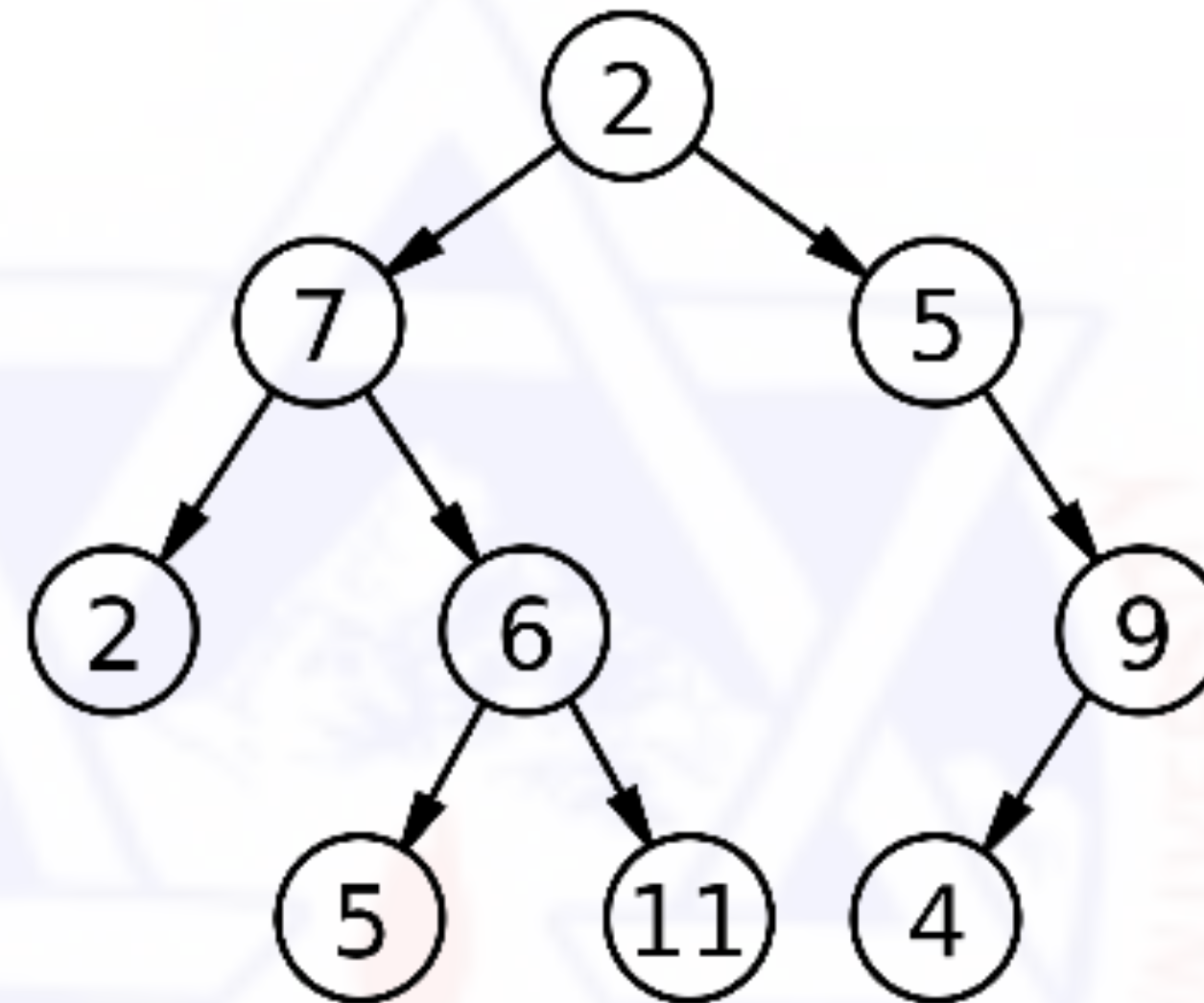
(c) Stack



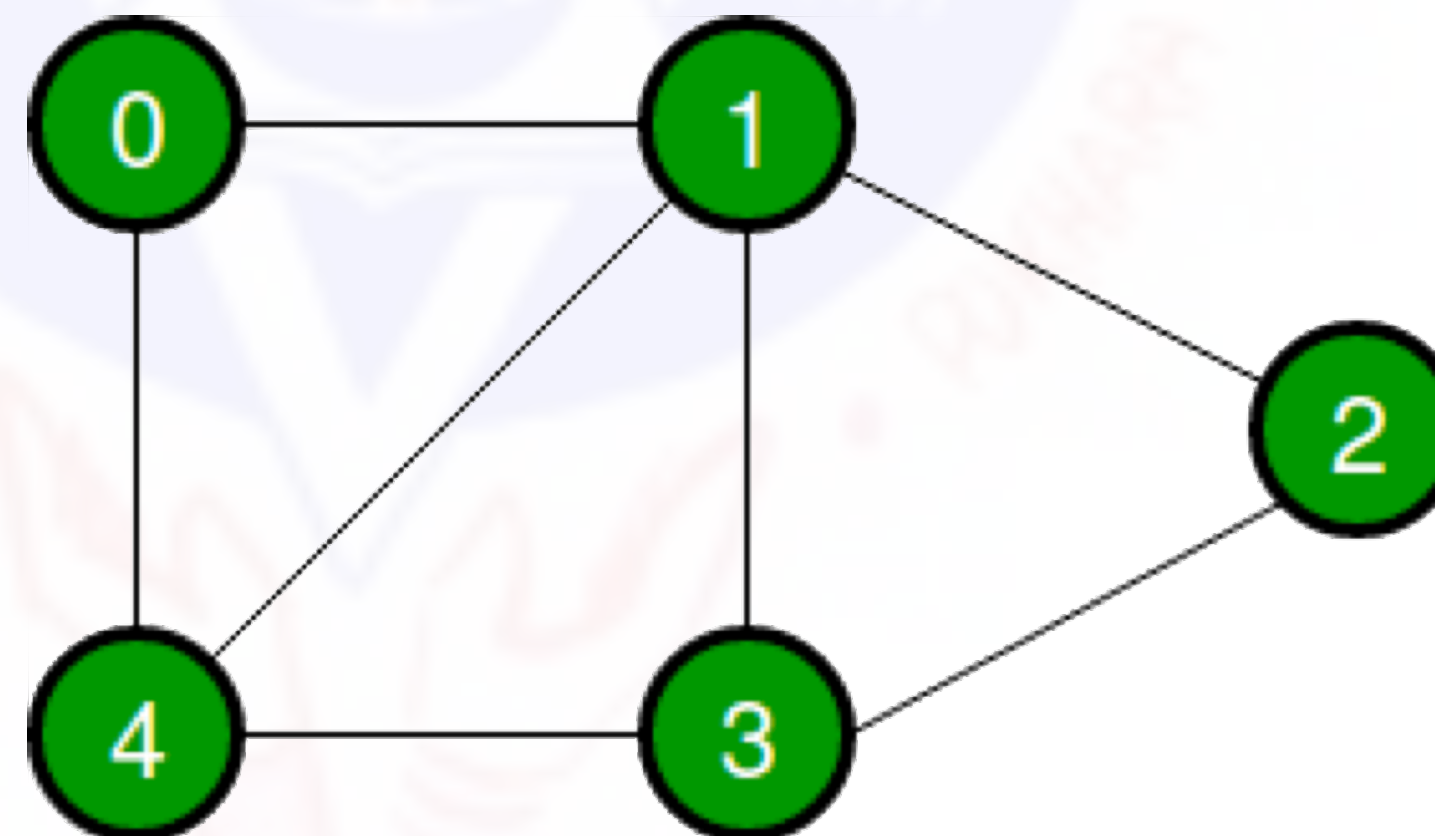
(d) Queue

Types of Data Structures

(e) Tree



(f) Graph



Operations in Data Structures

- Data structures are used to organize and store data in some logical ways so that algorithms can efficiently use those data.
- The algorithms access data stored on data structure for different purposes using some operations.
- The basic operations that can be performed in data structures are:
 - Traversing
 - Searching
 - Insertion
 - Deletion
 - Sorting
 - Merging

Note: The way of doing these operations vary from one type of data structure to another. For example, the insertion of an item into a stack is completely different when you insert the item into a queue.



Operations in Data Structures

- a)**Traversing:** - This operation is also called visiting. It is the operation of access each data/item stored in data structure exactly once so that certain data can be processed. For example, if you want to find an item in a tree data structure, then you will traverse each element of the tree using some traversing method to search the item.
- b)**Searching:** - It is the operation that is commonly performed in data structure to find some data with a given key value. To search an item in data structure you have to traverse the data structure.
- c)**Inserting:** - It is the operation which adds data/item into the data structure. Before you insert an item into the data structure, you have to find an appropriate location where you want to insert the item. You cannot insert an item into a data structure which is already full.



Operations in Data Structures

- d)**Deleting:** - It is the operation of removing an item from data structure. You cannot delete an item from a data structure which is already empty.
- e)**Sorting:** - It is the process of arranging the data in some logical order such as alphabetical or numeric order. For example, you should need to sort the elements in data structure before you perform binary search.
- f)**Merging:** - It is the process of combining the elements of two different sorted data structures into a single data structure. This operation is mainly performed during the sorting operation to sort two different sorted data structures.



Introduction to **ALGORITHM DESIGN**

Algorithm Design

- When it comes to solve problems in computer science, algorithms play a crucial role.
- The word **algorithm** came from the name of a Persian author, Abu Ja'far Mohammed ibn Musa al-Khowarizmi.
- An algorithm is a set of instructions applied to solve a particular problem.
- But the question is how to design an algorithm to solve that problem?
- It is crucial to construct an effective algorithm to the problem efficiently.
- An algorithm design technique means a unique approach or mathematical method for creating/constructing efficient solutions (algorithms) to the problems.
- They provide templates suited to solve a broad range of diverse problems



Algorithm Design

- Multiple algorithms can solve a problem, but not all algorithms can solve it efficiently.
- An algorithm created with the right design technique based on the nature of the problem can solve the problem much more efficiently
- Following are some of the main algorithm design techniques:
 1. Brute-force or exhaustive search
 - 2. Divide and Conquer**
 - 3. Greedy Algorithms**
 4. Dynamic Programming
 5. Branch and Bound Algorithm
 6. Randomized Algorithm
 - 7. Backtracking Algorithm**



Divide and Conquer Algorithms

- Divide and conquer is a technique for designing algorithms.
- The divide and conquer approach
 - decomposes the instance to be solved into a number of smaller sub-instances of the same problem
 - solves successively and independently each of these sub-instances
 - finally combines the sub-solutions thus obtained in such a way as to obtain the solution of the original instance.
- The divide and conquer approach generally uses the recursive approach to divide and solve the problem.
- Sometimes, it is possible to replace the recursive calls in divide and conquer by iterative loop.
- The iterative approach is likely to be faster than the recursive version.

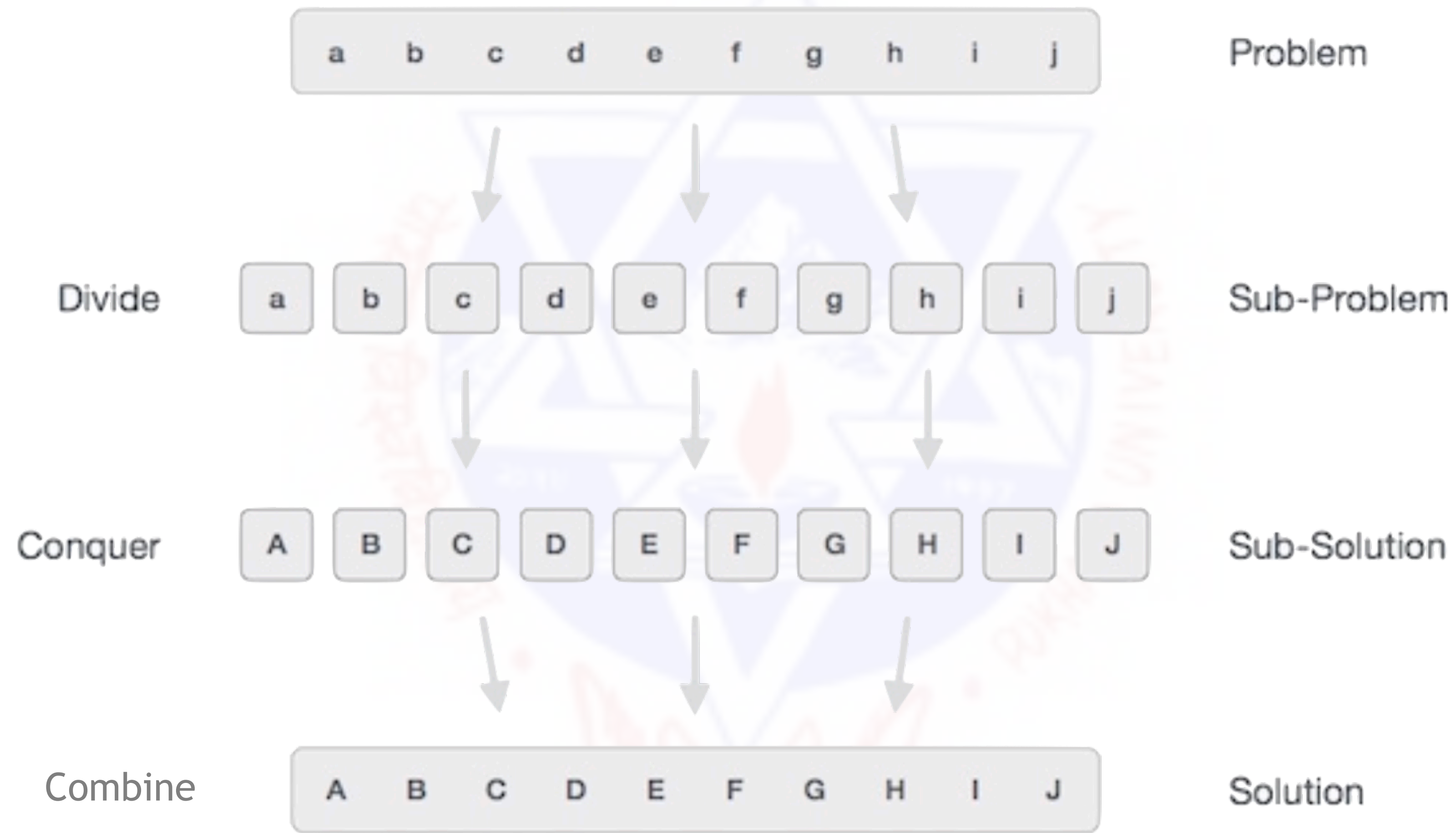


Divide and Conquer Algorithms

- In divide and conquer approach, the problem is divided into smaller sub-problems.
- When we keep on dividing the subproblems into even smaller sub-problems, we may eventually reach a stage where no more division is possible.
- Those "atomic" smallest possible sub-problem (fractions) are solved independently.
- The solution of all sub-problems is finally merged in order to obtain the solution of an original problem.
- Steps in divide and conquer algorithms:
 1. Divide/Break
 2. Conquer/Solve
 3. Merge/Combine



Divide and Conquer Algorithms



Divide and Conquer Algorithms

- **Divide**

- This step involves breaking the problem into smaller sub-problems.
- This step generally takes a recursive approach (iterative approach requires less cost) to divide the problem until no sub-problem is further divisible.
- At this stage, sub-problems become atomic in nature but still represent some part of the original problem.

- **Conquer/Solve**

- This step receives a lot of smaller sub-problems to be solved.
- The atomic sub-problems are recursively solved in this step.

- **Combine**

- When the atomic sub-problems are solved, this stage recursively combines them until they formulate a solution of the original problem.



Divide and Conquer Algorithms

- Given a function to compute on n inputs, the divide and conquer strategy splits the input into k distinct subsets, $1 < k \leq n$, yielding k subproblems.
- These subproblems must be solved and then a method must be found to combine sub-solutions into a solution of the whole problem.
- If the sub-problems are still relatively large, then the divide and conquer strategy can possibly be reapplied.
- Often the subproblems resulting from a divide and conquer strategy are of the same type as the original problem.
- For those cases reapplication of the divide and conquer strategy is naturally expressed by a recursive algorithm.
- The sub-problems are divided until they are small and simple enough to be solved without splitting.



Divide and Conquer Algorithms

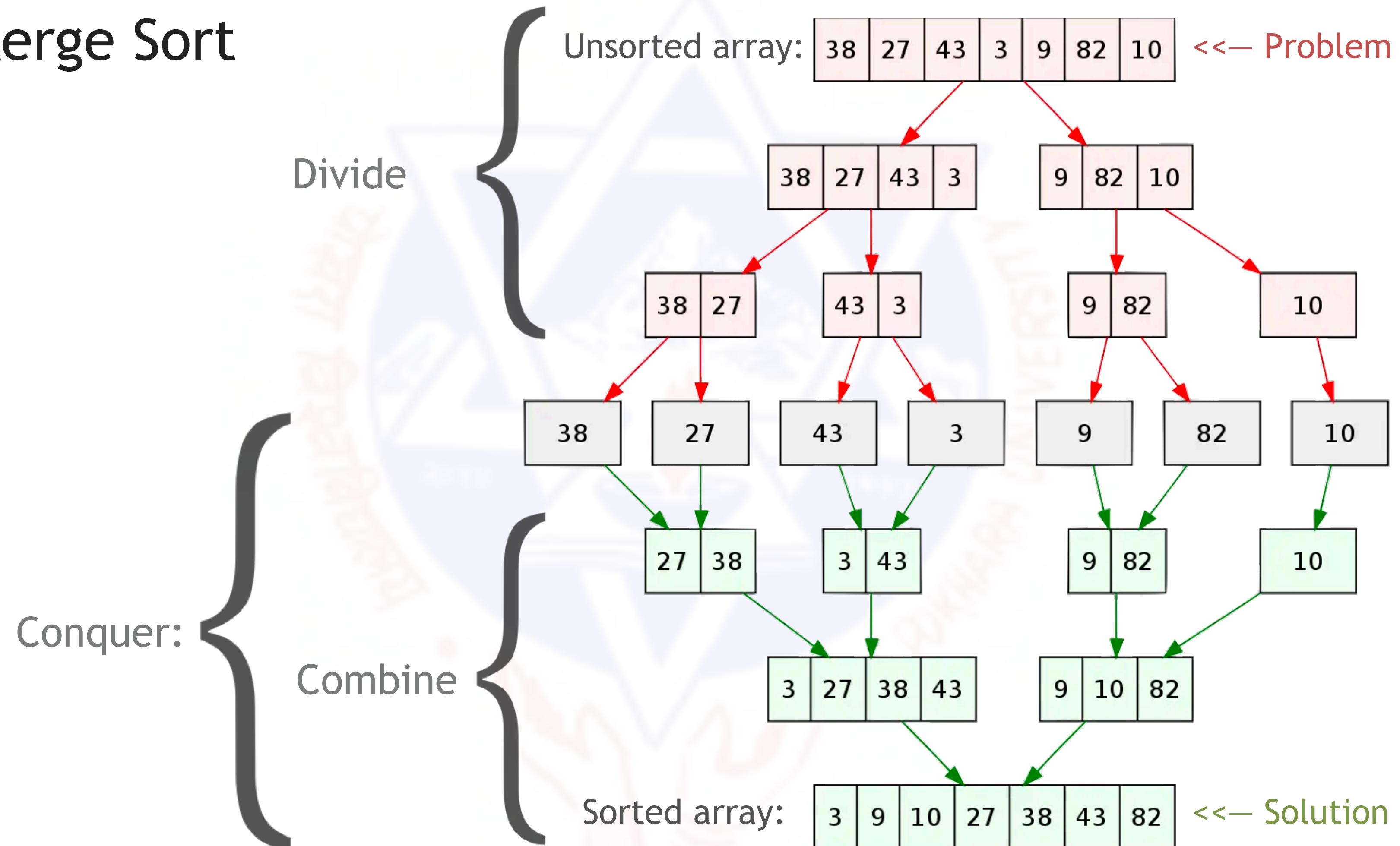
```
1  Algorithm DC(P)
2  {
3      if P is sufficiently small or simple then
4          return Solution(P)
5      else
6          {
7              Divide P into smaller instances P1, P2, ... Pk, k>=1
8              for all P1, P2, ... Pk, Apply DC(Pi)
9              return Combine(DC(P1), DC(P2), ..., DC(Pk))
10         }
11 }
```

- The algorithms that follow divide-and-conquer approach: **Merge Sort**, **Quick Sort**, **Binary Search** etc.



Divide and Conquer Algorithms

- Merge Sort



Greedy Algorithms

- Greedy method is the most straight forward algorithm design technique and can be applied to a wide variety of problems.
- Most of the problems have n inputs and require us to obtain a subset that satisfies some constraints.
- Any subset that satisfies these constraints is called a **feasible solution**.
- We need to find a feasible solution that either maximizes or minimizes the **objective function**. A feasible solution that does this is called an **optimal solution**.
- There is usually an obvious way to determine a feasible solution but not necessarily an optimal solution.



Greedy Algorithms

- The greedy algorithm works in stages, considering one input at a time.
- At each stage, a decision is made regarding whether a particular input is in an optimal solution. This is done by considering the inputs in an order determined by some selection procedure.
- If the inclusion of the next input into the partially constructed optimal solution will result in an infeasible solution then this input is not added to the partial solution. Otherwise, it is added.
- The selection procedure itself is based on some optimization measure. This measure may be the objective function.



Greedy Algorithms

```
1  Algorithm Greedy( $a, n$ )
2  //  $a[1 : n]$  contains the  $n$  inputs.
3  {
4       $solution := \emptyset$ ; // Initialize the solution.
5      for  $i := 1$  to  $n$  do
6      {
7           $x := \text{Select}(a)$ ;
8          if Feasible( $solution, x$ ) then
9               $solution := \text{Union}(solution, x)$ ;
10     }
11     return  $solution$ ;
12 }
```



Greedy Algorithms

- The function *Select* selects an input from $a[]$ and removes it.
- The selected input's value is assigned to x .
- Feasible is a boolean-valued function that determines whether x can be included into the solution vector.
- The function *Union* combines x with the solution and updates the objective function.
- The function *Greedy* describes the essential way that a greedy algorithm will look, once a particular problem is chosen and the functions *Select*, *Feasible* and *Union* are properly implemented.



Greedy Algorithms

- Greedy algorithms are typically used to solve optimization problems:
 - Find best order to execute a certain set of jobs on a computer,
 - Find shortest route in a graph
- In most common, a greedy algorithms has:
 - A set or list of candidates: the jobs to be executed, the nodes of graph etc.
 - A function that checks whether a particular set of candidates provides a solution, ignoring questions of optimality for the time being.
 - A function that checks whether a set of candidates is feasible (i.e. whether it is possible to obtain at least one solution (not necessarily optimal)).
 - A selection function that indicates at any time which is the most promising of the candidates not yet used.
 - An objective function that gives the value of a solution



Greedy Algorithms

```
function greedy ( $C : set$ ) : set
    {  $C$  is the set of all the candidates }
     $S \leftarrow \emptyset$  {  $S$  is a set in which we construct the solution }
    while not solution ( $S$ ) and  $C \neq \emptyset$  do
         $x \leftarrow$  an element of  $C$  maximizing select( $x$ )
         $C \leftarrow C \setminus \{x\}$ 
        if feasible ( $S \cup \{x\}$ ) then  $S \leftarrow S \cup \{x\}$ 
    if solution ( $S$ ) then return  $S$ 
        else return “there are no solutions” .
```



Greedy Algorithms

- To solve an optimization problem, greedy algorithm looks a set of candidates constituting a solution that optimizes (minimizes or maximizes) the values of the objective function.
- A greedy algorithm proceeds step by step.
- Initially, the set of chosen candidates is empty. Then at each step, selection function is used to add the best remaining candidate to this set.
- If the updated (enlarged) set of chosen candidates is no longer feasible, the candidate that is just added is removed and is never consider again.
- Each time a best remaining candidate is added to the set of chosen candidates and we check whether the set now constitute a solution to our problem.



Greedy Algorithms

- It is easy to see why such algorithms are called greedy: at every step, the procedure chooses the best morsel it can swallow without worrying about the future. It never changes its mind: once a candidate is included in the solution, it is there for good; once a candidate is excluded from the solution, it is never considered.
- The selection function is usually based on the objective function; they may even be identical.



Greedy Algorithms

Example We want to give change to a customer using the smallest possible number of coins. The elements of the problem are

- the candidates: a finite set of coins, representing for instance 1, 5, 10, and 25 units, and containing at least one coin of each type ;
- a solution: the total value of the chosen set of coins is *exactly* the amount we have to pay ;
- a feasible set: the total value of the chosen set *does not exceed* the amount to be paid ;
- the selection function: choose the highest-valued coin remaining in the set of candidates ; and
- the objective function: the number of coins used in the solution. □



Greedy Algorithms

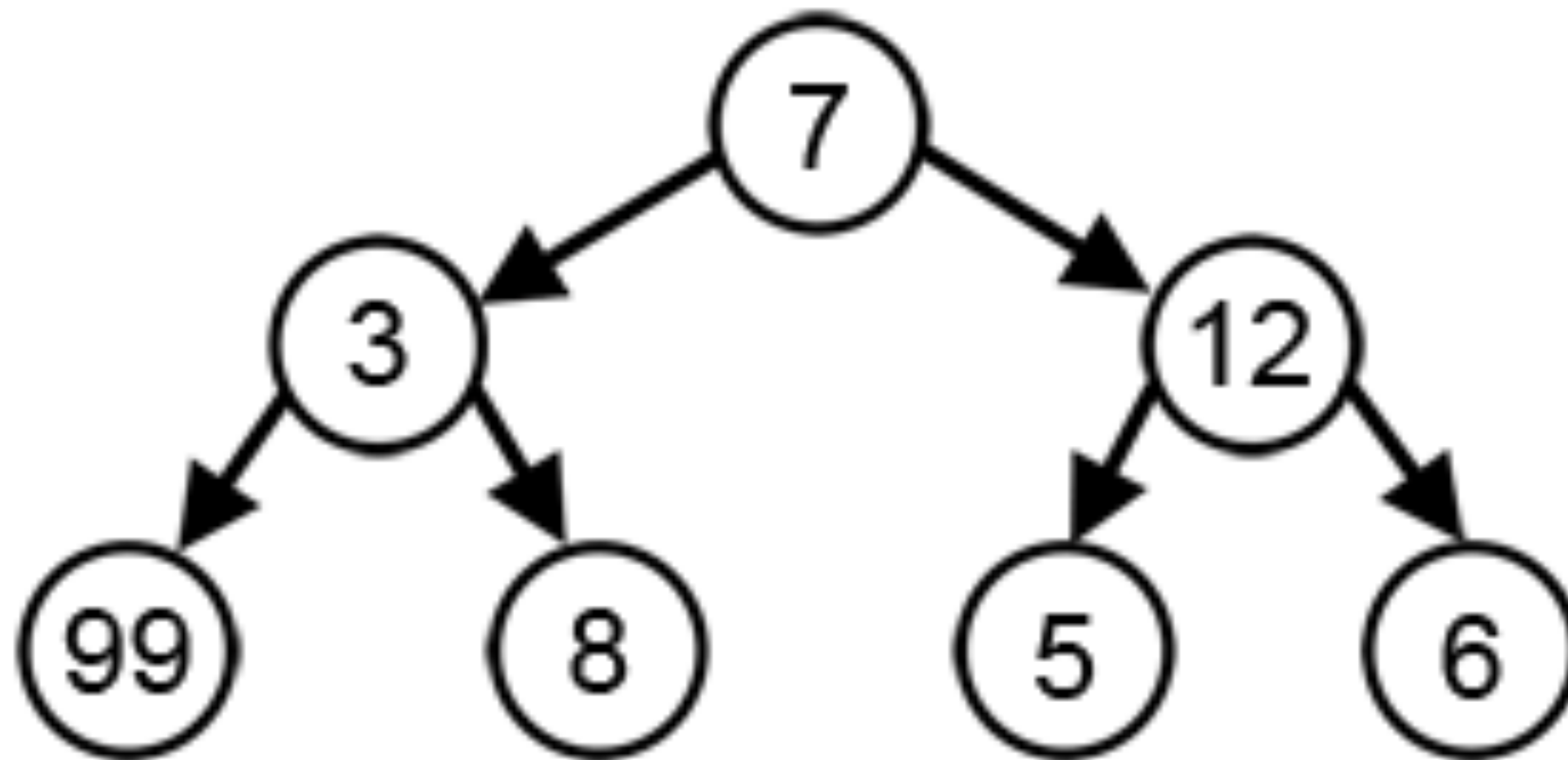
*** Problem** Prove that with the values suggested for the coins in the preceding example the greedy algorithm will always find an optimal solution provided one exists.

Prove, on the other hand, by giving specific counterexamples, that the greedy algorithm no longer gives an optimal solution in every case if there also exist 12-unit coins, or if one type of coin is missing from the initial set. Show that it can even happen that the greedy algorithm fails to find a solution at all despite the fact that one exists. □



Greedy Algorithms

Finding longest path using Greedy Algorithm:



Backtracking Algorithms

- Backtracking is a class of algorithms for finding solutions to some computational problems, notably constraint satisfaction problems, that incrementally builds candidates to the solutions, and abandons a candidate ("backtracks") as soon as it determines that the candidate cannot possibly be completed to a valid solution.
- Many problems which deal with searching for a set of solutions or which ask for an optimal solution satisfying some constraints can be solved using the backtracking strategy
- The term "backtrack" was first coined by American mathematician D. H. Lehmer in the 1950s.



Backtracking Algorithms

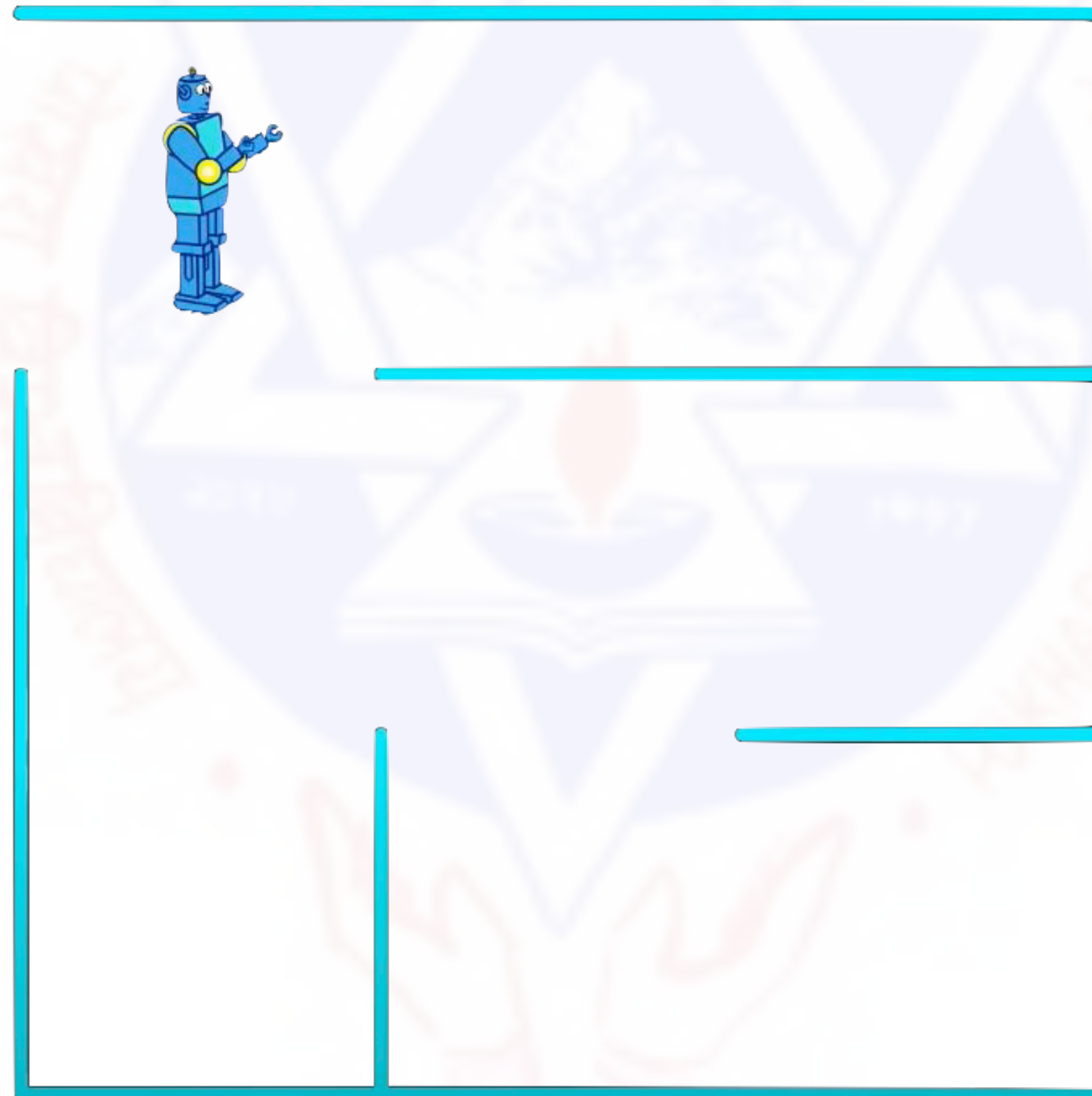
- Backtracking algorithms attempt to complete a search for a solution to a problem by constructing partial solutions, always ensuring that the partial solutions remain consistent with the requirements of the problem. The algorithm then attempts to extend a partial solution toward completion, but when an inconsistency with the requirements of the problem occurs, the algorithm backs up (backtracks) by removing the most recently constructed part of the solution and trying another possibility.



Backtracking Algorithms

Example: Finding a path through a maze.

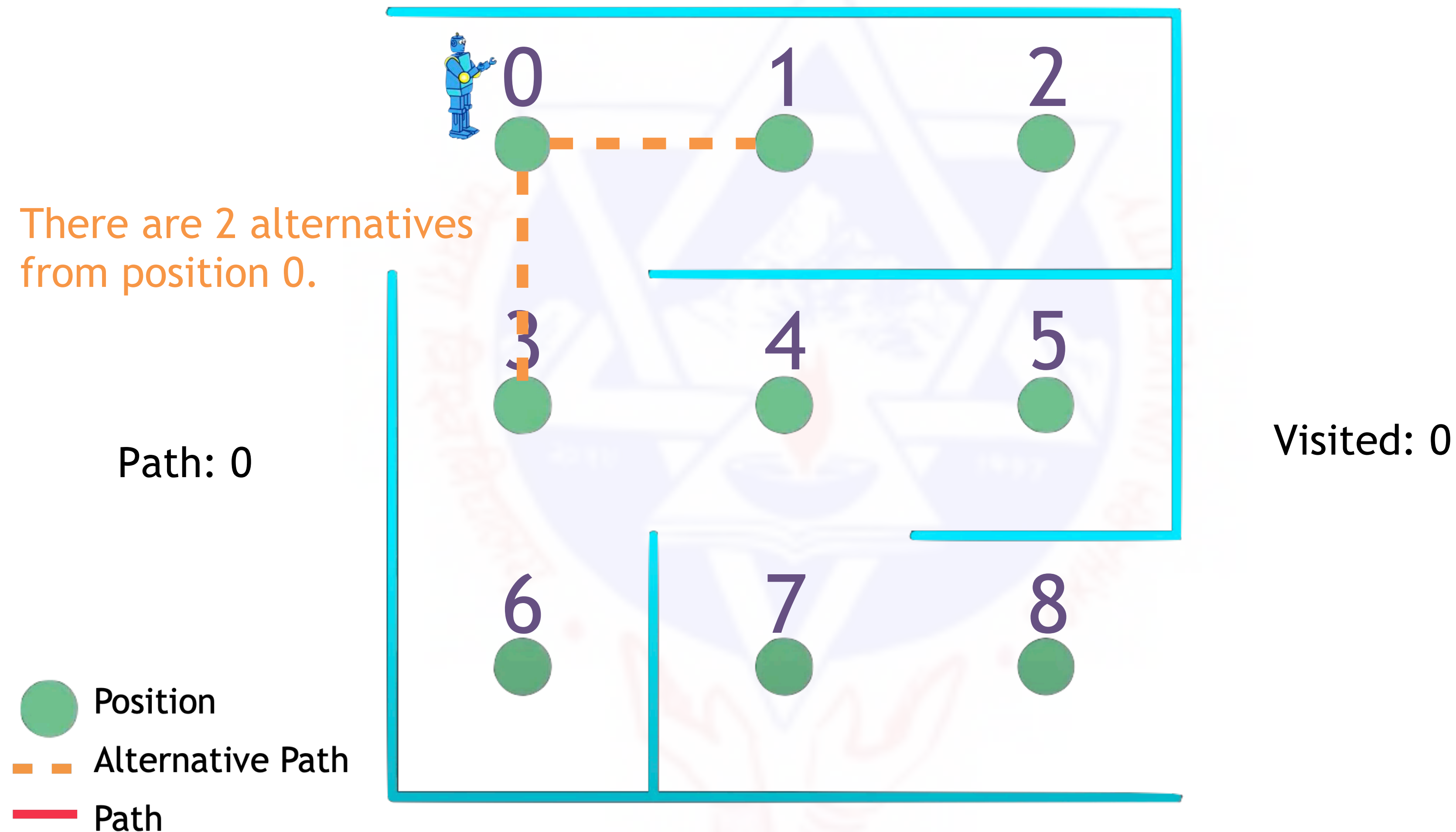
https://www.youtube.com/watch?v=gBC_Fd8EE8A



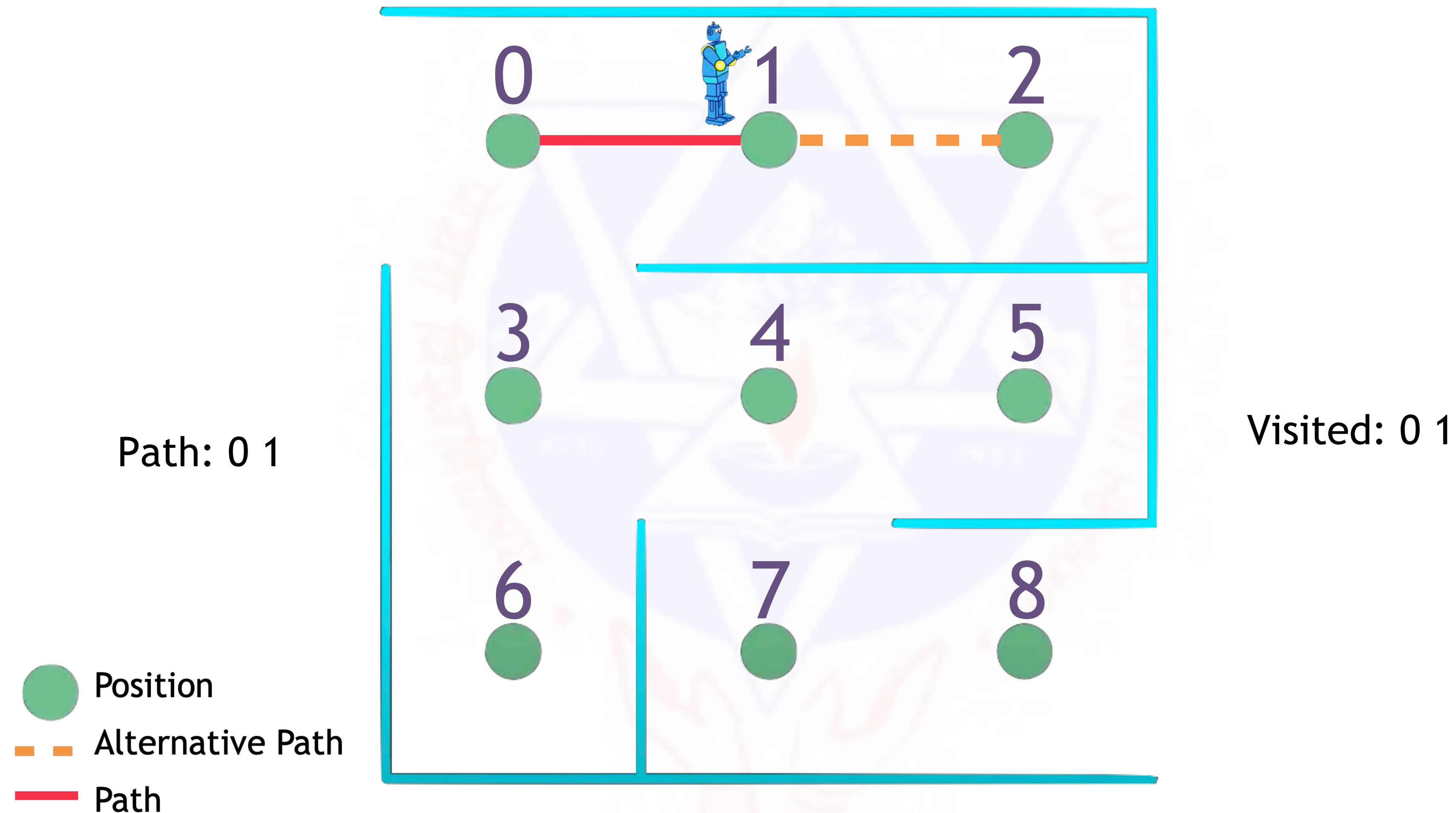
Finish



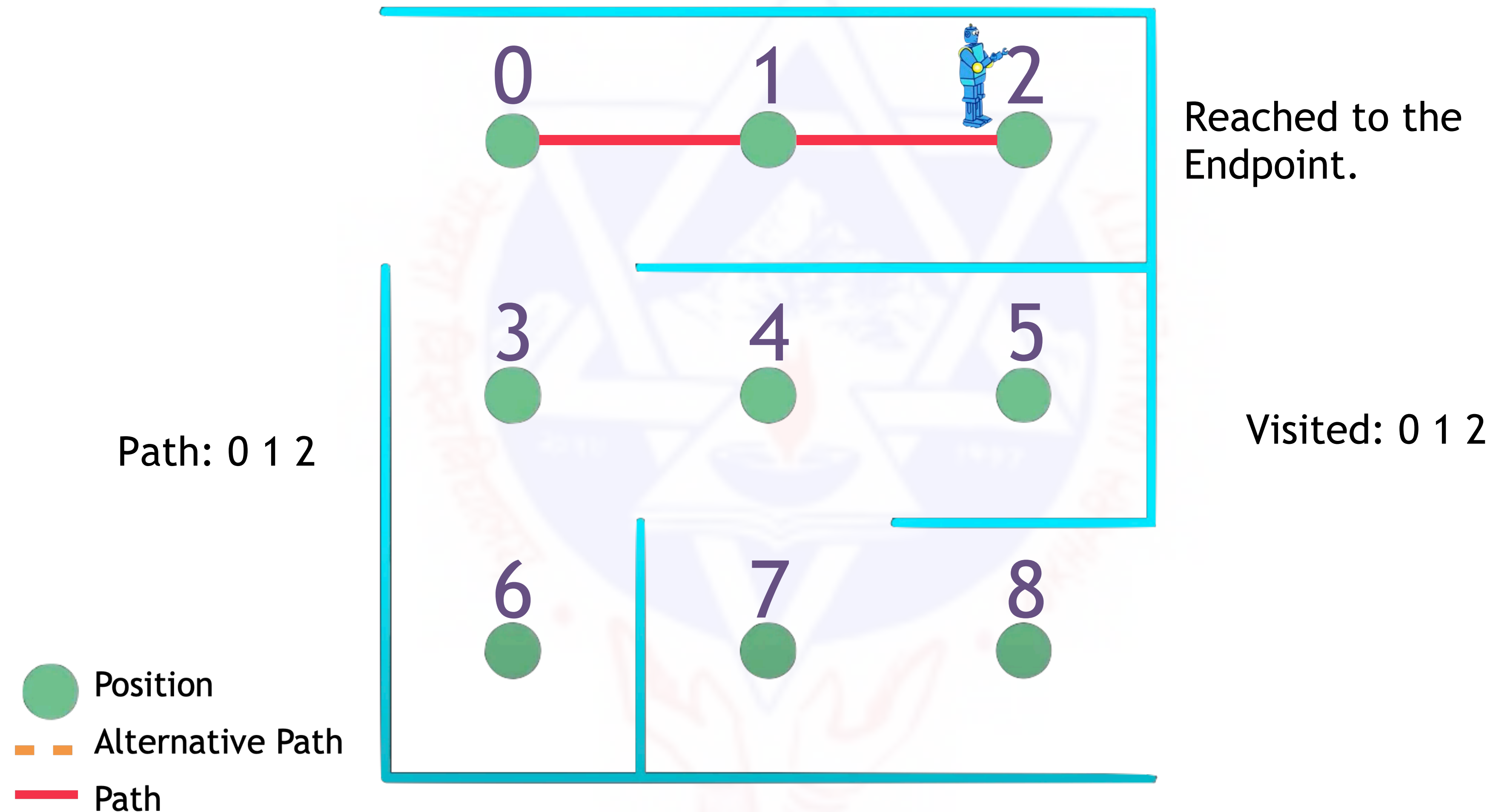
Backtracking Algorithms



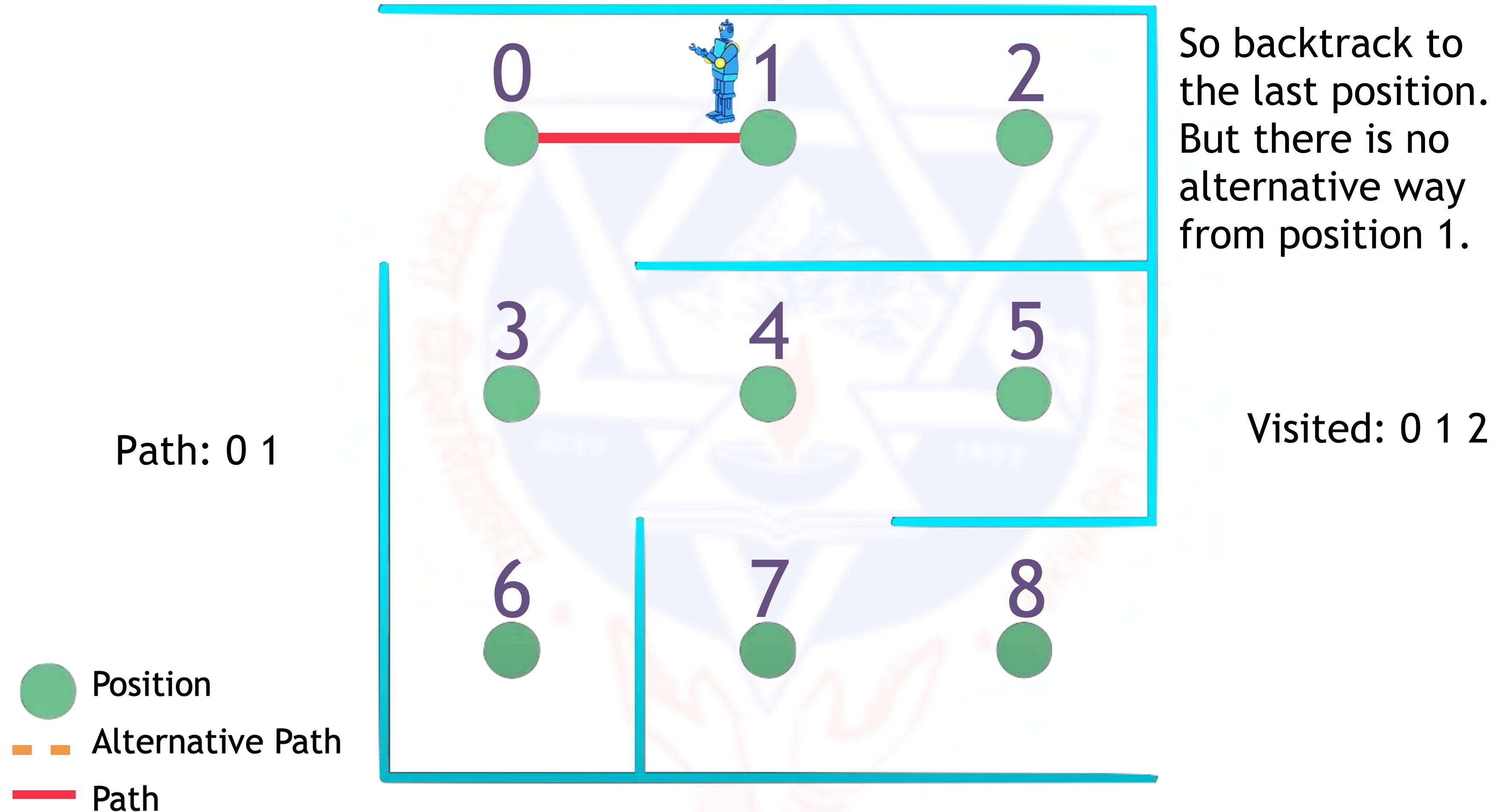
Backtracking Algorithms



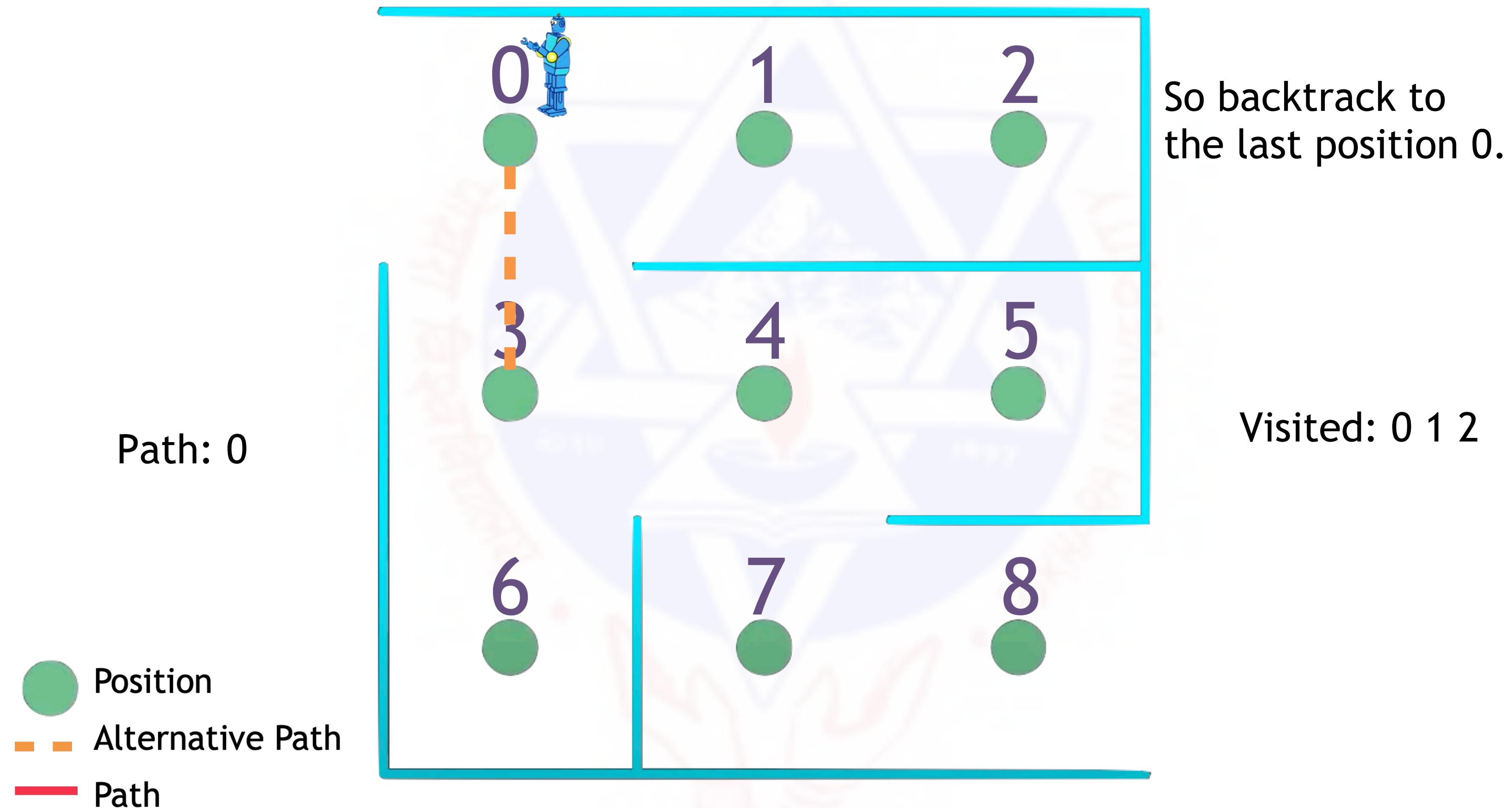
Backtracking Algorithms



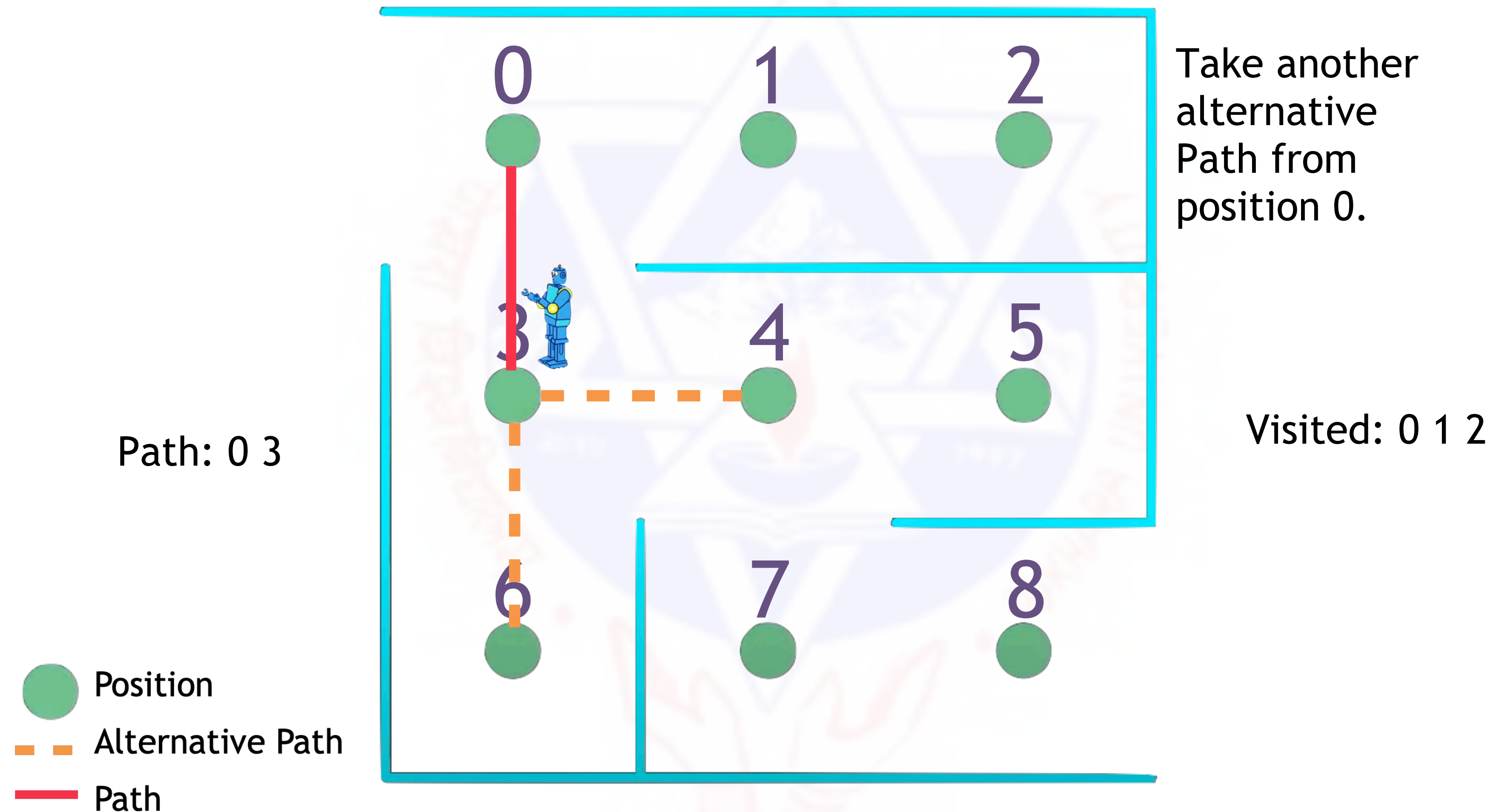
Backtracking Algorithms



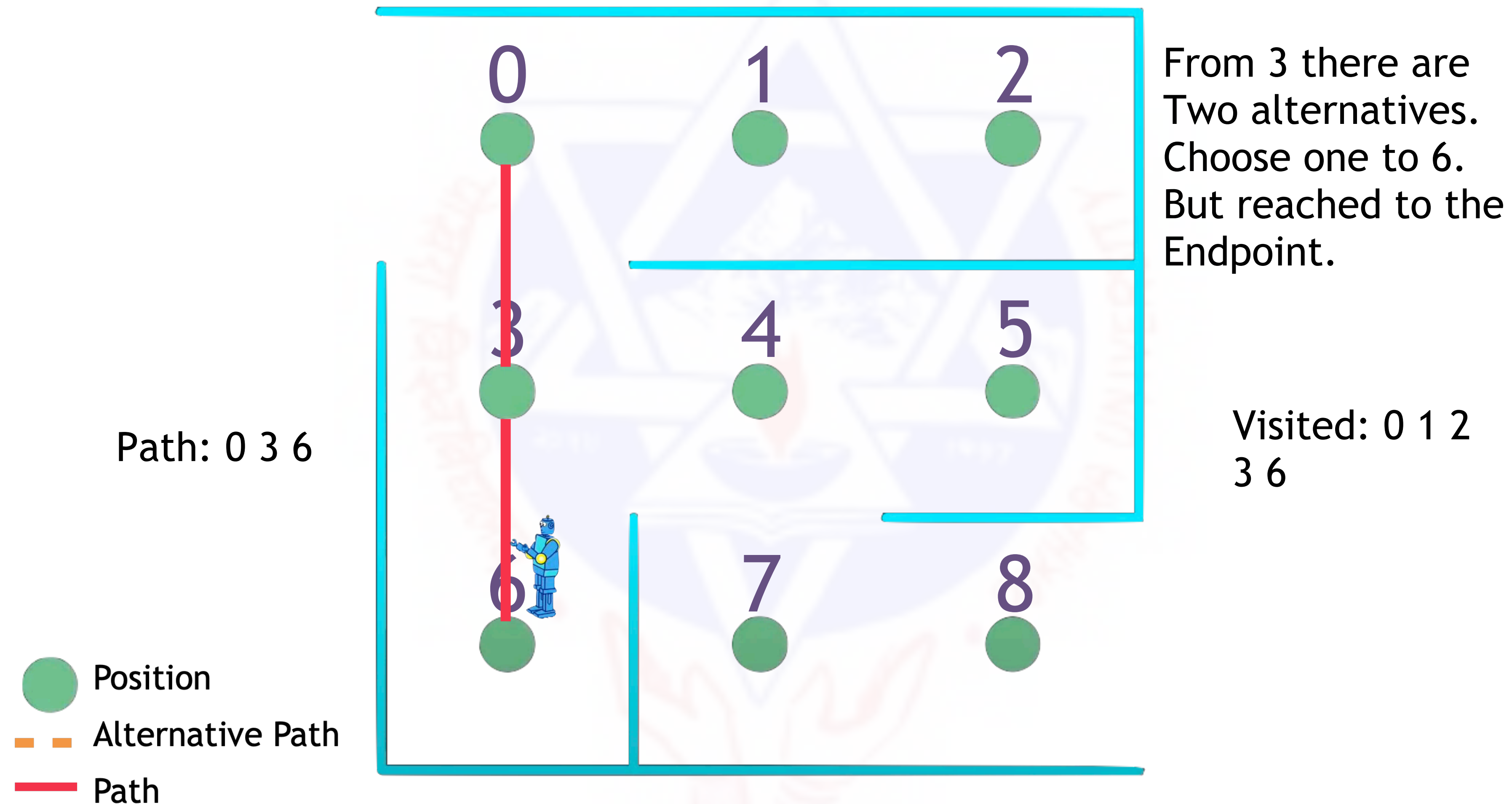
Backtracking Algorithms



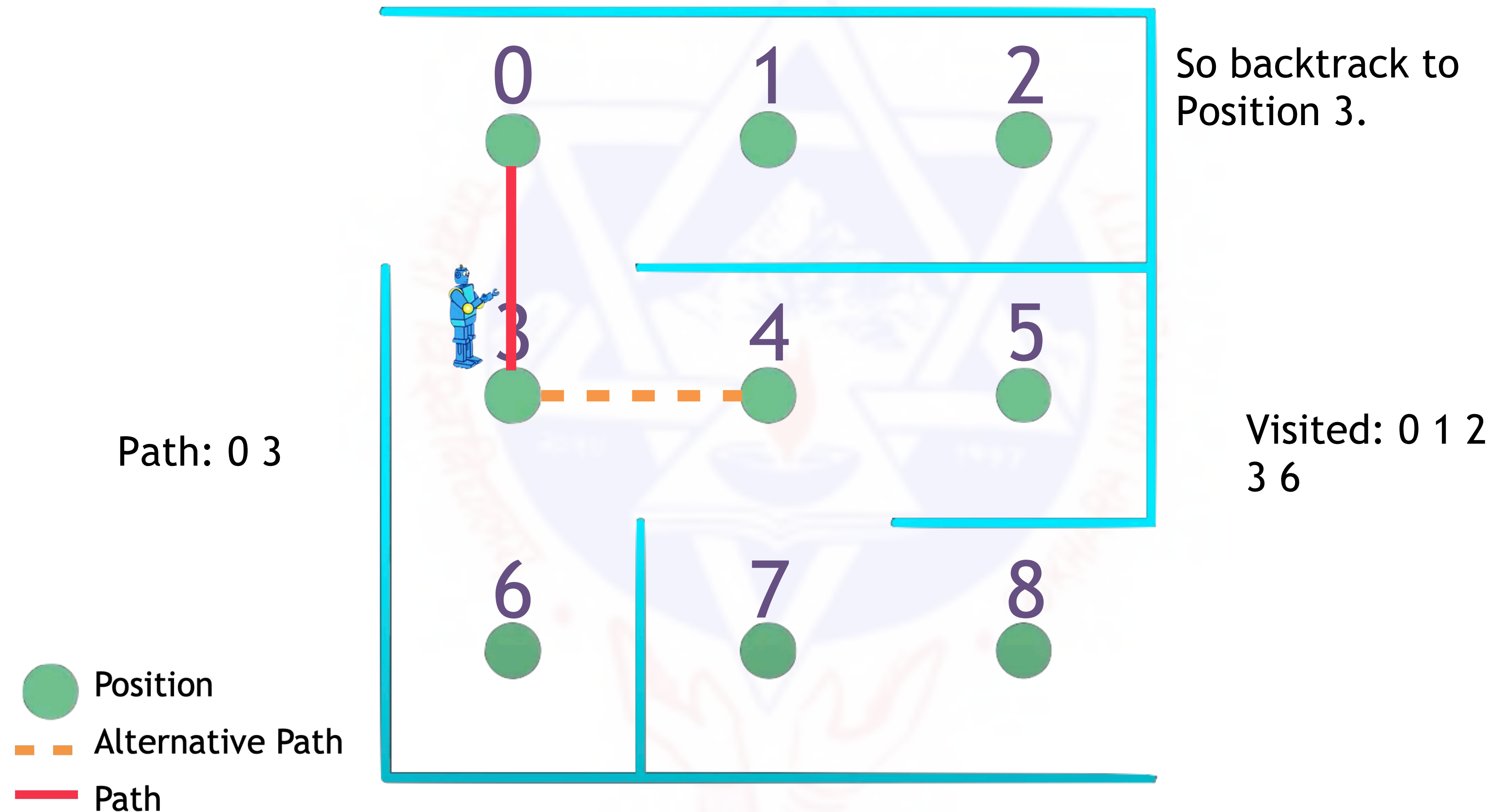
Backtracking Algorithms



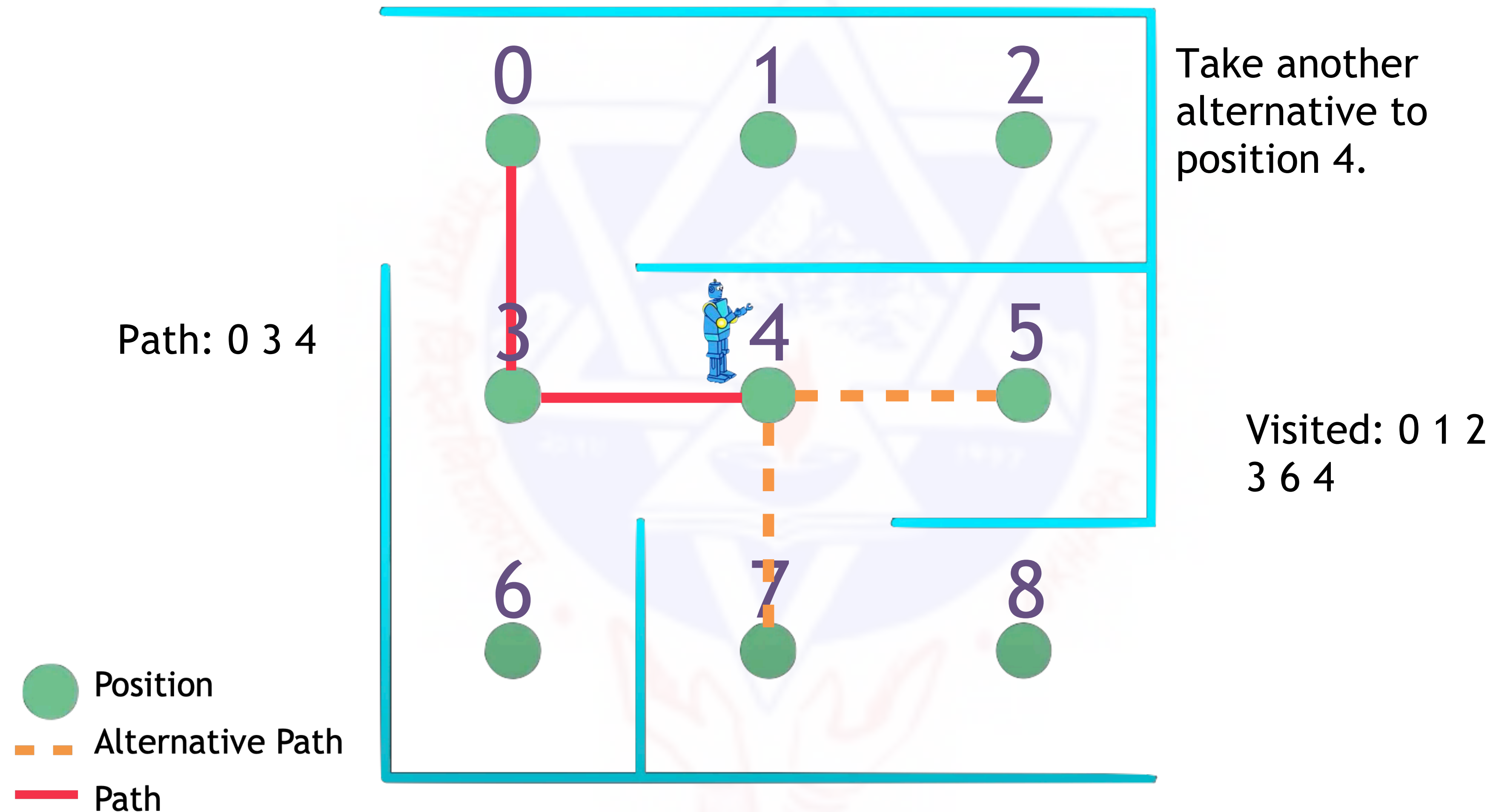
Backtracking Algorithms



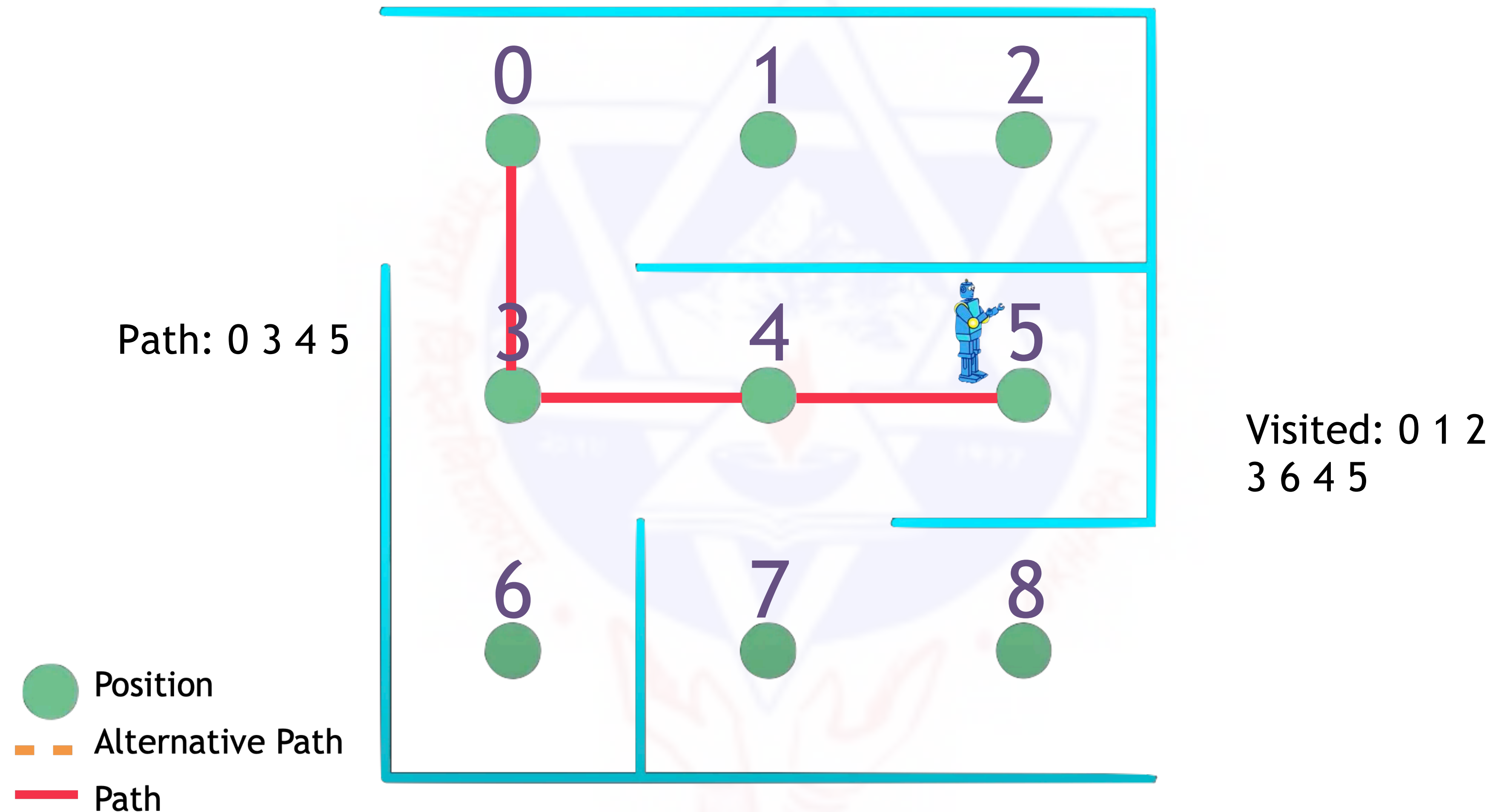
Backtracking Algorithms



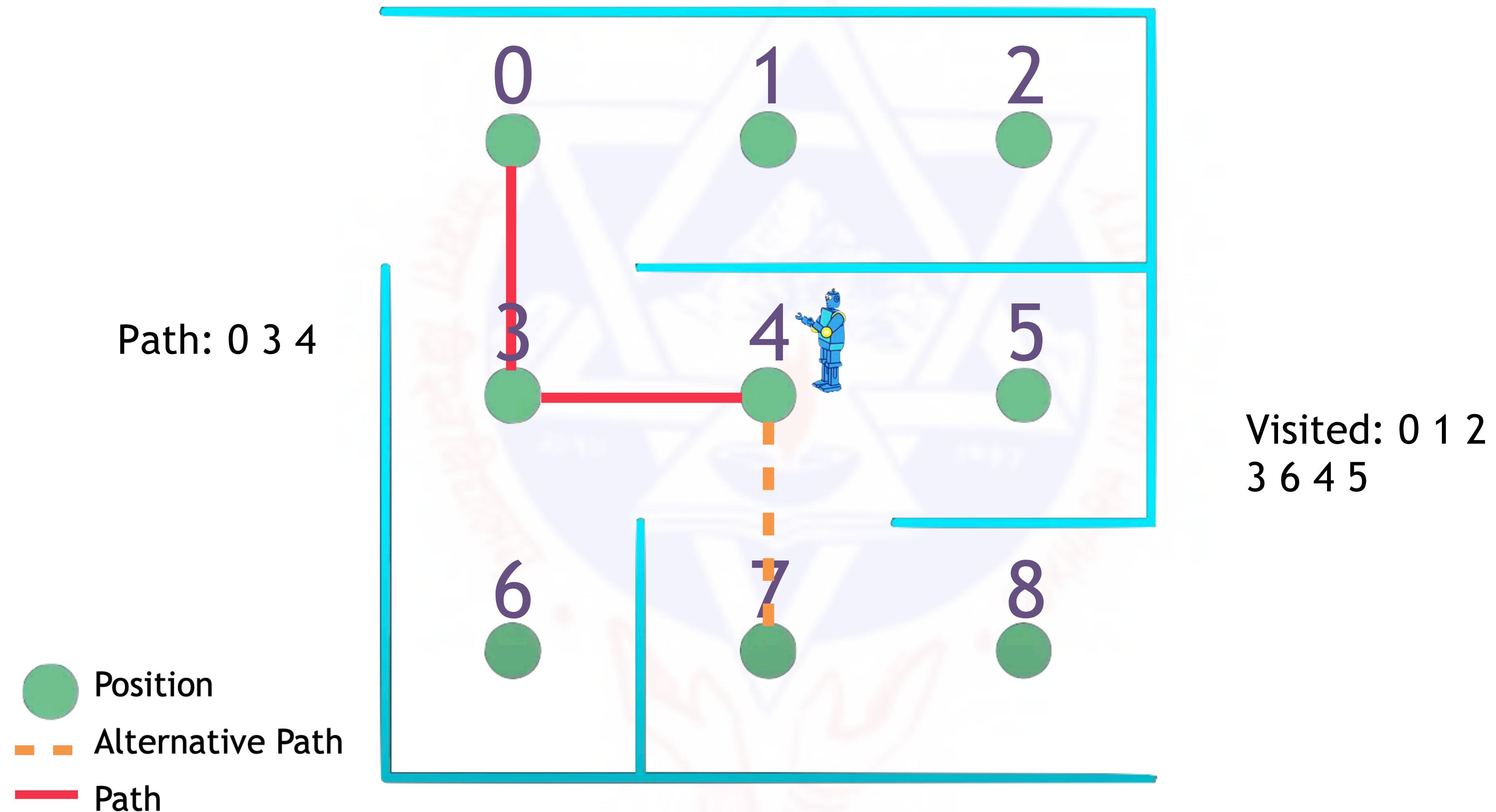
Backtracking Algorithms



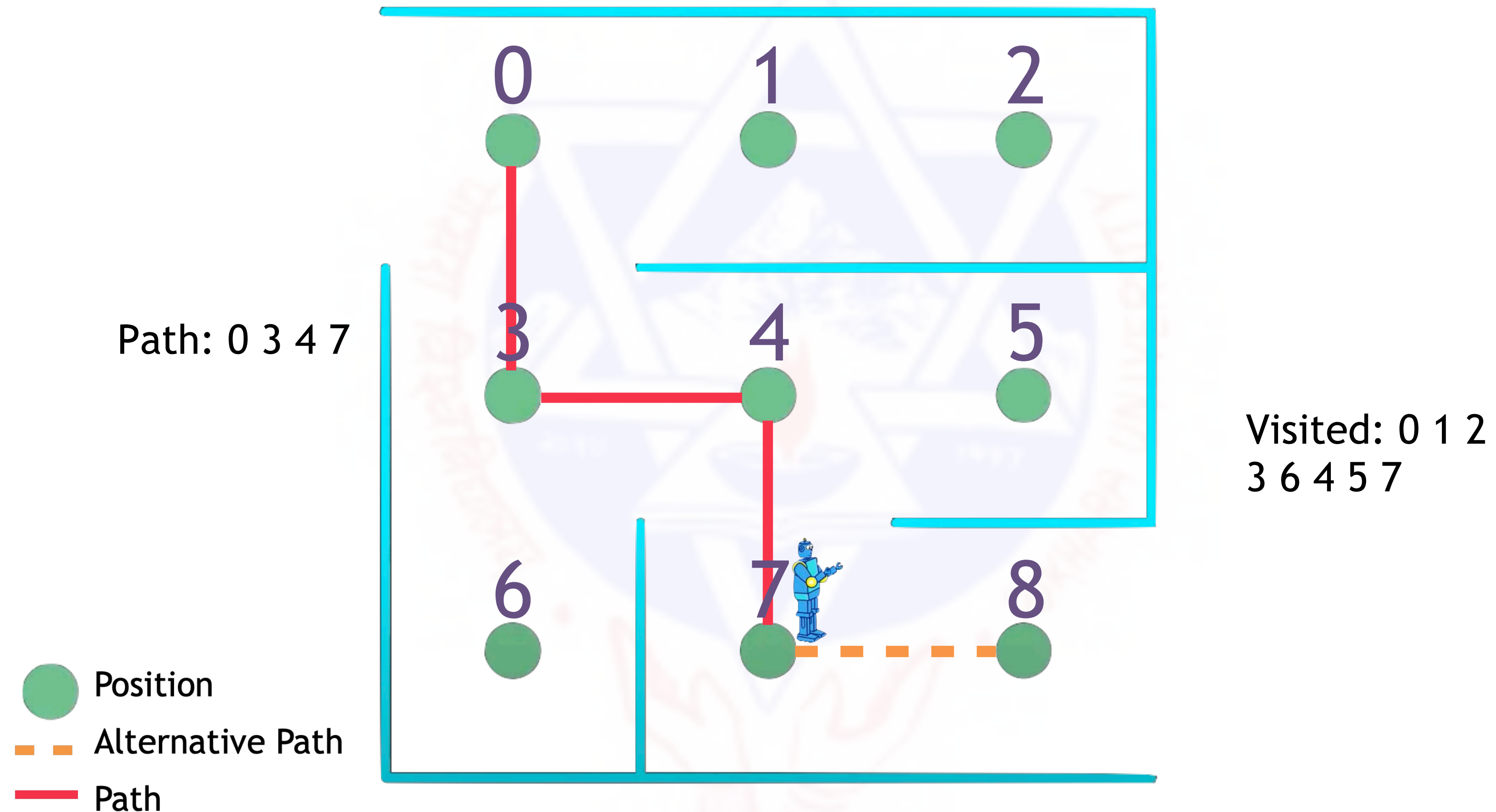
Backtracking Algorithms



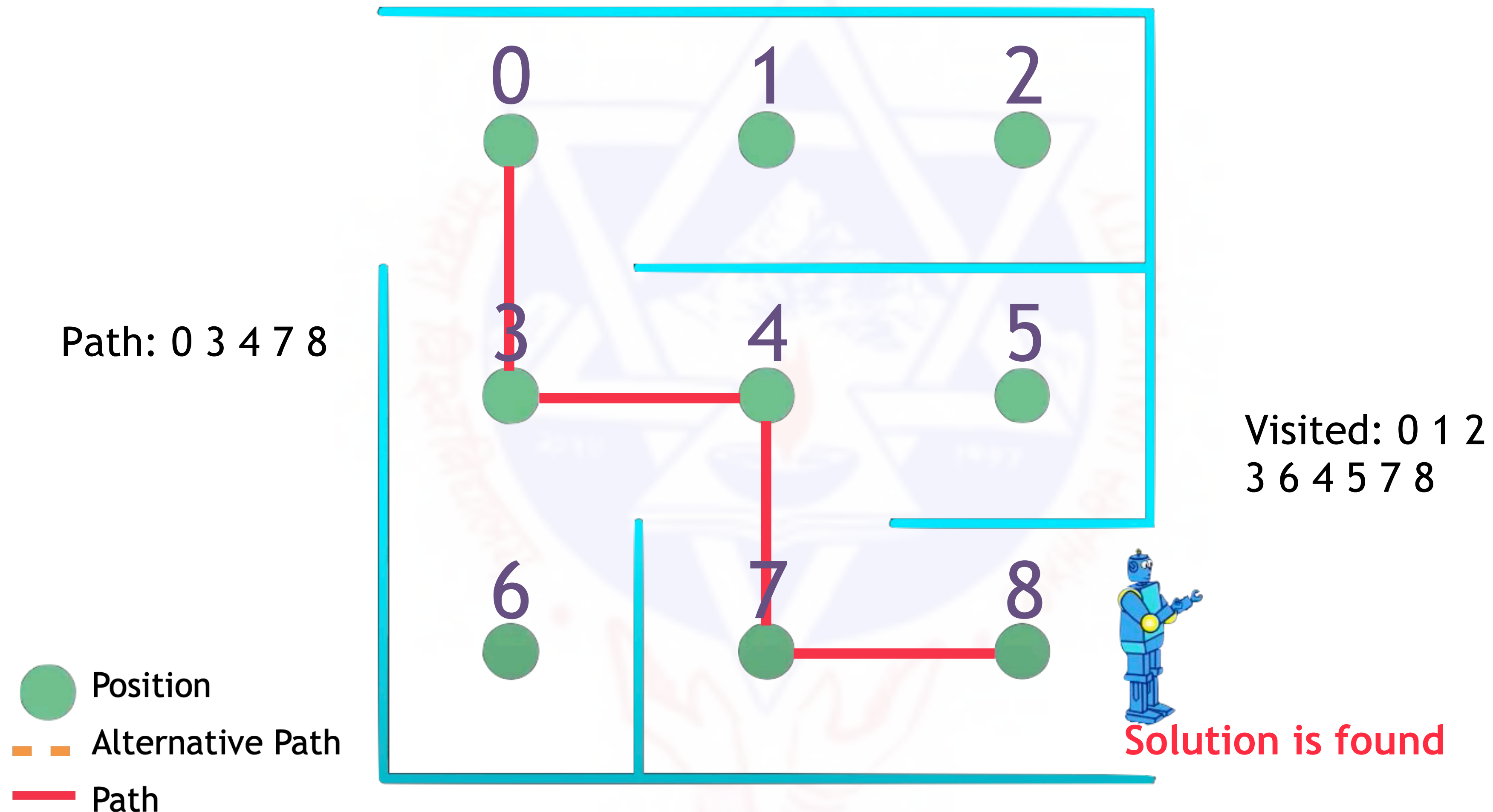
Backtracking Algorithms



Backtracking Algorithms



Backtracking Algorithms



Backtracking Algorithms

- In many applications of the backtrack method, the desired solution is expressed as an n -tuple $(x_1, \dots, x_n)$ where the x_i are chosen from some finite set S_i . Often the problem to be solved calls for finding one vector that maximizes (or minimizes or satisfies) a *criterion function* P .
- **Problem:** Sorting an array of integers $a[n]$. (Simple example although it is not solved by backtracking)
- **Solution:** its solution is expressible by
 - an n -tuple $(x_1, \dots, x_n)$ where x_i is the index in array $a[n]$ of the i^{th} smallest element.
 - a criterion function P is the inequality $a[x_i] \leq a[x_{i+1}]$ for $1 \leq i < n$.
 - a set S_i is a finite set of integers from 1 to n .



Backtracking Algorithms

- Many of the problems we solve using backtracking require that all the solutions satisfy a complex set of constraints. These constraints can be divided into two categories :
 - *explicit*: these constraints are the rules that restrict each x_i to take values from a given set. All the tuples that satisfy the *explicit* constraints define a possible solution space of the problem being solved.
 - *implicit*: these are the rules that determine which of the tuples in the solution space of the problem satisfy the criterion function P . Thus *implicit* constraints describe the way in which the x_i must relate to each other.



Backtracking Algorithms

- **n Queen Problem:** A classic combinatorial problem is to place n queens on an $n \times n$ chess board so that no two "attack" that is, so that no two of them are on the same row, column, or diagonal.
- **Solution:**
 - N queens problem is one of the most common examples of backtracking.
 - Our goal is to arrange N queens on an NxN chessboard such that no queen can attack any other queen.
 - A queen can attack horizontally, vertically, or diagonally.



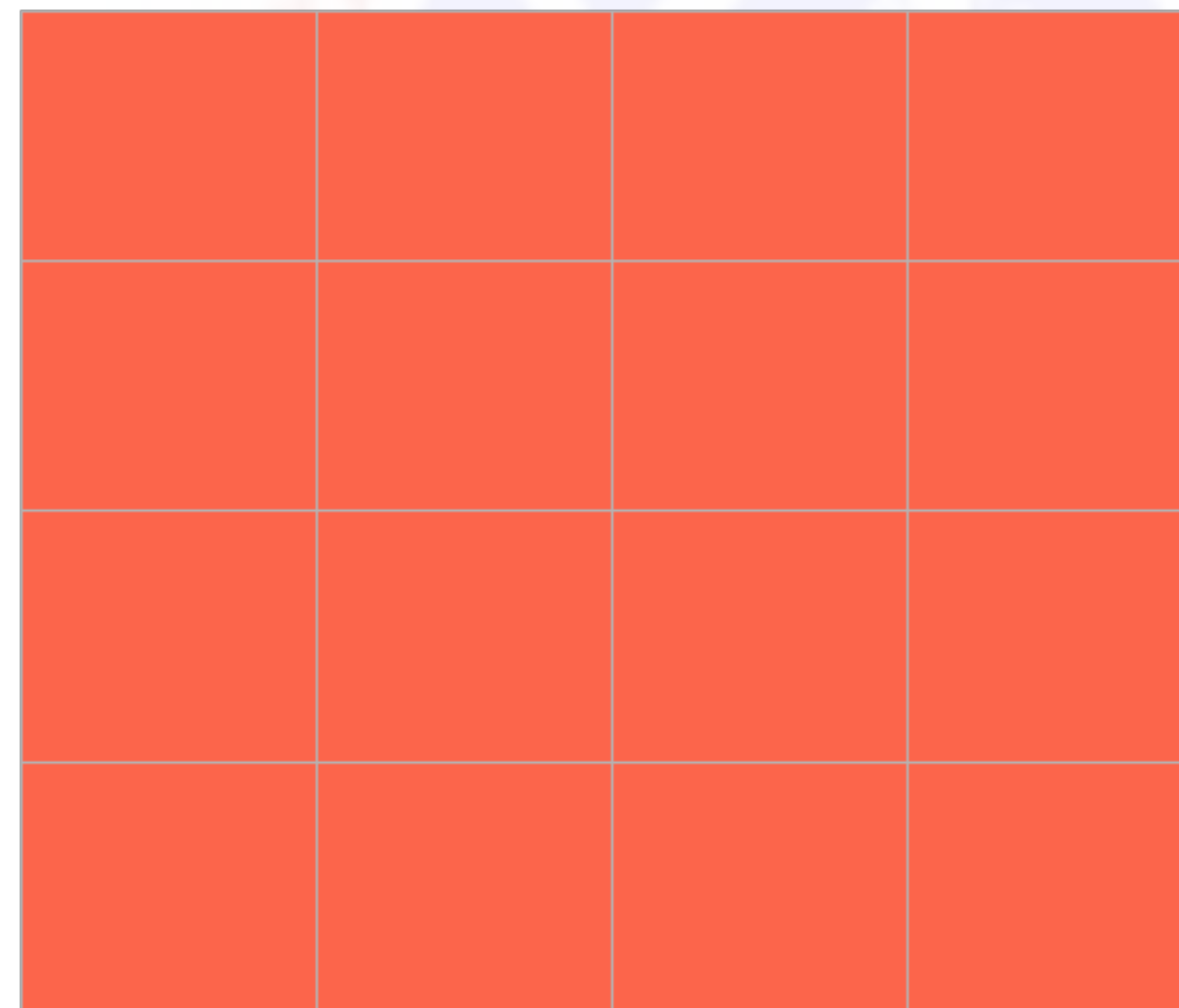
Backtracking Algorithms

- **n Queen Problem:** A classic combinatorial problem is to place n queens on an $n \times n$ chess board so that no two “attack”
- **Solution:**
 1. We start by placing the first queen anywhere arbitrarily and then place the next queen in any of the safe places.
 2. We continue this process until the number of unplaced queens becomes zero (a solution is found) or no safe place is left.
 3. If no safe place is left, then we backtrack and change the position of the previously placed queen.
- Let's test this algorithm on a 4X4 chessboard for 4-queens



4 Queen Problem using Backtracking

- We have a 4x4 chessboard.
- We have to place 4 queens on it.



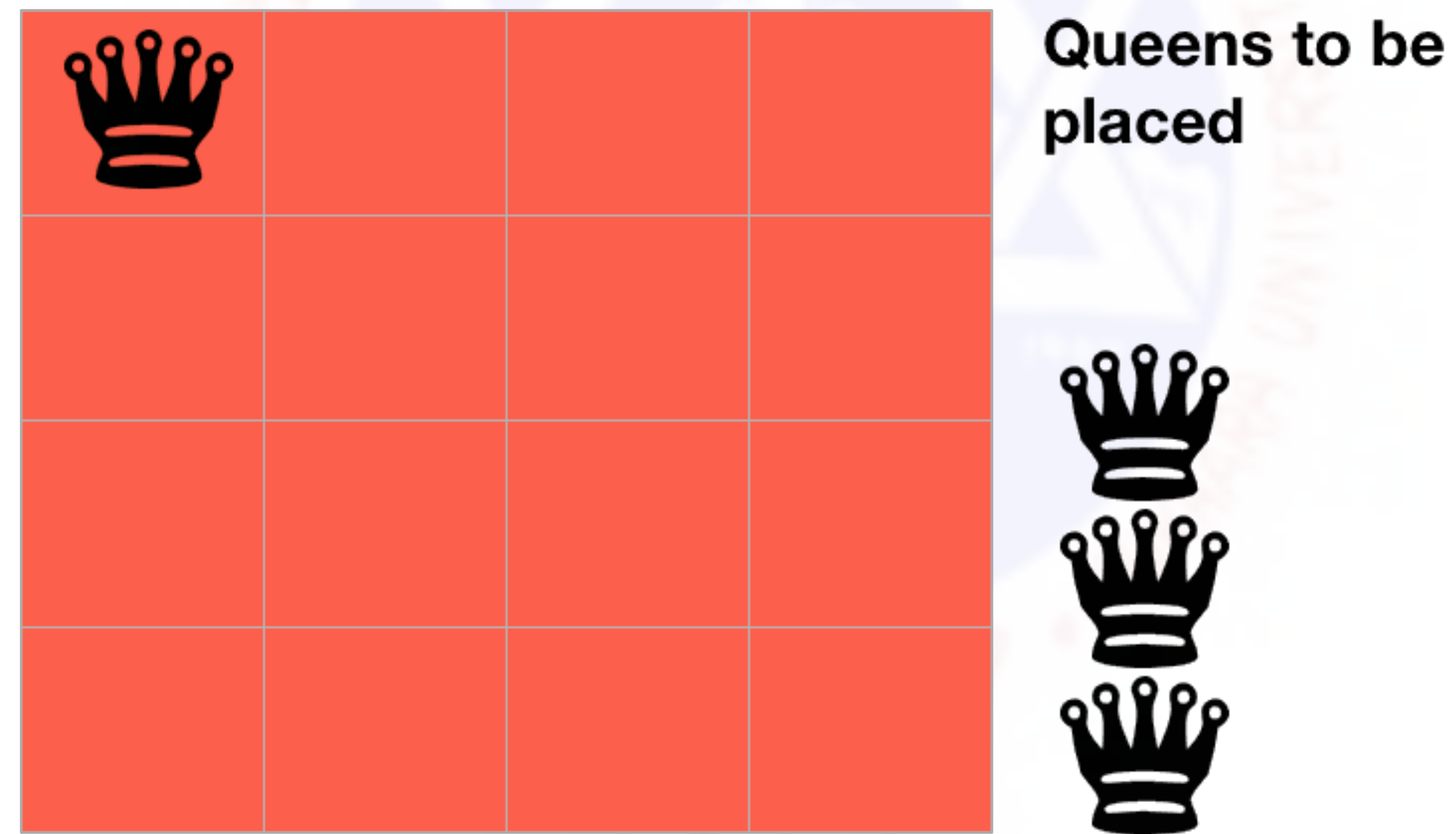
4x4 Chessboard

Queens to be placed



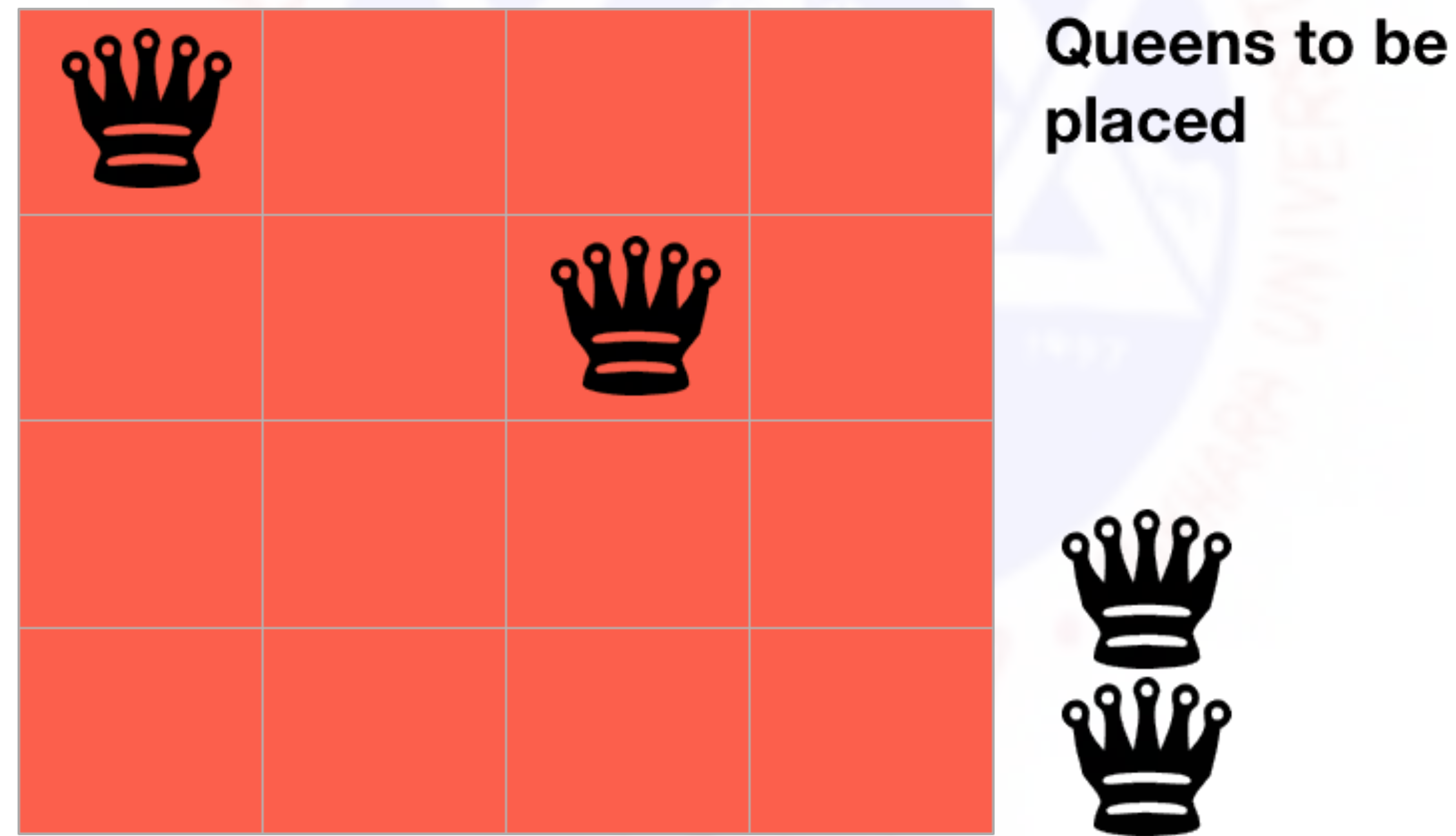
4 Queen Problem using Backtracking

- We will start by placing the first queen in the first row



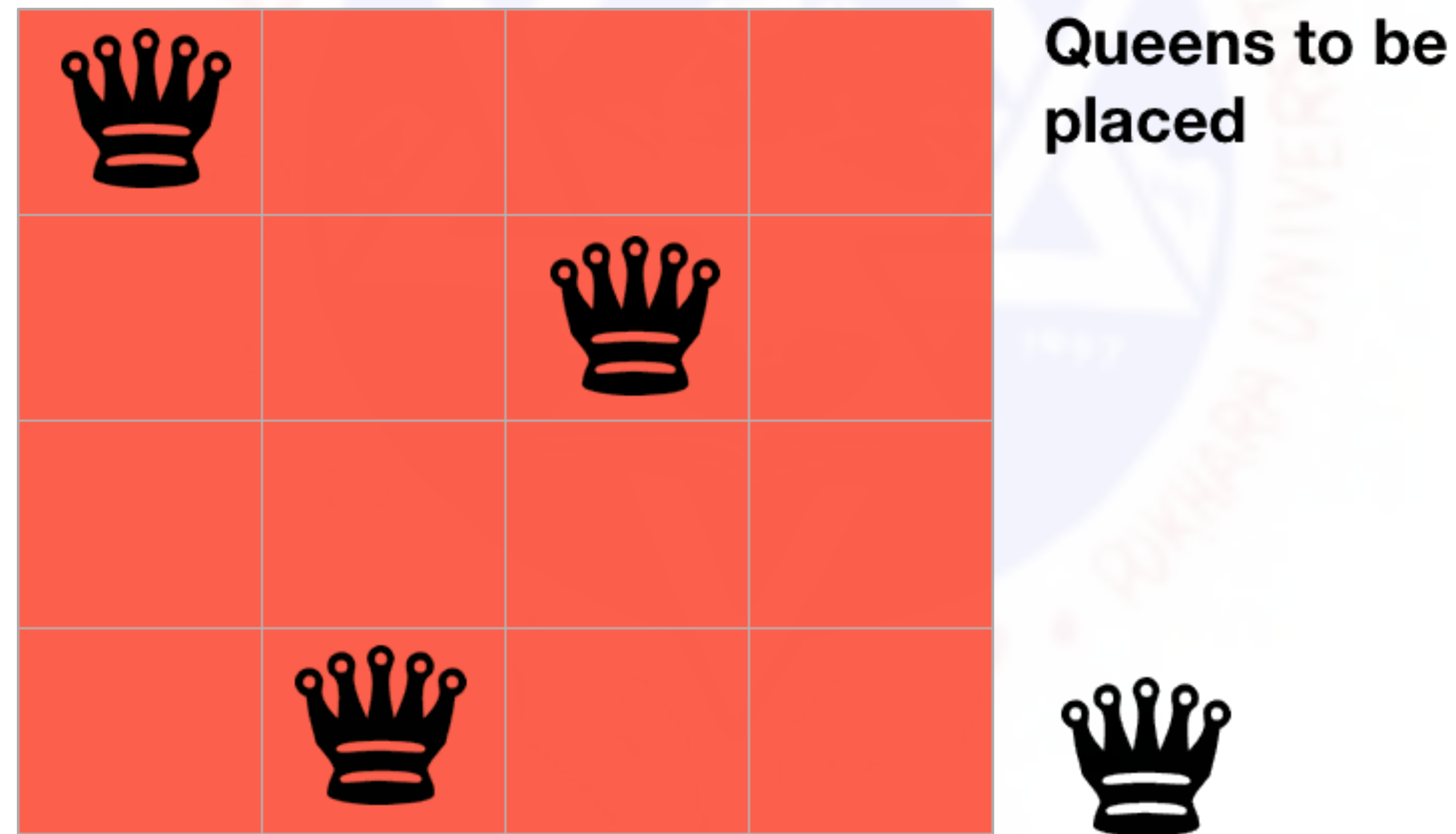
4 Queen Problem using Backtracking

- Now, we have to place the 2nd queen in a safe position. Also, we can't place the queen in the first row, so we will try putting the queen in the second row this time.



4 Queen Problem using Backtracking

- Let us place 3rd queen somewhere in the 3rd row but you can see there is no safe place.
- So we place 3rd queen in 4rth row 2nd column but no safe place is left for the 4th queen.

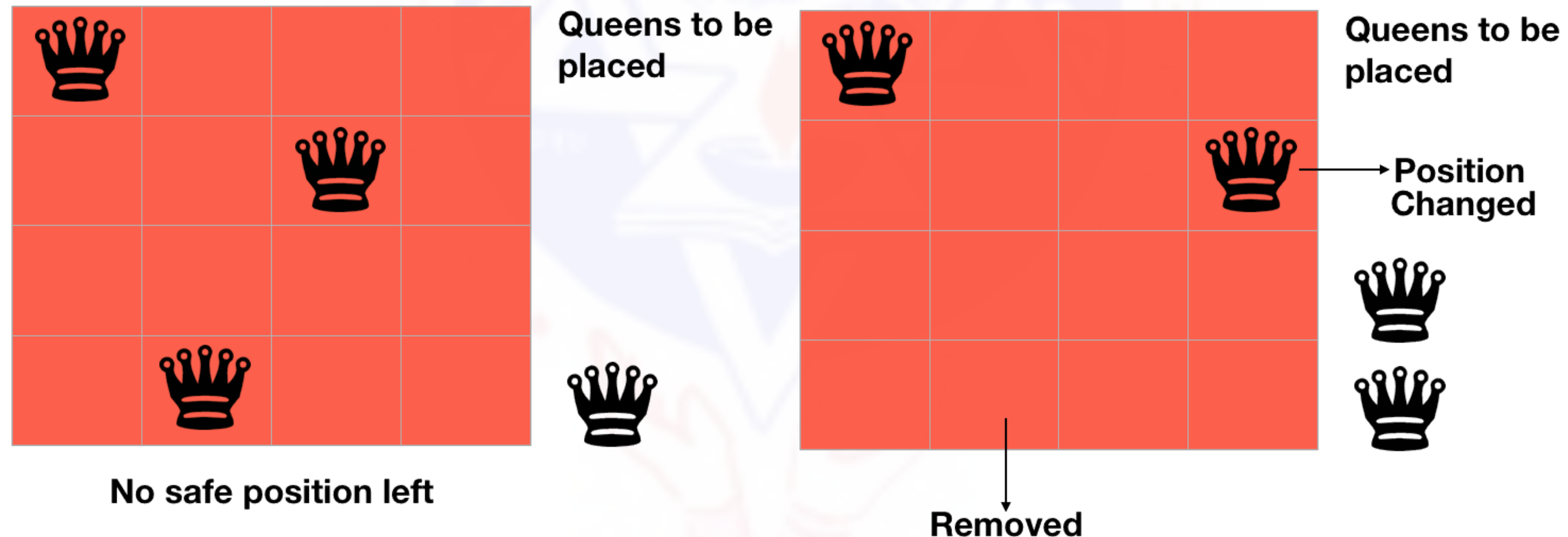


No safe position left



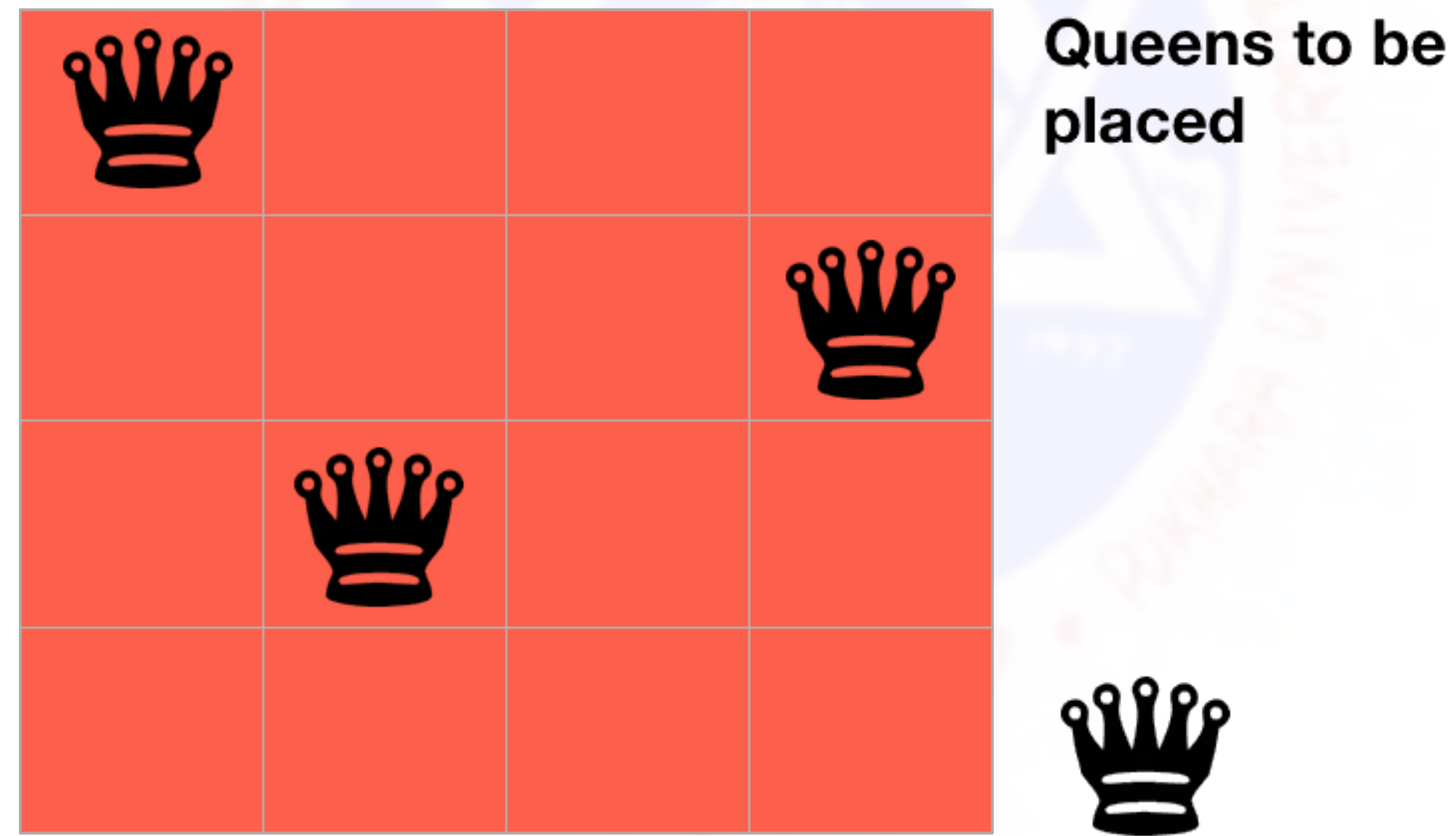
4 Queen Problem using Backtracking

- When we place 3rd queen in 4th row 2nd column, there is no safe position left for the 4th queen. Also, there is no other safe position to place 3rd queen.
- So we backtrack to the step where we placed the 2nd queen and change the position of the 2nd queen



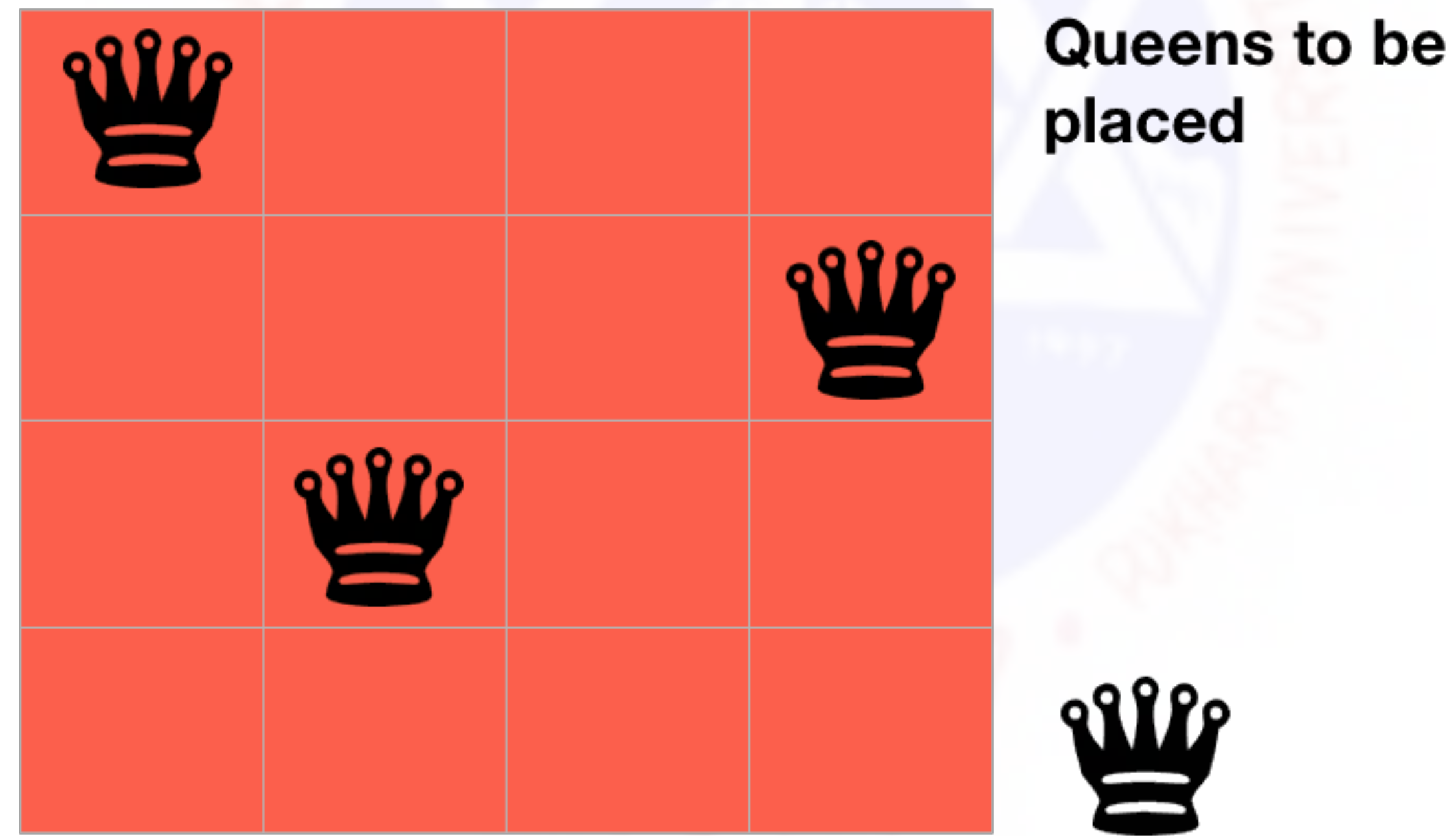
4 Queen Problem using Backtracking

- Again let us place 3rd queen somewhere safe position (3rd row and 2nd column) in the 3rd row.



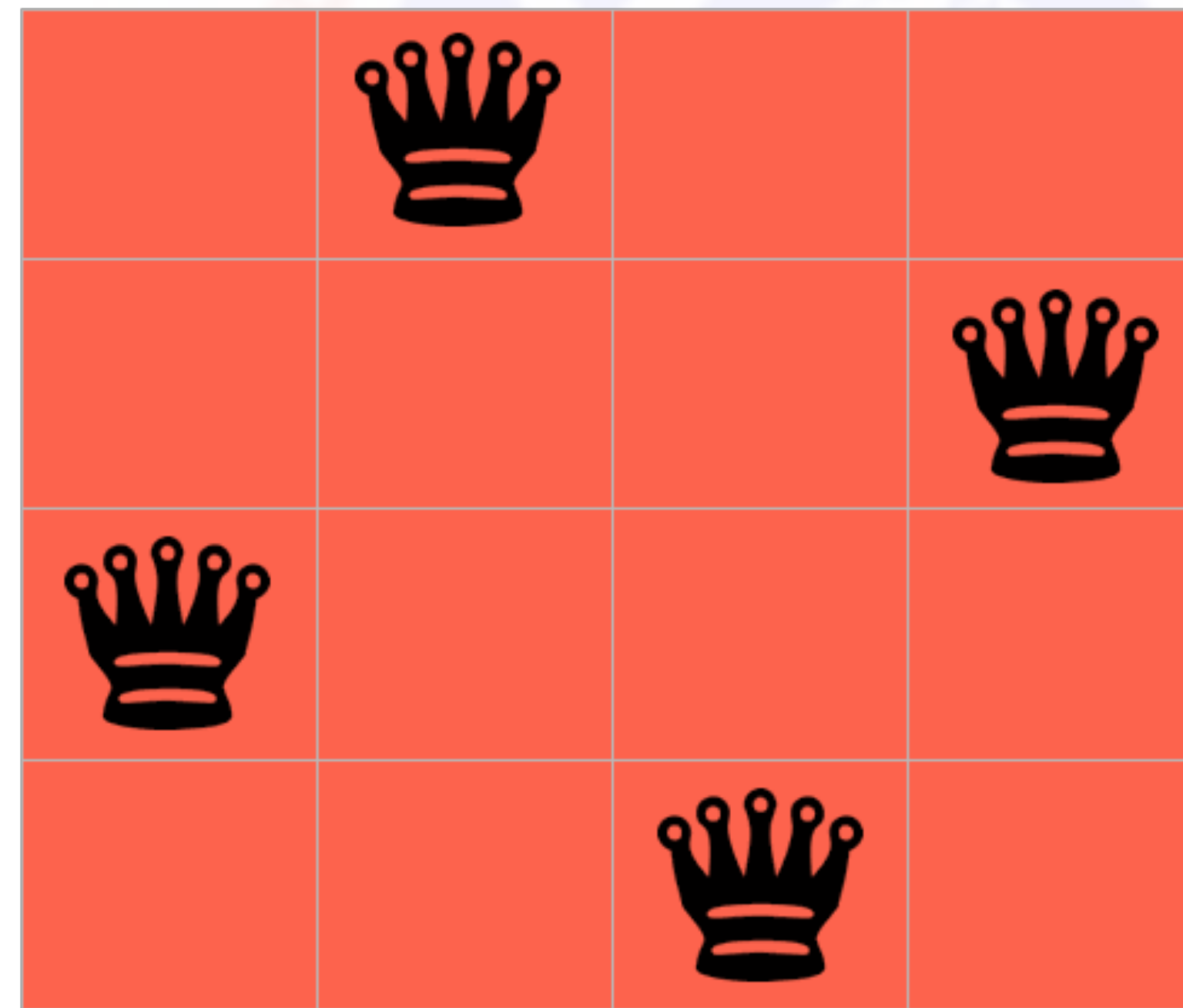
4 Queen Problem using Backtracking

- But again there is no safe position to place the 4th queen.



4 Queen Problem using Backtracking

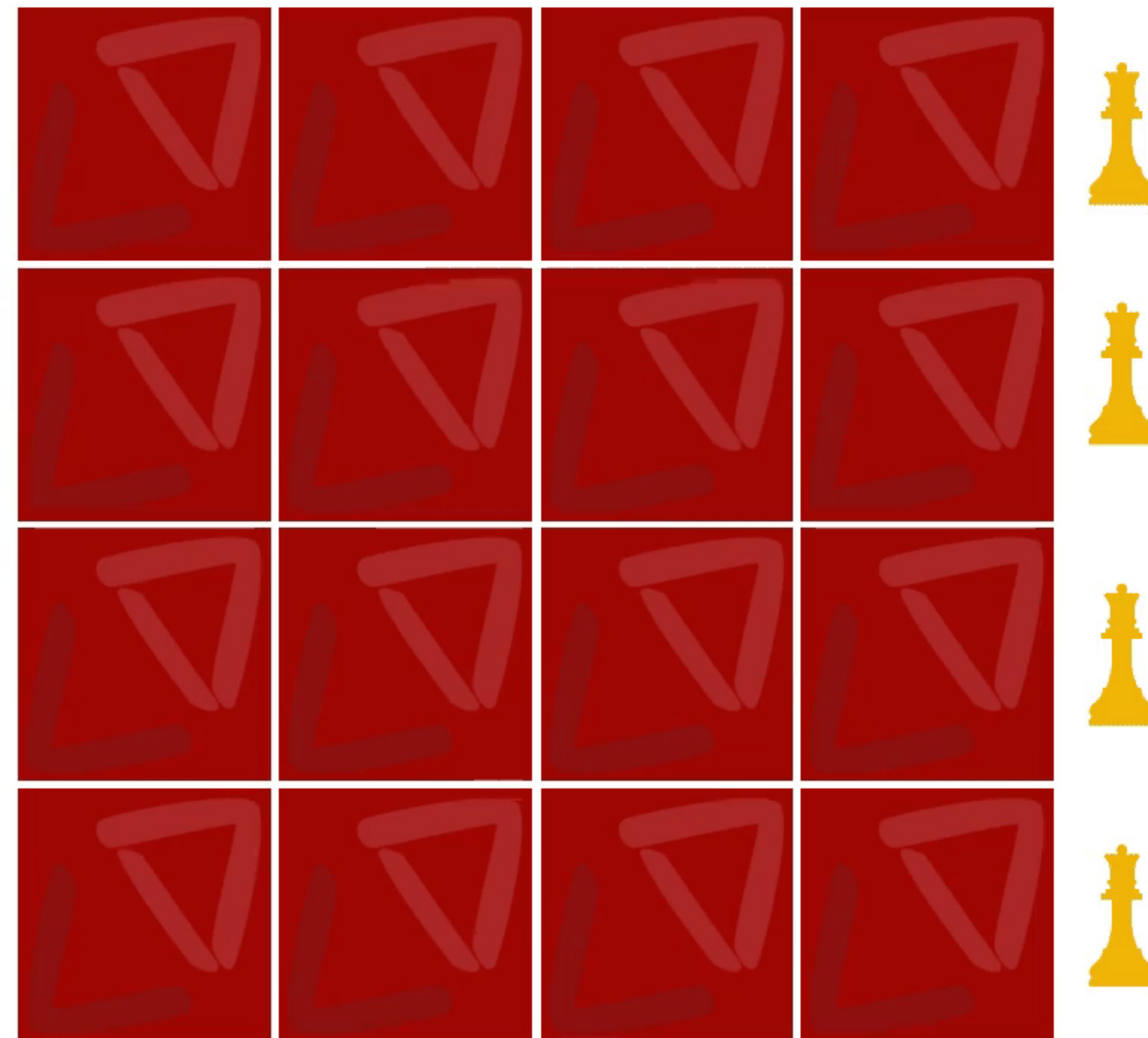
- So we again backtrack to the step in which the 1st queen is placed and repeat the same process until all the 4 queens are placed in safe position.



For coding this n queen problem go to link: <https://www.codesdope.com/course/algorithms-backtracking/>



4 Queen Problem using Backtracking



<https://www.codesdope.com/course/algorithms-backtracking/>



4 Queen Problem using Backtracking

1			

(a)

1			
▪	▪	2	

(b)

1			
		2	
▪	▪	▪	▪

(c)

1			
			2
▪	3		

(d)

1			
			2
	3		
▪	▪	▪	▪

(e)

	1		

(f)

	1		
▪	▪	▪	2

(g)

	1		
			2
3			
▪	▪	4	

(h)

Pseudocode: Backtracking Algorithm

ALGORITHM *Backtrack*($X[1..i]$)

//Gives a template of a generic backtracking algorithm

//Input: $X[1..i]$ specifies first i promising components of a solution

- //Output: All the tuples representing the problem's solutions

if $X[1..i]$ is a solution **write** $X[1..i]$

else

for each element $x \in S_{i+1}$ consistent with $X[1..i]$ and the constraints **do**

$X[i + 1] \leftarrow x$

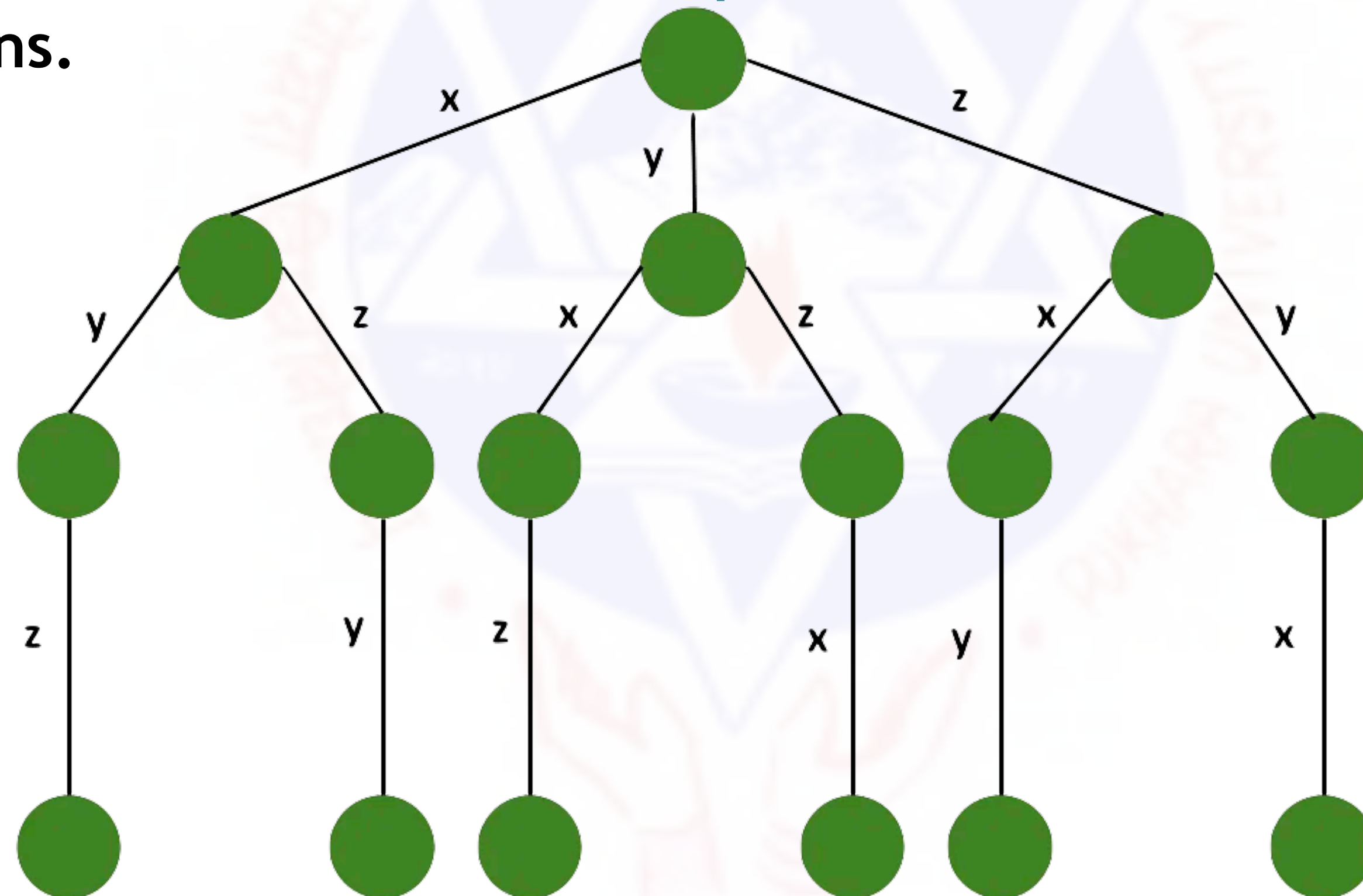
Backtrack($X[1..i + 1]$)

Here, $X[1..i]$ means X is an array with index 1 to i .



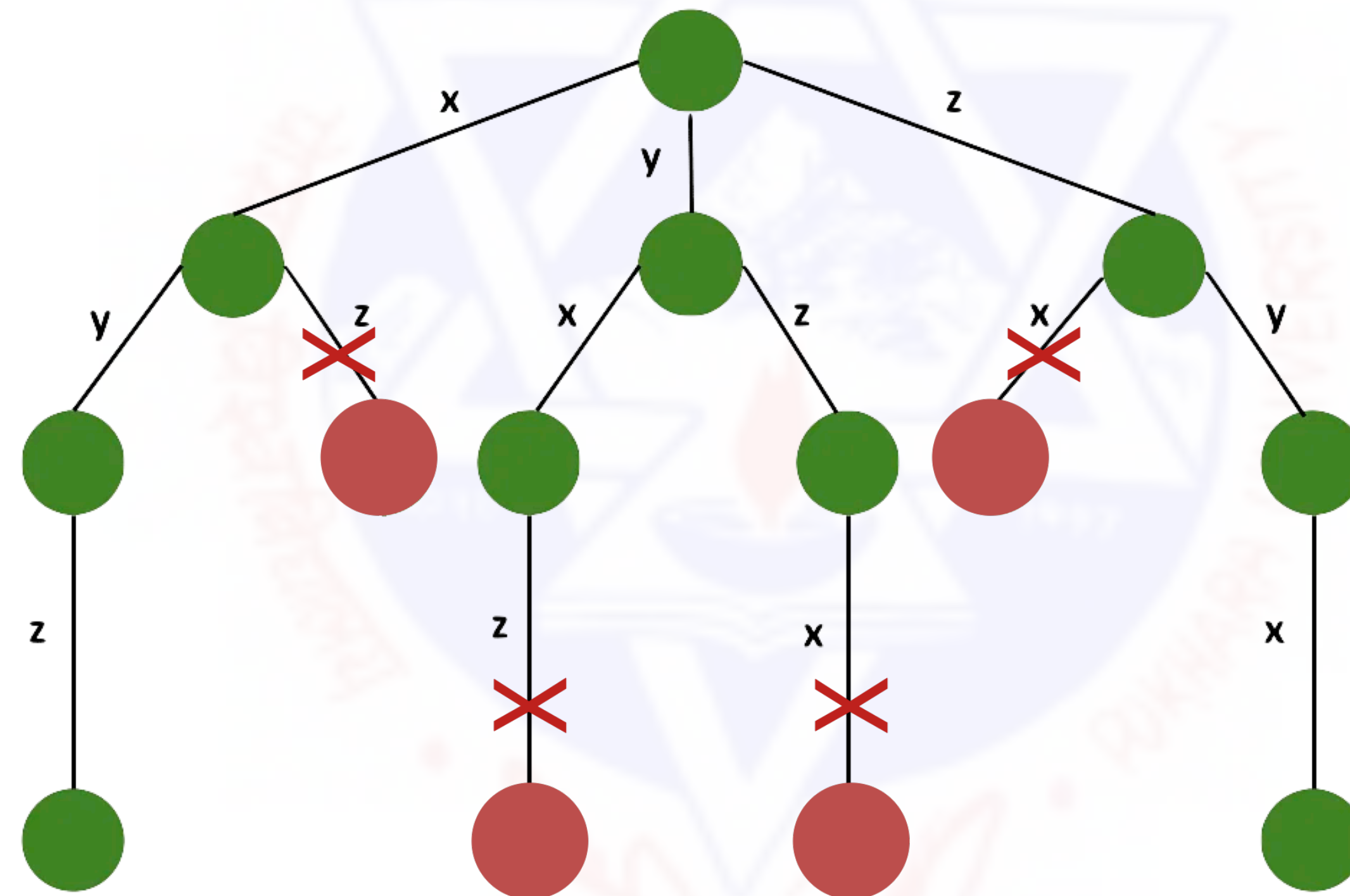
Backtracking Algorithms

- Problem: Arrange the three letters x, y, and z so that **z cannot be next to x**.
- Step 1: First construct a **state-space tree**. Look for all possible solutions.



Backtracking Algorithms

- **Step 2:** Compare the solutions with the given constraint. Choose the solutions that meet the given constraint.



- The valid solutions that satisfy the constraint are (x,y,z) and (z,y,x).

Backtracking Algorithms

- Backtracking is an algorithmic technique that consists of building a set of all the solutions incrementally. Since a problem would have **constraints**, the solutions that fail to satisfy them will be removed.
- It uses **recursive** or **iterative** method to find a solution set by building a solution step by step, increasing levels with time.
- In order to find these solutions, a search tree named **state-space tree** is used. In a state-space tree, each branch is a variable, and each level represents a solution.
- A backtracking algorithm uses the **depth-first search** method. When it starts exploring the solutions, a function is applied so that the algorithm can check if the so-far built solution satisfies the constraints. If it does, it continues searching. If it doesn't, the branch would be eliminated, and the algorithm goes back to the level before and search for next alternative.



Backtracking Algorithms

- There are three types of problems that can be solved by using backtracking:

1. Decision Problems

- For these problems, we search for a feasible solution.

2. Optimization Problems

- For these problems, we search for the best solution.

3. Enumeration Problems

- For these problems, we find all feasible solutions.
- Backtracking algorithms can be recursive or iterative.



Backtracking Algorithms: Applications

- Backtracking is an important tool for solving constraint satisfaction problems, such as crosswords, verbal arithmetic, Sudoku, and many other puzzles.
- It is often the most convenient technique for parsing, for the knapsack problem and other combinatorial optimization problems.
- It is also the basis of the so-called logic programming languages such as Icon, Planner and Prolog.
- Popular Problem that are solved using backtracking
 - To solve N-queen problem
 - To solve Maze problems
 - To solve Knight's Tour problem
 - Sum of subset problem
 - Graph coloring
 - To find all Hamilton paths in a graph



4.Algorithm Analysis:

4.1.Best, Worst and Average Case Analysis

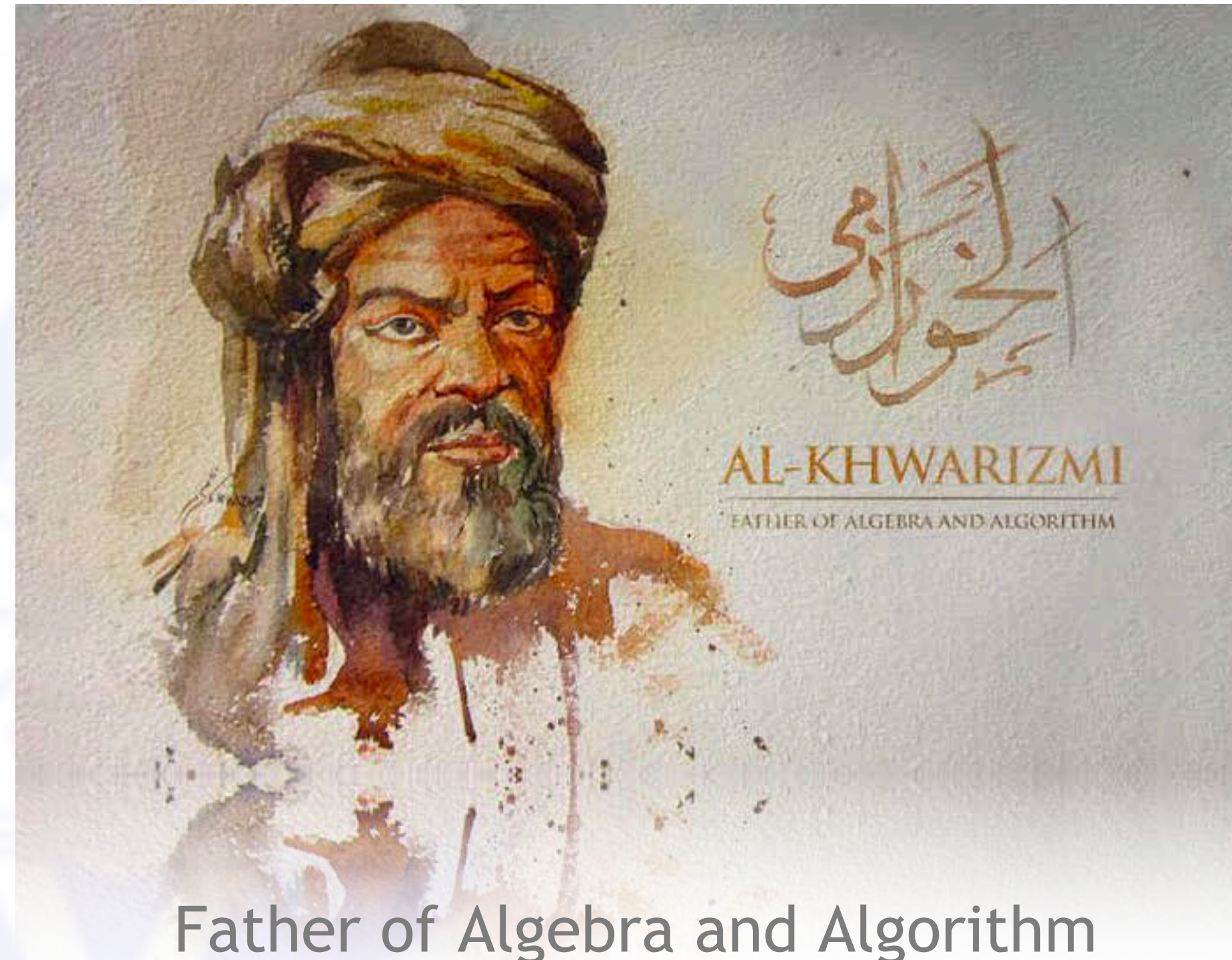
4.2.Rate of Growth

4.3.Asymptotic Notations- Big Oh, Big Omega and Big Theta

ALGORITHM ANALYSIS

What is and algorithm?

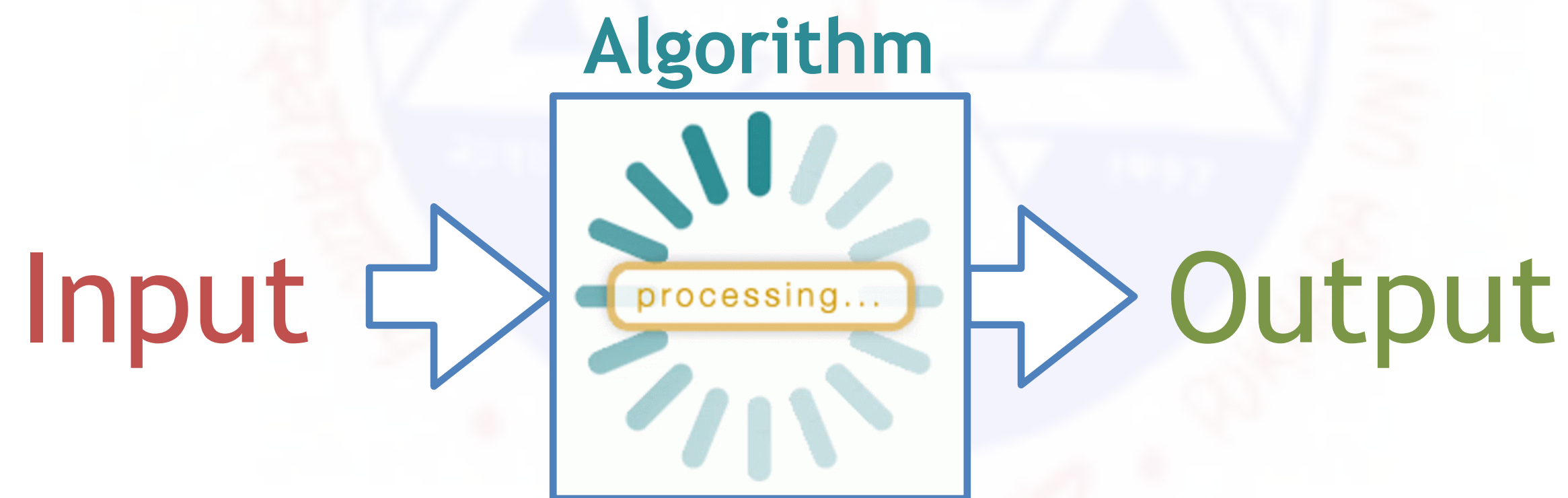
- The word “**algorithm**” came from the name of a Persian mathematician Abu Ja’far Mohammed ibn Musa al-KhowArizmi (c. 825 A.D.).
- Al-Khwārizmī’s contributions to mathematics, geography, astronomy, and cartography established the basis for innovation in algebra and trigonometry.
- On the *Calculation with Hindu Numerals*, written about 820, was principally responsible for spreading the **Hindu-Arabic numeral system** throughout the Middle East and Europe.
- It was translated into Latin as **Algoritmi de numero Indorum**. Al-Khwārizmī, rendered as (Latin) **Algoritmi**, led to the term “**algorithm**”.



<https://www.amust.com.au/2018/01/al-khwarizmi-from-algebra-to-computing/>

What is and algorithm?

- The word “algorithm” has taken on a special significance in computer science where algorithm has come to refer to a method that can be used by a computer for the solution of a problem.
- An algorithm is a finite set of instructions that performs a particular task.
- Mathematically, an algorithm is a method consisting of a sequence of steps (that maps input to output.) that transform an input to the corresponding output.



- Algorithms are systematic methods for solving problems. The methods that you learn at school for multiplying or dividing integers are algorithms.



Characteristics of Algorithms

- All algorithm must satisfy the following criteria:
 1. **Input:** Zero or more quantities are externally supplied.
 2. **Output:** At least one quantity is produced.
 3. **Definiteness:** Each instruction in algorithm must be clear and unambiguous.
 4. **Finiteness:** An algorithm must terminate after a finite number of steps.
 5. **Concrete steps:** An algorithm must contain a series of concrete steps.
Concrete means that the action described by a step in algorithm is completely understood and doable.
 6. **Correctness:** An algorithm must be correct and must produce desired output for a given input.
 7. **Feasible:** An algorithm must be **feasible** in terms of technical point of view or in terms of cost.
 8. **Language independent:** An algorithm must be **language independent** and its steps can be implemented using the code of any programming languages.



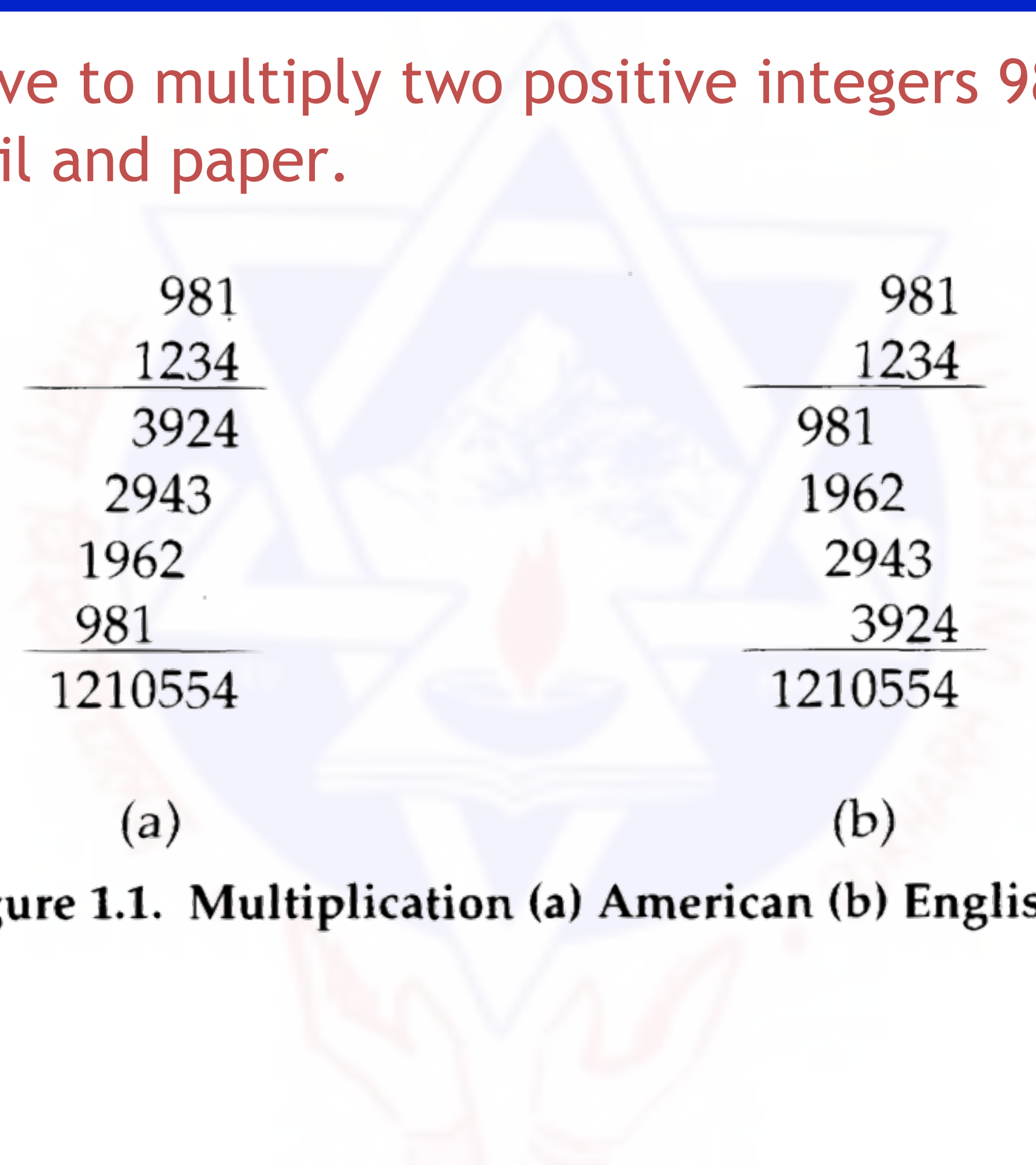
What is a program?

- **Programs:** We often think of a computer program as an instance, or concrete representation, of an algorithm in some programming language.



Efficiency of Algorithms

- Suppose you have to multiply two positive integers 981 and 1234 using only pencil and paper.



981	981
1234	1234
3924	981
2943	1962
1962	2943
981	3924
1210554	1210554
(a)	(b)

Figure 1.1. Multiplication (a) American (b) English

Efficiency of Algorithms

- Suppose you have to multiply two positive integers 981 and 1234 using only pencil and paper.

- Write the multiplicand and the multiplier side by side.
- Make two columns, one under each operand, by repeating the following rule until the number in the left-hand column is 1:
 - divide the number in the left-hand column by 2, ignoring any fractions, and
 - double the number in the right-hand column by adding it to itself.
 - Next cross out each row where the number in the left-hand column is even, and
 - finally add up the numbers that remain in the right-hand column.
- This algorithm, sometimes called multiplication *a la russe*, resembles the one used in the hardware of a binary computer.

981	1234	1234
490	2468	
245	4936	4936
122	9872	
61	19744	19744
30	39488	
15	78976	78976
7	157952	157952
3	315904	315904
1	631808	631808
		1210554

Figure 1.2. Multiplication *à la russe*



Efficiency of Algorithms

- Suppose you have to multiply two positive integers using only pencil and paper.
- For this algorithm, however, we require that both the multiplicand and the multiplier have the same number of figures, and furthermore that this number be a power of 2, such as 1, 2, 4, 8, 16, etc. This is easily fixed by adding zeros on the left if necessary: in our example, we add just one 0 on the left of the multiplicand, making it into 0981, so that both operands have four figures. Now to multiply 0981 by 1234 we first multiply the left half of the multiplicand (09) by the left half of the multiplier (12), and write the result (108) shifted left as many places as there are figures in the multiplier: four, in our example. Next we multiply the left half of the multiplicand (09) by the right half of the multiplier (34), and write the result (306) shifted left by half as many places as there are figures in the multiplier: two, in this case. Thirdly we multiply the right half of the multiplicand (81) by the left half of the multiplier (12), and write the result (972) also shifted left by half as many places as there are figures in the multiplier; and fourthly we multiply the right half of the multiplicand (81) by the right half of the multiplier (34) and write the result (2754), not shifted at all. Finally we add up the four intermediate results as shown in Figure 1.3 to obtain the answer 1210554.

	Multiply		Shift	Result
i)	09	12	4	108
ii)	09	34	2	306 . .
iii)	81	12	2	972 . .
iv)	81	34	0	2754
				1210554

Figure 1.3. Multiplying 0981 by 1234 by divide-and-conquer



Efficiency of Algorithms

- When we have a problem to solve, we can design several algorithms to solve it.
- Among several algorithms we can choose the best (i.e. most efficient).
- Here the most efficient algorithm is the algorithm that uses the minimum resources (execution time, memory space, etc.) to perform a task.
- Some algorithms may take less time to obtain the result and others may take less memory space during its execution
- The advantage of having multiple solutions to a problem is that you can choose an efficient algorithm based on your preference whether you need an algorithm that takes less time or less memory.
- Another important point is that an algorithm A might be more efficient than algorithm B for a specific variation of the given problem or for specific class of inputs.
 - For example, one sorting algorithm might be efficient for sorting small number of integers or another might be more efficient to sort large number of integers.



Efficiency of Algorithms

- Having multiple solutions (algorithms) to a problem raises the question of how to decide which of the several algorithm is best or most efficient (preferable).
- **Analysis of an algorithm** refers to the task of determining how much resources (computing time, storing space etc.) the algorithm requires.
- **Analysis of algorithms** is the process of determining computational complexity of algorithms.
- The computational complexity of an algorithm is the amount of resources required for the algorithm to produce its output.
- The resources required for an algorithms includes the computing time the algorithm takes to produce an output (time complexity), the space required for the algorithm (space complexity) and the other resources like internet, communication band etc. it may need for its processing.
- Two approaches for determining efficiency of algorithms:
 - Empirical approach
 - Theoretical approach



Efficiency of Algorithms

- **Empirical Approach:**
 - this approach consists of programming the competing algorithms, trying then on different instances of input with the help of a computer and choosing the one which is fast in execution and uses minimum space.
- **Theoretical Approach:**
 - This approach consists of determining mathematically the quantity of resources needed by each algorithm as a function of the size of the instances considered.



Efficiency of Algorithms

- **Empirical Approach:**
 - The empirical comparison works as below:
 - Develop the algorithms using some programming languages.
 - Execute those algorithms in some computers.
 - Provide a suitable range of inputs.
 - Measure how much resources the algorithms use to produce the output.
 - Choose the algorithm which uses minimum resources.
- Is this approach a better solution for comparing the algorithms? Can you figure out the weakness of this approach?



Efficiency of Algorithms

- **Factors affecting Empirical Approach:**
 - The following are some factors that may influence a fair comparison of algorithms:
 - The effort involved in the programming and testing the two algorithms.
 - How efficiently the coding of the two programs that use the algorithms are written. One of the program may be better written than other.
 - The choice of test case or input data set may unfairly favor one of the two algorithm.
 - Efficiency of the operating system used.
 - The specifications of the hardware used.



Efficiency of Algorithms

- Factors affecting Empirical Approach:
 - Empirical approach depends on the various factors like the programming language and compiler's quality you used to implement the algorithm, the speed of the CPU, bus and other peripheral hardwares, the quality of the operating systems that you are using, the hardware specification of the computer where you are running the algorithms, other programs that are currently running at your system or on your networks during the comparison, coding efficiency of programmer who convert the algorithm into a program etc.
 - Therefore, you have to develop the two algorithms with the same effort with same programming quality and under the same conditions, you have to compare them by compiling using the same compiler on the same computer.
 - This approach does not work if you want to compare the two algorithms which are developed in two different environments.



Efficiency of Algorithms

- Theoretical approach:
 - The advantages of the theoretical approach is that it depends on neither the computer being used (machine independent), nor the programming language, nor the skill of the programmer.
 - These all can be achieved in theoretical approach by using **asymptotic analysis of algorithms** or simply **asymptotic analysis** to measure their efficiencies.
 - **Asymptotic analysis** attempts to estimate the resource consumption of an algorithm.
 - **Asymptotic analysis** allows us to compare the relative costs of two or more algorithms for solving the same problem.



Input Size

- One primary factor that need to be consider to analysis the performance of an algorithm is size of input
- The input size is the number of data items that need to be processed.
- Suppose you are given a list of integer numbers to sort in ascending order.
 - The number of data items in the given list is the input size.
 - As the input size increases, the time taken by the algorithm to sort the list grows



Running Time

- The running time of an algorithm depends on the size of input.
- The running time of the algorithm is defined as the number of primitive operations or steps executed in the algorithm.
- Different line in the algorithm may take different time for execution. However, we assume each line of algorithm or pseudocode require a constant amount of time to execute.
-



Running Time

- Let us consider an algorithm to find the largest element in an array of size n

Algorithm Largest(A, n):

Input: An array A with n integer elements.

Output: The largest element in A .

$currentLarge \leftarrow A[0]$

for $i \leftarrow 1$ **to** $n - 1$ **do**

if $currentLarge < A[i]$ **then**

$currentLarge \leftarrow A[i]$

return $currentLarge$

Fig. 2.1: Pseudo-code of an algorithm that finds the largest element (integer) in an array

Running Time

- Suppose you are given a task to find the total cost for running time of the algorithm **Largest(A, n)**.
- What you have to do is you need to find the time cost of each statement in the algorithm and the number of times each statement of the algorithm are executed.

Algorithm	Largest(A, n)	Cost	Times
1	<i>currentLarge</i> $\leftarrow$ A[0]	C1	1
2	for i $\leftarrow$ 1 to n-1 do	C2	n
3	if <i>currentLarge</i> < A[i] then	C3	n-1
4	<i>currentLarge</i> $\leftarrow$ A[i]	C4	n-1
5	return <i>currentLarge</i>	C5	1

Fig. 2.2: Cost calculation of algorithm **Largest(A, n)**



Running Time

- Suppose you are given a task to find the total cost for running time of the algorithm **Largest(A, n)**.
- What you have to do is you need to find the time cost of each statement in the algorithm and the number of times each statement of the algorithm are executed.

Algorithm	Largest(A, n)	Cost	Times
1	<i>currentLarge</i> $\leftarrow$ A[0]	C1	1
2	for i $\leftarrow$ 1 to n-1 do	C2	n
3	if <i>currentLarge</i> < A[i] then	C3	n-1
4	<i>currentLarge</i> $\leftarrow$ A[i]	C4	n-1
5	return <i>currentLarge</i>	C5	1

- The total running time $T(n)$ of the algorithm is given by

$$T(n) = c_1 + c_2n + c_3(n - 1) + c_4(n - 1) + c_5$$



Running Time

$$T(n) = c_1 + c_2n + c_3(n - 1) + c_4(n - 1) + c_5$$

- The value of $T(n)$ depends on the position of the largest number in the given array.
- Suppose the elements in the array are sorted in descending order.
- The first element in the array is the largest element.
- If the first element of the array is largest, then the statement 3 is always false and statement 4 is never executed.
- This is called **best-case analysis** of the algorithm and it is given by

$$\begin{aligned} T(n) &= c_1 + c_2n + c_3(n - 1) + c_4 * 0 + c_5 \\ &= (c_2 + c_3)n + (c_1 - c_3 + c_5) \end{aligned}$$

- This can be expressed in the form of $an + b$ where $a = c_2 + c_3$ and $b = c_1 - c_3 + c_5$.
- Therefore, $T(n)$ of the given algorithm in **best case** is a linear function of n .



Running Time

$$T(n) = c_1 + c_2n + c_3(n - 1) + c_4(n - 1) + c_5$$

- Suppose the elements in the array are sorted in ascending order.
- This means last element in the array is the largest element.
- If the last element of the array is largest, then the statement 3 is always true and statement 4 is always executed. That is statement 4 is executed for $(n - 1)$ times.
- This is called **worst-case analysis** of the algorithm. Therefore, $T(n)$ in worst-case is given by

$$\begin{aligned} T(n) &= c_1 + c_2n + c_3(n - 1) + c_4(n - 1) + c_5 \\ &= (c_2 + c_3 + c_4)n + (c_1 - c_3 - c_4 + c_5) \end{aligned}$$

- This can also be expressed in the form of $an + b$ where $a = c_2 + c_3 + c_4$ and $b = c_1 - c_3 - c_4 + c_5$.
- Therefore, $T(n)$ of the given algorithm in worst case is also a linear function of n .



Running Time

Algorithm	INSERTION-SORT (A, n)	Cost	Times
1	for $j \leftarrow 1$ to $n - 1$ do	c_1	n
2	$\text{temp} \leftarrow A[j]$	c_2	$n - 1$
3	▷ Insert $A[j]$ into the sorted array	0	0
4	$i \leftarrow j - 1$	c_4	$n - 1$
5	while $i \geq 0$ and $\text{temp} \leq A[i]$ do	c_5	$\sum_{j=1}^{n-1} t_j$
6	$A[i+1] \leftarrow A[i]$	c_6	$\sum_{j=1}^{n-1} (t_j - 1)$
7	$i \leftarrow i - 1$	c_7	$\sum_{j=1}^{n-1} (t_j - 1)$
8	$A[i+1] \leftarrow \text{temp}$	c_8	$n - 1$



Running Time

- The total running time $T(n)$ of the INSERTION-SORT (A, n) algorithm is given by

$$T(n) = c_1n + c_2(n - 1) + c_4(n - 1) + c_5 \sum_{j=1}^{n-1} t_j + c_6 \sum_{j=1}^{n-1} (t_j - 1) + c_7 \sum_{j=1}^{n-1} (t_j - 1) + c_8(n - 1)$$



Running Time

- **Best-case of INSERTION-SORT (A, n)**: Suppose the elements in the given array are already sorted. In such case, $\text{temp} \leq A[I]$ is always false. Therefore, the sixth and seventh never executed i.e. the number of time, the sixth and seventh statements execute is zero. However, the fifth statement executes only for one time for a particular value of $j = 1, 2, \dots, n - 1$. Therefore, $T(n)$ for best-case is given by
 - $T(n) = c_1n + c_2(n - 1) + c_4(n - 1) + c_5(n - 1) + c_6 * 0 + c_7 * 0 + c_8(n - 1)$
 - $= c_1n + c_2(n - 1) + c_4(n - 1) + c_5(n - 1) + c_8(n - 1)$
 - $= c_1n + c_2n - c_2 + c_4n - c_4 + c_5n - c_5 + c_8n - c_8$
 - $= (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8)$
 - $= an + b$ where $a = c_1 + c_2 + c_4 + c_5 + c_8$ and $b = -(c_2 + c_4 + c_5 + c_8)$.
- Therefore, $T(n)$ of the given **INSERTION-SORT (A, n)** algorithm in best-case is a linear function of n .



Running Time

- **Worst-case of INSERTION-SORT (A, n) :** Suppose the elements in the given array are already reversed order i.e. the array is already sorted in descending order. In such case, $\text{temp} \leq A[i]$ is always true. If you trace the algorithm, you will find that for $j = 1$, the sixth and seventh statements executed for 1 times, for $j = 2$, they execute for 2 times, for $j = 3$, they execute for 3 times and for $j = n - 1$, they execute for $n - 1$ times. Therefore, the number of times, the sixth and seventh statements execute is equal to
 - $= 1 + 2 + 3 + \dots + n - 1$
 - $= (1 + 2 + 3 + \dots + (n - 1) + n) - n$
 - $= \frac{n(n + 1)}{2} - n$
 - $= \frac{n(n - 1)}{2}$
- The fifth statement i.e. the header of the *while*-loop executes one more time than that of the sixth and seventh statements. For $j = 1$, the fifth statement executed for 2 times, for $j = 2$, it executes for 3 times, for $j = 3$, it executes for 4 times and finally for $j = n - 1$, it executes for n times. Therefore, the number of times, the fifth statement executes is equal to
 - $= 2 + 3 + \dots + n - 1 + n$
 - $= (1 + 2 + 3 + \dots + n - 1 + n) - 1$
 - $= \frac{n(n + 1)}{2} - 1$



Running Time

- Therefore, $T(n)$ of the given **INSERTION-SORT (A, n)** algorithm for worst-case is given by
- $T(n) = c_1n + c_2(n - 1) + c_4(n - 1) + c_5(\frac{n(n + 1)}{2} - 1) + c_6(\frac{n(n - 1)}{2}) + c_7(\frac{n(n - 1)}{2}) + c_8(n - 1)$
- $= (\frac{c_5 + c_6 + c_7}{2})n^2 + (c_1 + c_2 + c_4 + c_8 - \frac{c_6 + c_7 - c_5}{2})n - (c_2 + c_4 + c_5 + c_8)$
- This worst-case running time $T(n)$ can be expressed in the form of $an^2 + bn + c$ where $a = \frac{c_5 + c_6 + c_7}{2}$
 $b = c_1 + c_2 + c_4 + c_8 - \frac{c_6 + c_7 - c_5}{2}$ and $c = -(c_2 + c_4 + c_5 + c_8)$
- Therefore, $T(n)$ of the given **INSERTION-SORT (A, n)** algorithm in worst-case is a quadratic function of n .



Best, worst and average case analysis

- You have just studied what are the best-case and worst-case analysis with the example of **INSERTION-SORT (A, n)** algorithm.
- For a sorting algorithm, best-case occurs if the input array is already sorted.
- In contrast, the worst-case occurs if the input array is in reverse order i.e. the element of the array are sorted in descending order.
- We normally use the worst-case analysis for determining the running time of an algorithm since it gives the longest time that the algorithm may take for the worst scenario of input set.
- The running time doesn't go beyond the worst-case running time.



Best, worst and average case analysis

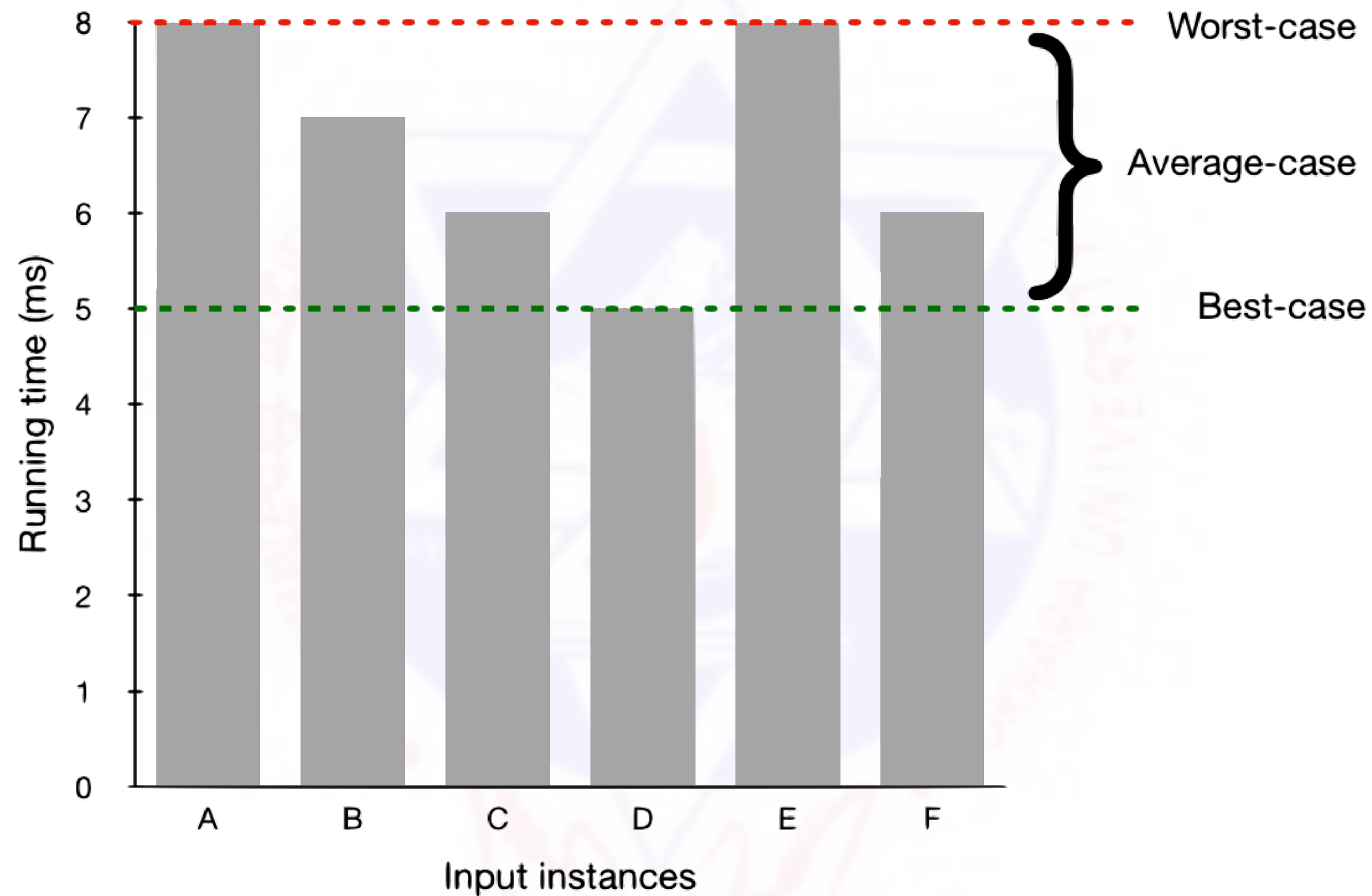


Fig. 2.3 : Running time of an algorithm in different instances of input set

Best, worst and average case analysis

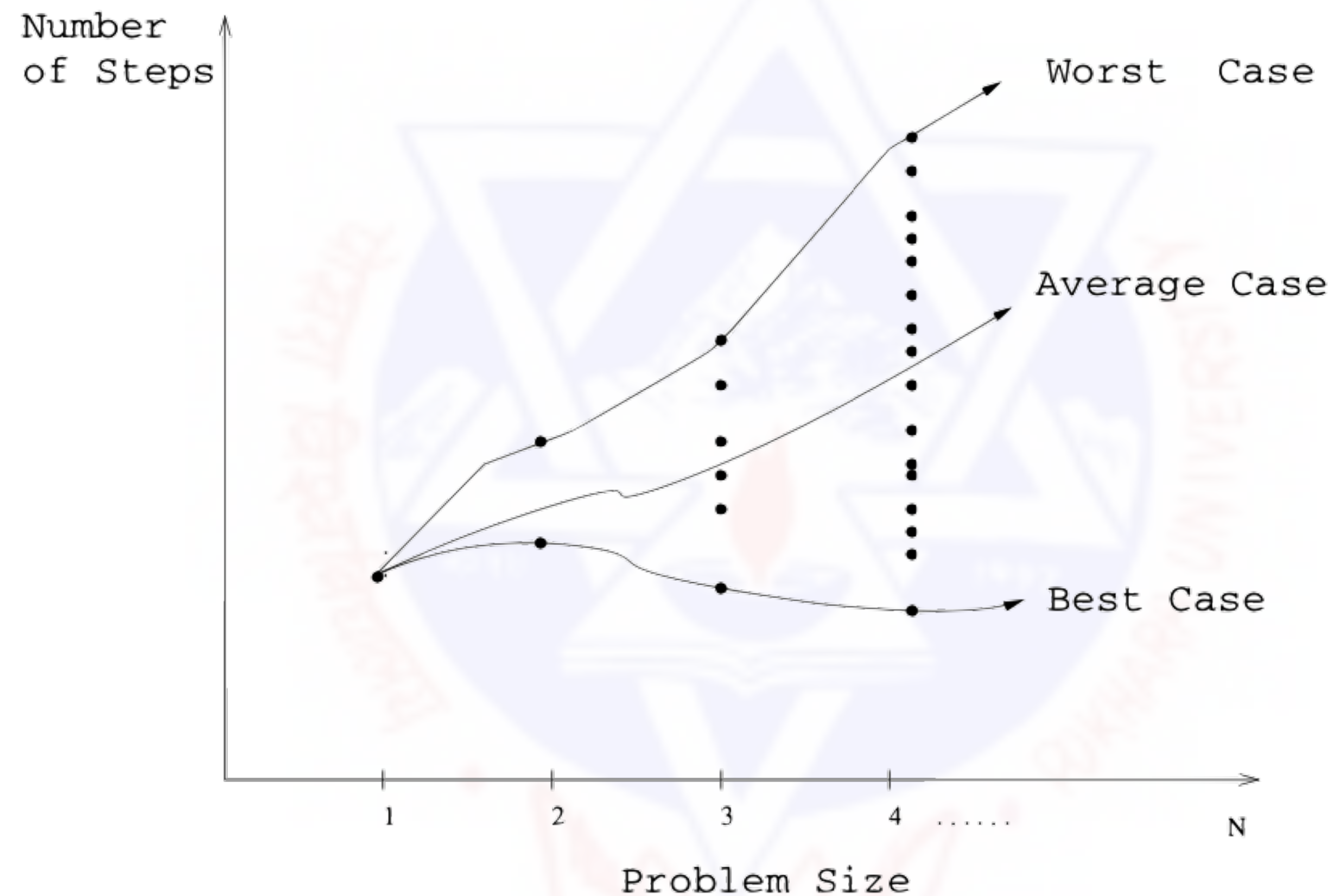
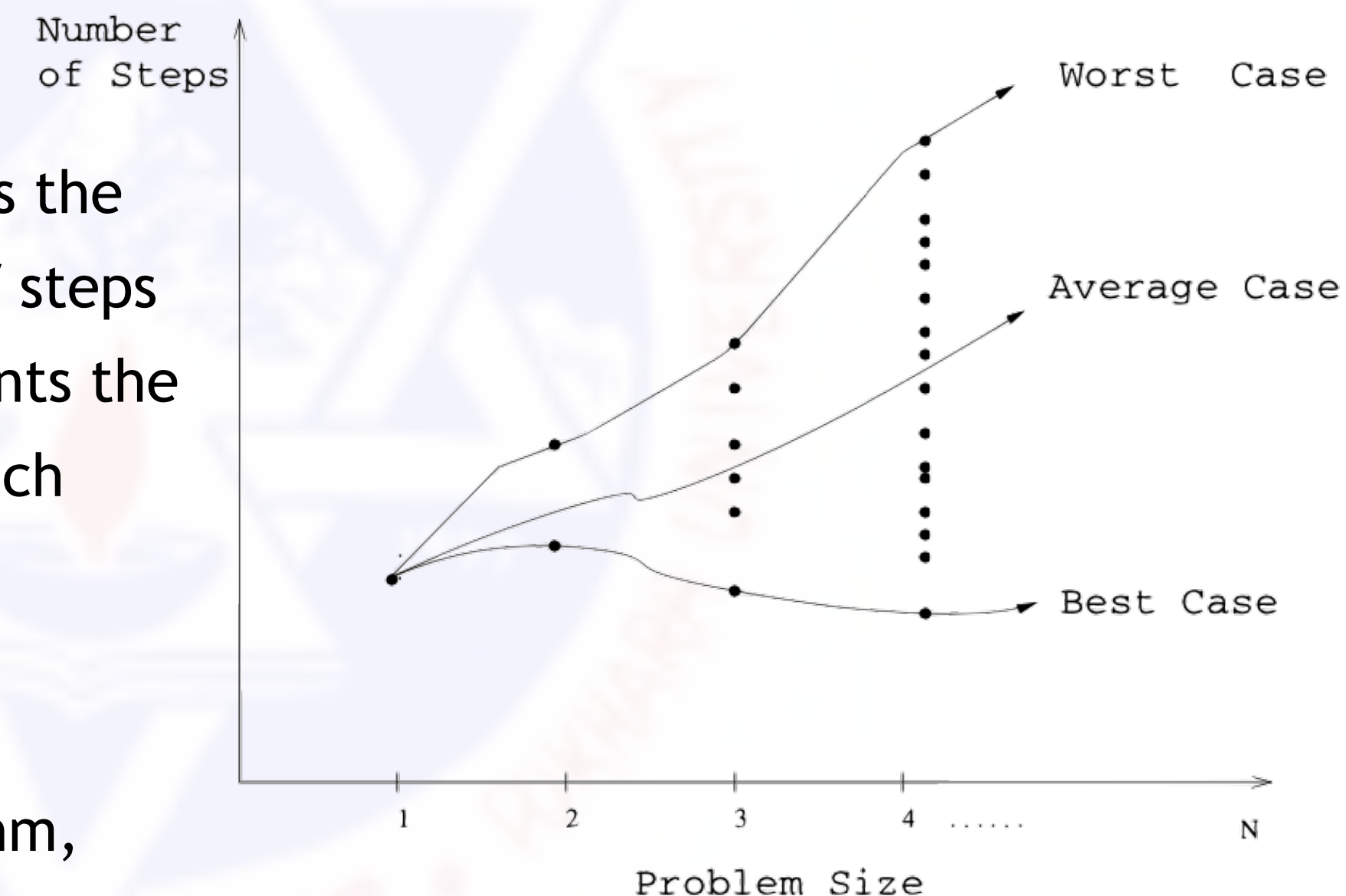


Figure 2.1: Best, worst, and average-case complexity

Best, worst and average case analysis

- The **worst-case** complexity of the algorithm is the function defined by the maximum number of steps taken in any instance of size n . This represents the curve passing through the highest point in each column.

- The **best-case** complexity of the algorithm is the function defined by the minimum number of steps taken in any instance of size n . This represents the curve passing through the lowest point of each column.



- The **average-case** complexity of the algorithm, which is the function defined by the average number of steps over all instances of size n .

Figure 2.1: Best, worst, and average-case complexity

Best, worst and average case analysis

- Average-case running time is roughly as bad as the worst case running time.
- There is no way to exactly define what the average-input for a problem.
- We can use a randomized algorithm to randomly choose n numbers so that half of the elements are assumed to be in the sorted form if it is the case of sorting algorithm.
- The average-case running time is roughly the half of the worst-case running time.

$$\bullet T(n)_{avg-case} = \frac{T(n)_{worst-case}}{2} = \frac{an^2 + bn + c}{2} = \frac{a}{2}n^2 + \frac{b}{2}n + \frac{c}{2}$$

- Which is again turned into a quadratic function of input size of n .
- Therefore, we normally use the worst-case running time to determine the performance of an algorithm.



Rate of Growth

- The amount of resource an algorithm consumes/demands depends on the amount of the input size.
- If you increase the input size, the running time (space required) may also increase.
- As we go increasing its input size, the amount of resources required for the algorithm keep on growing.
- The rate of growth (or order of growth or growth rate) is defined as the rate at which the resource consumption (i.e. time or space or any other resource requirement) of an algorithm increases as the input size grows.



Rate of Growth

- In our previous examples, you saw the running time of algorithm can be a **linear** or **quadratic** function of the input size.

Table 2.2: Some common growth rate functions

SN	Function of n	Name of Growth Rate	Example
1	c	Constant Growth Rate	Stack's Push or Pop
2	$\log_b(n)$	Logarithmic (for base b) Growth	Binary Search
3	n	Linear Growth Rate	Sequential search
4	$n \log_b(n)$	Linear logarithmic Growth Rate	Quick sort
5	n^2	Quadratic Growth Rate	Insertion Sort
6	n^3	Cubic Growth Rate	Matrix Multiplication
7	2^n	Exponential Growth Rate	Tower of Hanoi



Growth Rates & Amount of Resource Demand

- You should know how much more resource your algorithm will demand when the amount of input size is increased.

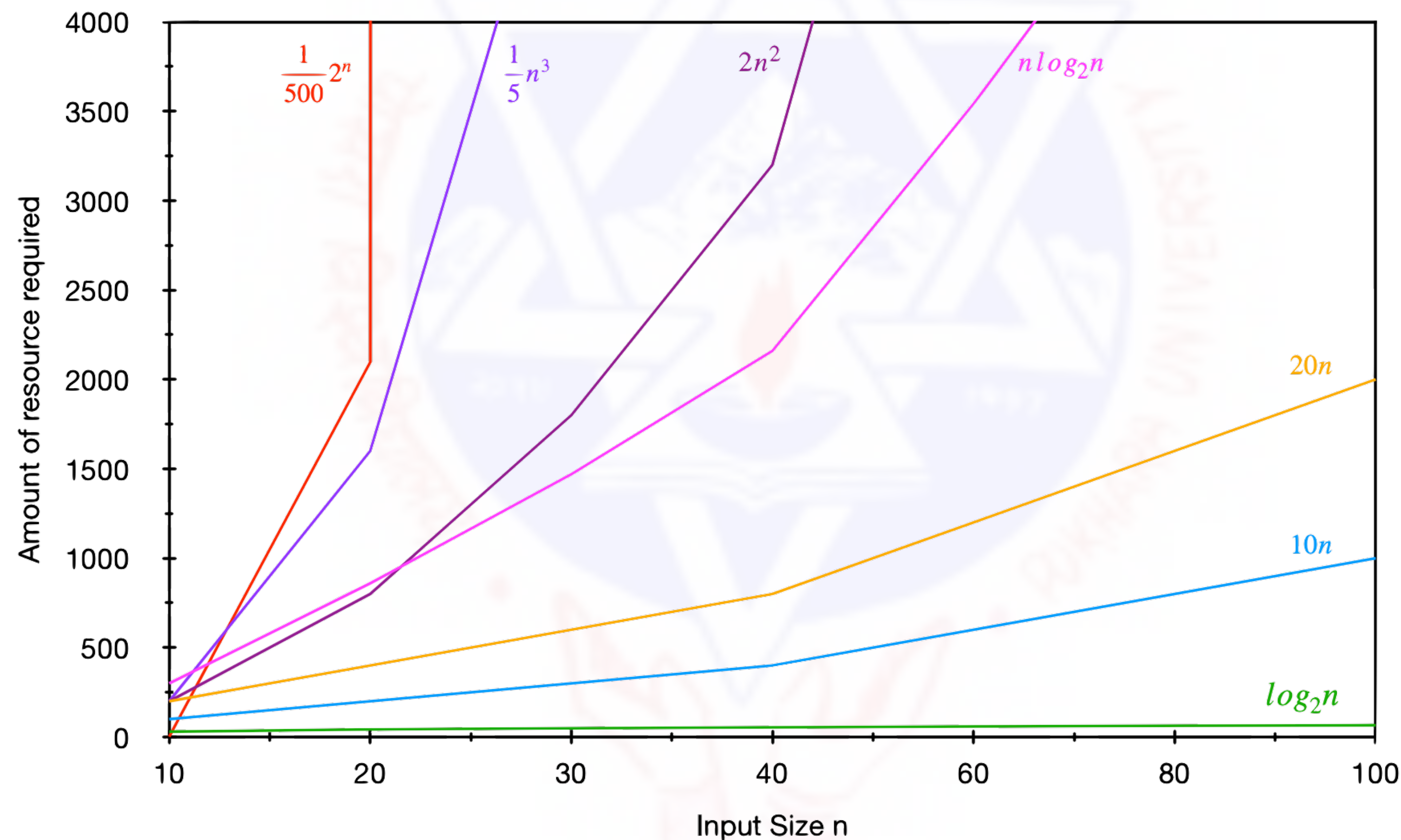
Table 2.3: Amount of running time of different algorithms with different growth rates for varying input size $n = 10, 20, 30, 40, 60, 80$ and 100

SN	Functions with common growth rates	Input Size (n)						
		10	20	30	40	60	80	100
1	Logarithmic ($\log_2 n$)	30	43	49	54	59	63	66
2	Linear function ($10n$)	100	200	300	400	600	800	1000
3	Linear function ($20n$)	200	400	600	800	1200	1600	2000
4	Linear Logarithmic ($n \log_2 n$)	300	860	1470	2160	3540	5056	6640
5	Quadratic function ($2n^2$)	200	800	1800	3200	7200	12800	20000
6	Cubic function ($\frac{1}{5}n^3$)	200	1600	5400	12800	43200	102400	200000
7	Exponential function ($\frac{1}{500}2^n$)	2	2097	2.1E+06	2.2E+09	2.3E+15	2.4E+21	2.5E+27



Growth Rates & Amount of Resource Demand

- The Fig. 2.5 shows graphical visualization of the same information in the Table 2.3.



Growth Rates & Amount of Resource Demand

- The Fig. 2.5 shows graphical visualization of the same information in the Table 2.3.
- Here, you can easily understand how the amount of resource required by algorithm (with different growth rates) grows as the input size increases.
- The logarithmic ($\log_2 n$) growth rate shows demanding of very small amount of resource (e.g. running time) even in a very large grow in the input size.
- The linear growth rate grows more quickly than that of the logarithmic growth rate. However, it grows much slowly than that of the quadratic growth rate, cubic growth rate and exponential growth rate.



Growth Rates & Amount of Resource Demand

- If the growth rate of running time of an algorithm is more slower, the algorithm is more efficient.
- It means the algorithm demands very small amount of resource even in the large increase in the input size.
- Suppose you have two algorithms- A and B. If the algorithm A has the growth rate of quadratic function and algorithm B has the growth rate of linear function, then the algorithm B is more efficient than A. Suppose you have another algorithm C whose growth rate is of logarithmic function, then the algorithm C is the most efficient.



Comparison of Algorithms using Growth Rate

- Suppose you are given a task to analyze two algorithms- algorithm A which has the running time $T_A(n) = 100n + 2$ and algorithm B with running time $T_B(n) = n^2 + n + 2$.

Table 2.6: Comparing growth rates of Algorithms A and B

Input size n	Running time of Algorithm A	Running time of Algorithm B
5	502	32
10	1002	112
50	5002	2552
99	9902	9902
100	10002	10102
500	50002	250502
1000	100002	1001002
10000	1000002	100010002



Comparison of Algorithms using Growth Rate

- For $n = 5$, running time of A is approx. 15 times longer than that of the B.
- For $n = 10$, it is approx. 9 times longer than that of the B.
- For $n = 99$, the running times of both algorithms are same.
- After $n = 99$, the algorithm A needs less running time even for the large n .
- For $n = 500, 1000$ and 10000 , the rate of growth of running time of algorithm A is much slower than that of the algorithm B.
- Therefore, **the algorithm A is much better than the algorithm B.**
- The function whose leading term is n^2 grows faster than the function whose leading term is n .
- The running time of B is better than A for small value of n (i.e. $n < 99$) because the leading term of algorithm B has a large coefficient (i.e. 100).



Assumption for Finding Growth Rate

- Let us take the example of the **insertion-sort** algorithm.
- In best-case, $T(n) = an + b \approx n$
- In worst-case, $T(n) = an^2 + bn + c \approx n^2$ where a , b and c are some constants.
- Both the multiplicative coefficients and the lower-order terms of $T(n)$ are relatively insignificant for large enough input n .
- Therefore, we only take the leading term of $T(n)$ ignoring all the lower-order terms and constant coefficient.
- You can see in Table 2.6 where the algorithm A has large coefficient (100) in the leading term. Due to this, the algorithm appears to be inefficient until $n < 99$ but when $n > 99$, the algorithm A beats the algorithm B.



Assumption for Finding Growth Rate

- To determine the order of growth, we have ignored the insignificant terms and coefficients.
- That is we are not finding the exact order of growth. It means we are determining the **asymptotic efficient** of algorithm.
- In other words, we are doing **asymptotic analysis of algorithms**.
- In the case of algorithms presented in Table 2.6, algorithm A is asymptotically more efficient than the algorithm B.
- Note that an algorithm with lower order of growth in worst-case is more efficient than another.
- Beware of very large constant factors. An algorithm running in time $1,000,000n$ is still $O(n)$ but might be less efficient on your data set than one running in time $2n^2$, which is $O(n^2)$



Asymptotic Analysis

- To compare the efficiency of algorithms, it is not often important to find their exact running time or the exact memory space they need.
- It is sufficient to find the rate of growth of a resource that the algorithms demands as their input size increases.
- To find only the rate of growth of resources, the lower-order terms and coefficients in $T(n)$ can be discarded by only considering the leading term as $n \rightarrow \infty$.
- These rate of growths of algorithms are then compared to choose an efficient algorithm.
- This is called the **asymptotic analysis of algorithm**.
- A function $g(n) = n^3 + 2n^2 - 7$ behaves asymptotically like n^3 and
- A function $h(n) = n \log n + 5n$ behaves asymptotically like $n \log n$.



Asymptotic Analysis

- The asymptotic analysis of algorithms uses standard notations called *asymptotic notations*.
- The following are the commonly used asymptotic notations to describe the running time of algorithms:
 1. Big Oh Notation (O)
 2. Big Omega Notation (Ω)
 3. Big Theta Notation (Θ)
- **Upper bound:** The upper bound for growth of $T(n)$ shows highest growth rate that an algorithm can have
- **Lower bound:** it is the lowest growth rate that an algorithm can have.



Asymptotic Analysis

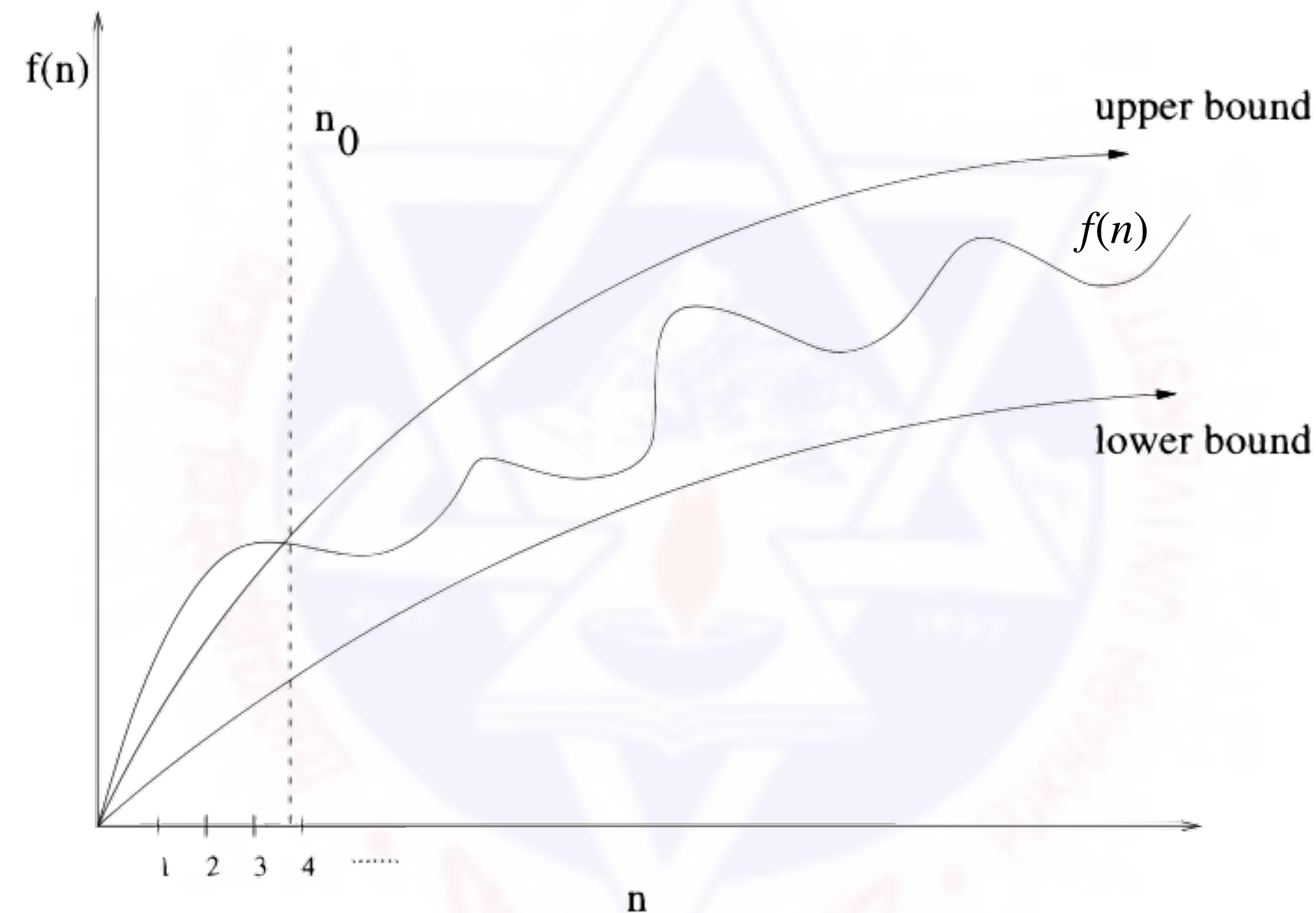


Figure 2.2: *Upper* and *lower* bounds valid for $n > n_0$ smooth out the behavior of complex functions

The Big Oh Notation (O)

- The big Oh notation (O) describes the highest growth rate (i.e. the maximum possible amount) of a resource that an algorithm demands for the input size $n \rightarrow \infty$.
- The big Oh notation (O) represents the *asymptotic upper bound* of a resource that an algorithm demands. That is **it describes the growth rate for the worst-case of an algorithm.**
- **For example, the worst-case running time of insertion sort is $O(n^2)$.**
- **Read this $O(n^2)$ as “in order of square n ”.**



The Big Oh Notation (O)

- Let us consider $T(n)$ is a running time of an algorithm for input size n and $g(n)$ is another function of n for upper bound.
- Suppose a positive constant c is a coefficient of the upper bound function $g(n)$ and positive number n_0 is an instance of the amount of the input size.
- The big Oh notation to represent the upper bound of $T(n)$, written as $O(g(n))$ (read as big oh of g of n), is then mathematically defined as:

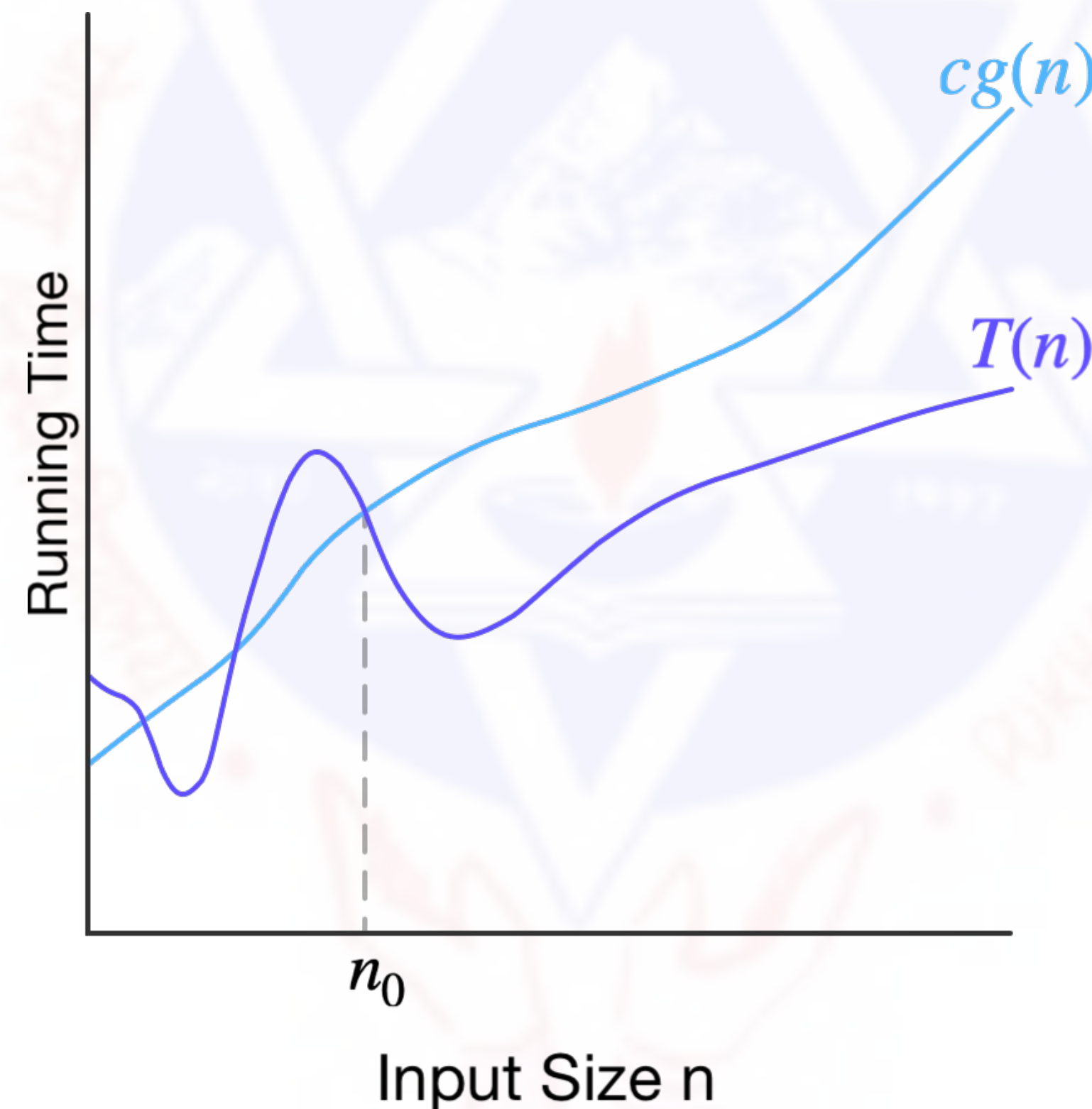
$$O(g(n)) = \{T(n): \text{there exists positive constants } c \text{ and } n_0 \text{ such that } 0 \leq T(n) \leq cg(n) \text{ for all } n \geq n_0\}.$$

i.e. $T(n) = O(g(n))$ (We have used '=' instead of ' $\in$ ' to indicate $T(n) \in O(g(n))$)



The Big Oh Notation (O)

The big Oh notation to represent upper bound of $T(n)$, written as $O(g(n))$ is defined as:
 $O(g(n)) = \{T(n): \text{there exists positive constants } c \text{ and } n_0 \text{ such that } 0 \leq T(n) \leq cg(n) \text{ for all } n \geq n_0\}.$



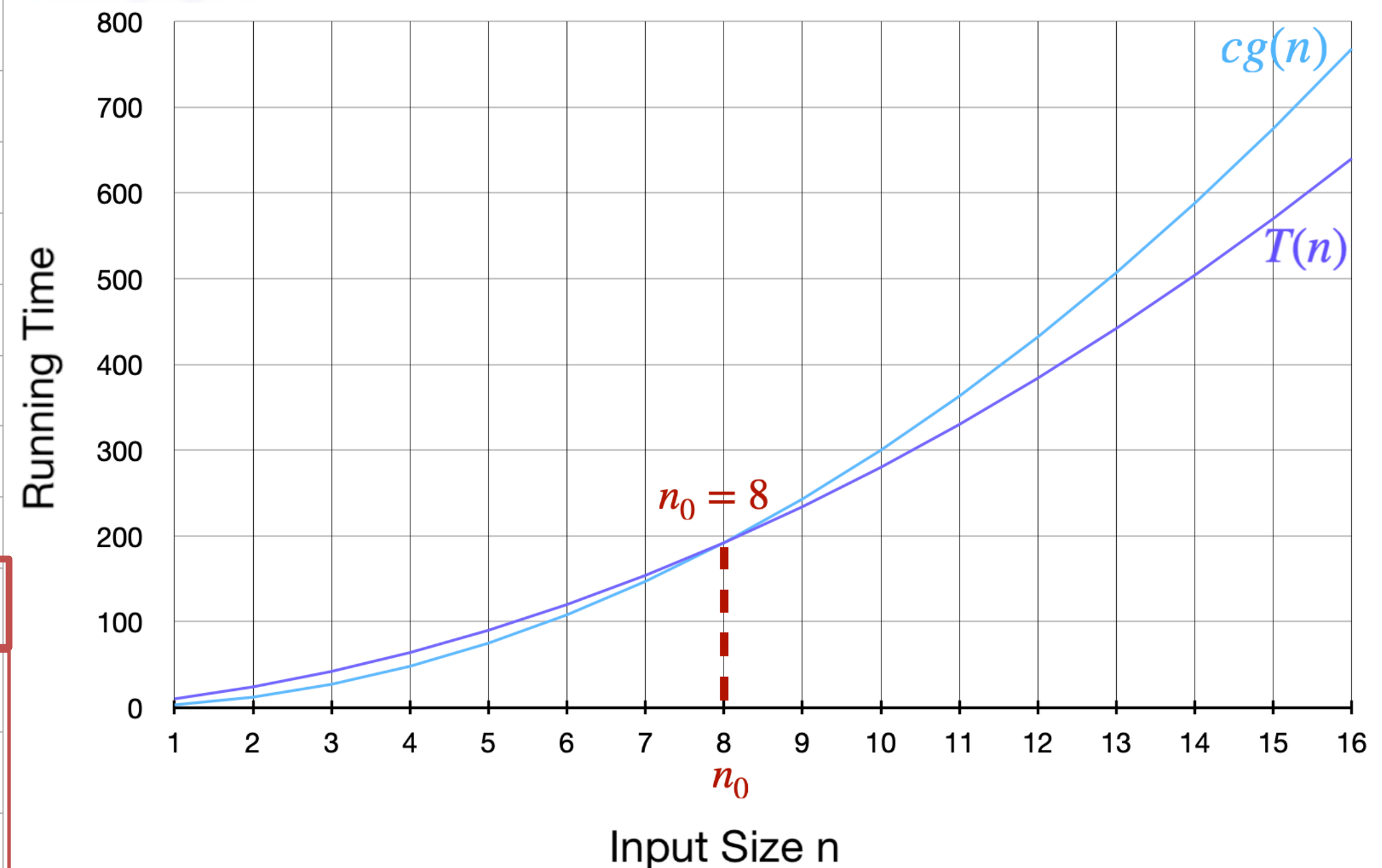
The Big Oh Notation (O)

- **Example:** To illustrate how the big oh notation (O) is used to represent the upper bound of running time of an algorithm,
 - let us consider the running time function of an algorithm as $T(n) = 2n^2 + 8n$.
 - Suppose the upper bound function $g(n) = n^2$ (The $g(n)$ is nothing other than the leading term in $T(n)$ ignoring its coefficient) and a positive constant $c = 4$.
 - The values of $T(n)$ and $cg(n)$ for $n = 1, 2, 3, \dots, 15$ are shown in Table 2,7.



The Big Oh Notation (O)

n	$T(n) = 2n^2 + 8n$	$cg(n) = 4n^2$	Remarks
1	10	4	
2	24	12	
3	42	27	
4	64	48	
5	90	75	
6	120	108	
7	154	147	
8	192	192	$0 \leq T(n) \leq cg(n)$ and $(n_0 = 8)$
9	234	243	$0 \leq T(n) \leq cg(n)$ and $(n \geq n_0)$
10	280	300	$0 \leq T(n) \leq cg(n)$ and $(n \geq n_0)$
11	330	363	$0 \leq T(n) \leq cg(n)$ and $(n \geq n_0)$
12	384	432	$0 \leq T(n) \leq cg(n)$ and $(n \geq n_0)$
13	442	507	$0 \leq T(n) \leq cg(n)$ and $(n \geq n_0)$
14	504	588	$0 \leq T(n) \leq cg(n)$ and $(n \geq n_0)$
15	570	675	$0 \leq T(n) \leq cg(n)$ and $(n \geq n_0)$



The time complexity is given by

$$T(n) = O(g(n))$$

$$= O(n^2) \text{ since } g(n) = n^2.$$



The Big Omega Notation (Ω)

- The big omega Ω notation describes the least possible amount of the running time $T(n)$ of an algorithm that it needs for a large enough input size (i.e. $n \rightarrow \infty$).
- Since the big omega notation describes the asymptotic lower bound of the running time for an algorithm, it is used to describe the best case of an algorithm.
- For example, the best-case running time of insertion sort is $\Omega(n)$.



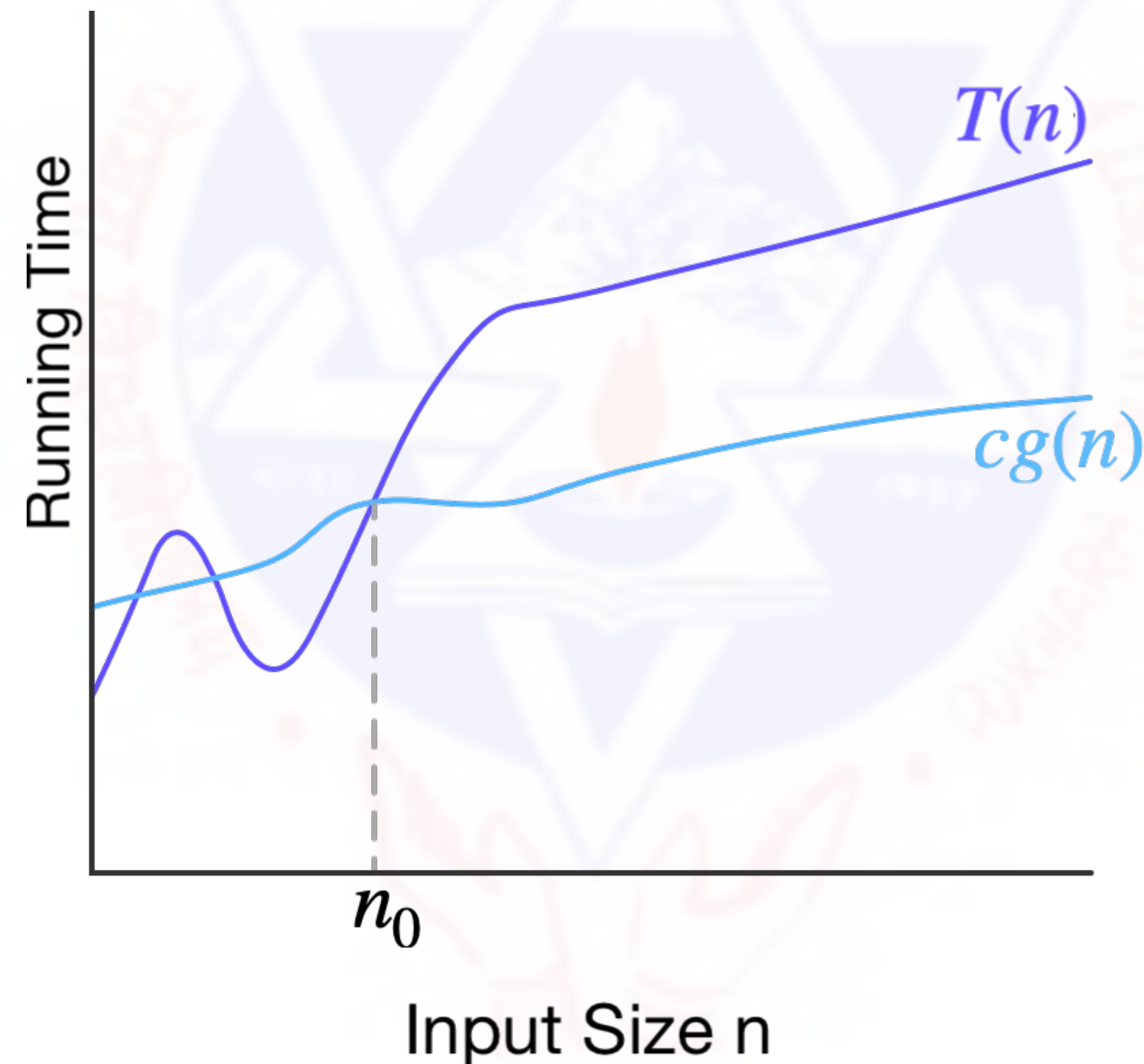
The Big Omega Notation (Ω)

- Let us consider $T(n)$ is a running time of an algorithm for input size n and $g(n)$ is another function of n to describe the lower bound.
- Suppose a positive constant c is a coefficient of the lower bound function $g(n)$ and positive number n_0 is an instance of the amount of the input size.
- The big omega notation to represent the lower bound of $T(n)$, written as $\Omega(g(n))$ (read as big omega of g of n), is then mathematically defined as:
 - $\Omega(g(n)) = \{T(n): \text{there exists positive constants } c \text{ and } n_0 \text{ such that } 0 \leq cg(n) \leq T(n) \text{ for all } n \geq n_0\}$.
 - i.e. $T(n) = \Omega(g(n))$ (We have used '=' instead of ' $\in$ ' to indicate $T(n) \in \Omega(g(n))$)



The Big Omega Notation (Ω)

The big omega notation to represent lower bound of $T(n)$, written as $\Omega(g(n))$, is defined as:
 $\Omega(g(n)) = \{T(n): \text{there exists positive constants } c \text{ and } n_0 \text{ such that } 0 \leq cg(n) \leq T(n) \text{ for all } n \geq n_0\}$.



The Big Theta Notation (Θ)

- The big theta notation Θ describes both the asymptotic upper bound and lower bound of running time of algorithms.
- That is the big theta notation binds the running time function $T(n)$ of an algorithm from below and above.
- This notation is, therefore, generally used to describe the average-case of an algorithm.



The Big Theta Notation (Θ)

- Let us consider $T(n)$ is a running time of an algorithm for input size n and $g(n)$ is another function of n to describe the upper and lower bound.
- Suppose a positive constants c_1 and c_2 are coefficients of $g(n)$ and positive number n_0 is an instance of the amount of the input size.
- The big theta notation to represent the lower bound of $T(n)$, written as $\Theta(g(n))$ (read as big theta of g of n), is then mathematically defined as:
$$\Theta(g(n)) = \{T(n): \text{there exists positive constants } c_1, c_2 \text{ and } n_0 \text{ such that } 0 \leq c_1g(n) \leq T(n) \leq c_2g(n) \text{ for all } n \geq n_0 \}.$$

i.e. $T(n) = \Theta(g(n))$ (We have used '=' instead of ' $\in$ ' to indicate $T(n) \in \Theta(g(n))$)

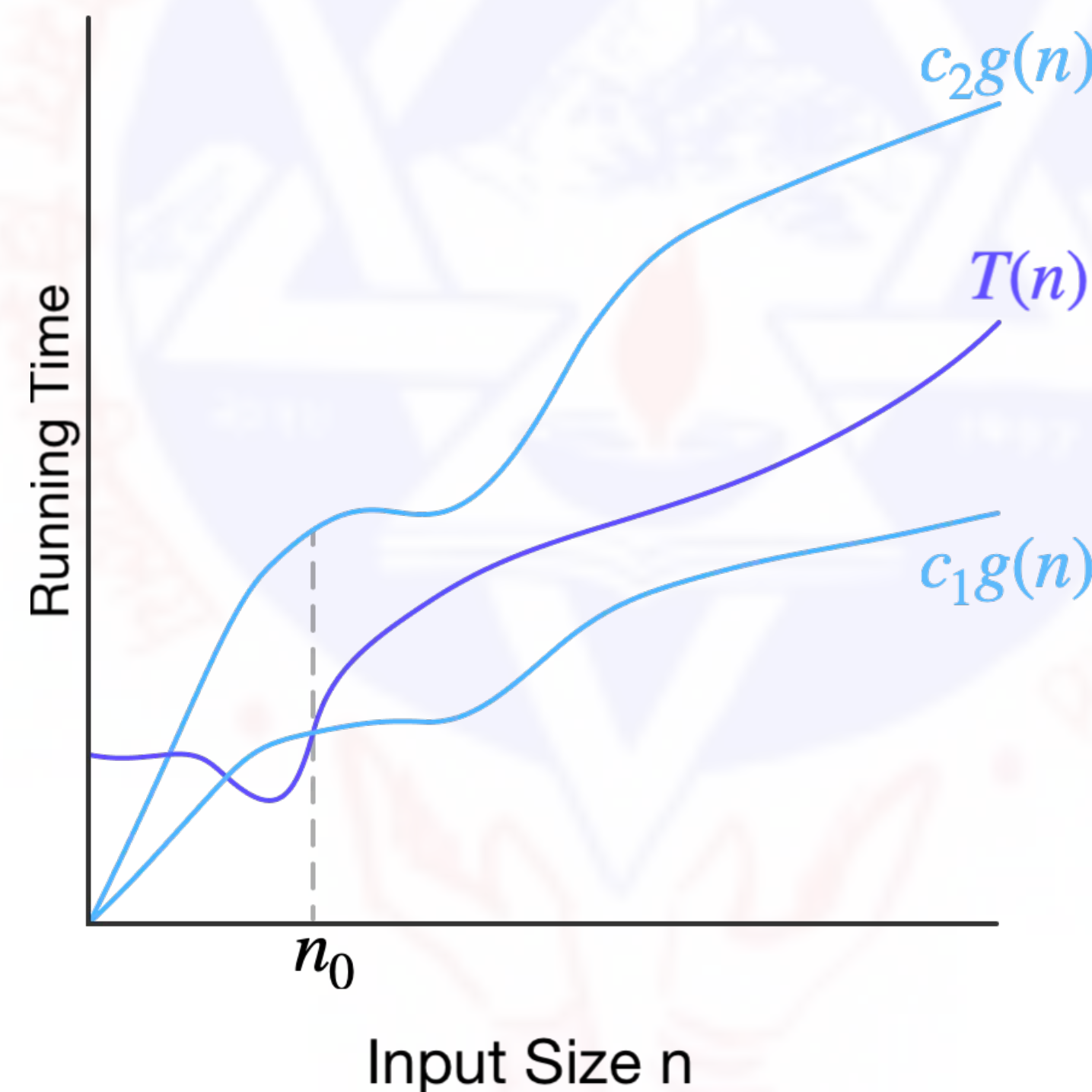


The Big Theta Notation (Θ)

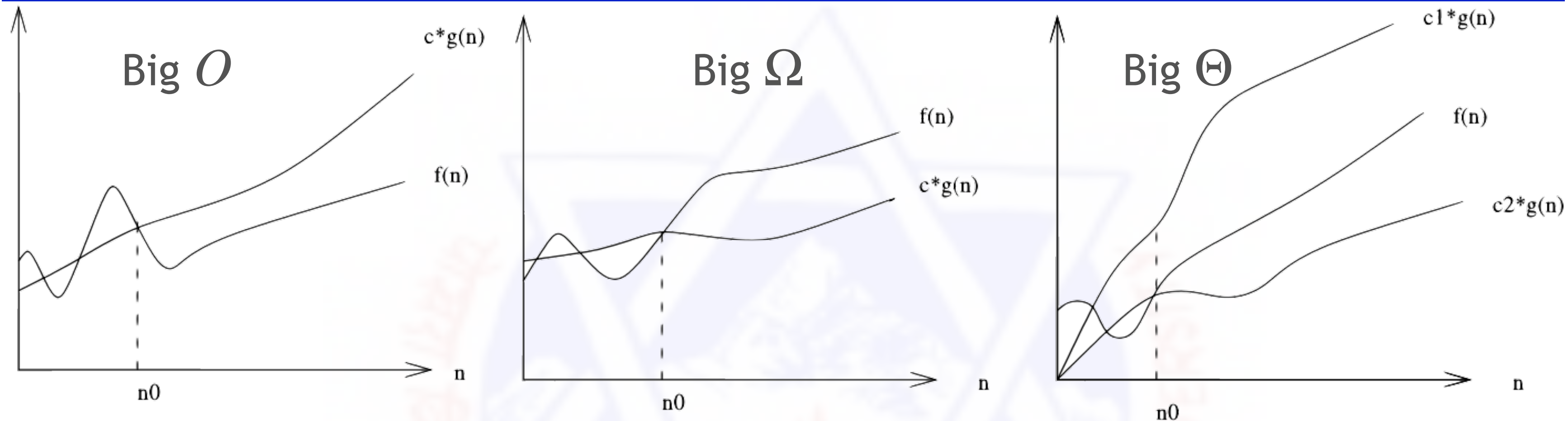
The big theta notation $\Theta(g(n))$ to represent the lower bound of $T(n)$, is defined as:

$\Theta(g(n)) = \{T(n): \text{there exists positive constants } c_1, c_2 \text{ and } n_0 \text{ such that}$

$$0 \leq c_1g(n) \leq T(n) \leq c_2g(n) \text{ for all } n \geq n_0 \}.$$



The Big Theta Notation (Θ)



- $f(n) = O(g(n))$ means $c \cdot g(n)$ is an *upper bound* on $f(n)$. Thus there exists some constant c such that $f(n)$ is always $\leq c \cdot g(n)$, for large enough n (i.e. , $n \geq n_0$ for some constant n_0).
- $f(n) = \Omega(g(n))$ means $c \cdot g(n)$ is a *lower bound* on $f(n)$. Thus there exists some constant c such that $f(n)$ is always $\geq c \cdot g(n)$, for all $n \geq n_0$.
- $f(n) = \Theta(g(n))$ means $c_1 \cdot g(n)$ is an upper bound on $f(n)$ and $c_2 \cdot g(n)$ is a lower bound on $f(n)$, for all $n \geq n_0$. Thus there exist constants c_1 and c_2 such that $f(n) \leq c_1 \cdot g(n)$ and $f(n) \geq c_2 \cdot g(n)$. This means that $g(n)$ provides a nice, tight bound on $f(n)$.

Introduction to **ALGORITHM ENGINEERING**

Algorithm Engineering

- Algorithmics is the sub-discipline of computer science that studies the systematic development of efficient algorithms.
- Algorithm Engineering (AE) is a methodology for algorithmic research that views design, analysis, implementation, and experimental evaluation of algorithms as a cycle driving algorithmic research.
- Algorithm engineering focuses on the design, analysis, implementation, optimization, profiling and experimental evaluation of computer algorithms, bridging the gap between algorithm theory and practical applications of algorithms in software engineering.
- Further components are realistic models, algorithm libraries, and a multitude of interrelations to applications. Fig. 1 gives an overview.



Algorithm Engineering

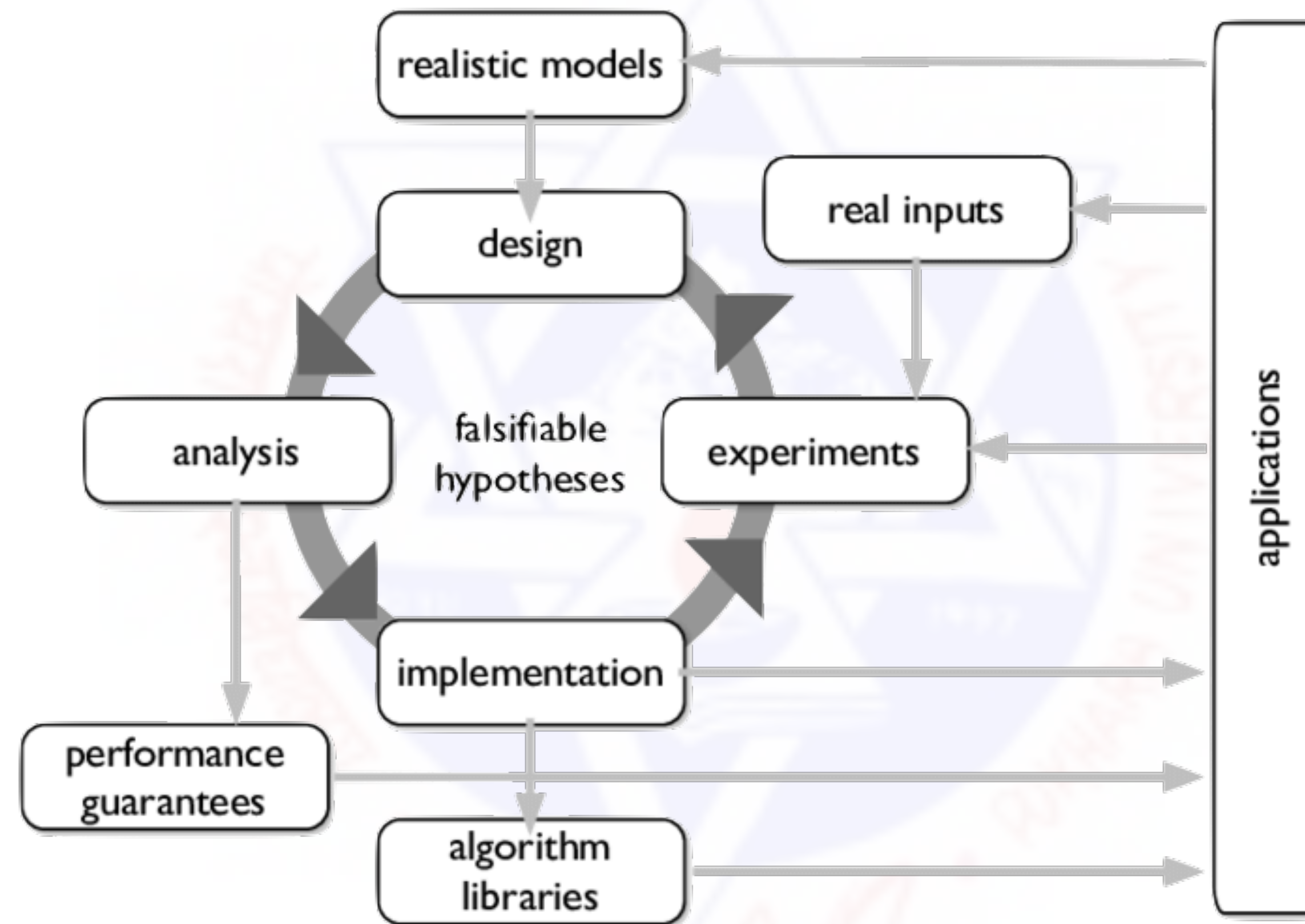


Fig. 1: Algorithm engineering as a cycle of design, analysis, implementation, and experimental evaluation driven by falsifiable hypotheses



THANK YOU